



Eidgenössische Technische Hochschule Zürich  
Swiss Federal Institute of Technology Zurich

# NVSHMEM Analysis and Assessment

Master Thesis

Yijun Ma

April 10, 2025

Advisors: Prof. Dr. Torsten Hoefler, Tiancheng Chen, Siyuan Shen  
Department of Computer Science, ETH Zürich

---

## Abstract

NVSHMEM, the NVIDIA shared memory library, is a partitioned global address space (PGAS) shared memory library for NVIDIA GPU clusters. Despite an official API documentation and programming guide, its internal implementation is not well documented and understood. In this work, we conduct an in-depth analysis and assessment of NVSHMEM from a systems and programming model perspective, particularly in comparison to similar-purpose libraries like NCCL. This covers critical aspects such as its symmetric memory model, one-sided remote communication semantics, and device-side communication support. Furthermore, we will also look into the usage of NVSHMEM in cutting-edge user-level applications and libraries, e.g. DeepEP’s sparse and expert-parallel ML workloads.

---

# Contents

---

|                                                                                     |           |
|-------------------------------------------------------------------------------------|-----------|
| <b>Contents</b>                                                                     | <b>ii</b> |
| <b>1 Introduction</b>                                                               | <b>1</b>  |
| 1.1 Background: The Exascale “Communication Wall” Challenge                         | 1         |
| 1.2 Theoretical Framework: The Evolution of Parallel Communication Models . . . . . | 2         |
| 1.2.1 The Limitations of Two-Sided Messaging in MPI . . . . .                       | 3         |
| 1.2.2 The Partitioned Global Address Space (PGAS) Model                             | 3         |
| 1.2.3 The “GPU-Centric” Paradigm Shift . . . . .                                    | 3         |
| 1.3 NVSHMEM: A High-Level Architectural Overview . . . . .                          | 4         |
| 1.4 Related Works and Comparative Analysis . . . . .                                | 5         |
| 1.4.1 MPI . . . . .                                                                 | 5         |
| 1.4.2 NCCL . . . . .                                                                | 6         |
| 1.4.3 Portable PGAS Libraries (UPC++, GASNet-EX) . . . . .                          | 7         |
| 1.4.4 Vendor-Specific Implementations (ROC_SHMEM, Intel SHMEM) . . . . .            | 7         |
| 1.4.5 Summary . . . . .                                                             | 8         |
| 1.5 Overview and Contributions . . . . .                                            | 8         |
| <b>2 NVSHMEM Overview</b>                                                           | <b>10</b> |
| 2.1 Key Concepts and Terminology . . . . .                                          | 10        |
| 2.1.1 The Symmetric Memory Model . . . . .                                          | 10        |
| 2.1.2 The Dual-Interface Design: Host and Device . . . . .                          | 11        |
| 2.1.3 Transport Layer Abstraction and Hardware Mapping .                            | 12        |
| 2.1.4 Memory Consistency and Ordering Models . . . . .                              | 13        |
| 2.2 API . . . . .                                                                   | 14        |
| 2.2.1 Setup / Exit . . . . .                                                        | 15        |
| 2.2.2 Memory Management . . . . .                                                   | 16        |
| 2.2.3 Team Management . . . . .                                                     | 17        |
| 2.2.4 One-sided Remote Memory Access . . . . .                                      | 18        |

|          |                                                                 |            |
|----------|-----------------------------------------------------------------|------------|
| 2.2.5    | Atomics . . . . .                                               | 20         |
| 2.2.6    | Memory Ordering . . . . .                                       | 20         |
| 2.2.7    | Synchronization . . . . .                                       | 22         |
| 2.2.8    | Collective . . . . .                                            | 24         |
| 2.2.9    | Example NVSHMEM Program . . . . .                               | 25         |
| <b>3</b> | <b>NVSHMEM Source Code Analysis</b>                             | <b>29</b>  |
| 3.1      | Initialization and Bootstrapping . . . . .                      | 29         |
| 3.2      | Team Management . . . . .                                       | 32         |
| 3.3      | Collective Kernel Launch . . . . .                              | 34         |
| 3.4      | Memory Management . . . . .                                     | 35         |
| 3.4.1    | Virtual memory management . . . . .                             | 35         |
| 3.4.2    | Allocation . . . . .                                            | 37         |
| 3.4.3    | Mapping and remote address computation . . . . .                | 40         |
| 3.4.4    | Direct pointers . . . . .                                       | 42         |
| 3.5      | One-sided Remote Memory Access . . . . .                        | 42         |
| 3.5.1    | Fast path: direct GPU memory access . . . . .                   | 42         |
| 3.5.2    | Slow path: IBGDA and proxy execution . . . . .                  | 44         |
| 3.5.3    | Comparison with NCCL's device API . . . . .                     | 48         |
| 3.6      | Point-to-Point Synchronization . . . . .                        | 49         |
| 3.6.1    | <code>nvshmem_signal_wait_until</code> . . . . .                | 49         |
| 3.6.2    | <code>nvshmem_wait_*</code> on generic variables . . . . .      | 50         |
| 3.6.3    | <code>nvshmem_test_*</code> : non-blocking predicates . . . . . | 51         |
| 3.6.4    | Semantics and usage . . . . .                                   | 52         |
| 3.7      | Collective Communication . . . . .                              | 53         |
| 3.7.1    | Broadcast . . . . .                                             | 55         |
| 3.7.2    | Alltoall . . . . .                                              | 57         |
| 3.7.3    | <code>fcollect</code> (Allgather) . . . . .                     | 60         |
| 3.7.4    | Allreduce . . . . .                                             | 62         |
| 3.7.5    | Reduce-scatter . . . . .                                        | 65         |
| 3.7.6    | Barrier and <code>sync</code> . . . . .                         | 67         |
| 3.8      | Memory Ordering . . . . .                                       | 69         |
| <b>4</b> | <b>Benchmarking</b>                                             | <b>71</b>  |
| 4.1      | Simple Benchmarking . . . . .                                   | 71         |
| 4.1.1    | P2P . . . . .                                                   | 71         |
| 4.1.2    | Collective . . . . .                                            | 85         |
| 4.2      | Performance Comparison with NCCL . . . . .                      | 93         |
| 4.2.1    | Collective <code>perftest</code> . . . . .                      | 94         |
| 4.2.2    | Jacobi . . . . .                                                | 95         |
| <b>5</b> | <b>Case Study: DeepEP</b>                                       | <b>100</b> |
| <b>6</b> | <b>Discussion</b>                                               | <b>105</b> |

|          |                                                  |            |
|----------|--------------------------------------------------|------------|
| 6.1      | Potential improvements & optimizations . . . . . | 105        |
| <b>7</b> | <b>Conclusion</b>                                | <b>106</b> |
| <b>8</b> | <b>Acknowledgements</b>                          | <b>107</b> |
|          | <b>Bibliography</b>                              | <b>108</b> |
| <b>A</b> | <b>nvshmem_init() Pseudocode</b>                 | <b>112</b> |

# Introduction

---

### 1.1 Background: The Exascale “Communication Wall” Challenge

Present-day high performance computing (HPC) workloads utilize vast parallelism to achieve order-of-magnitude increase in computational performance. From CPU-dominated manycore systems, to accelerator-assisted heterogeneous HPC architectures, to present day’s more common single instruction multiple thread (SIMT) GPU-based architectures, architecture revolution has brought computational performance from petascale to exascale, featured by systems such as Frontier, Aurora, and the Summit supercomputer at Oak Ridge National Laboratory [1]. Nevertheless, the “communication wall” remains an everlasting challenge.

In traditional HPC paradigms, the Message Passing Interface (MPI) [2] has served as the pervasive standard for inter-process communication. MPI assumes a distributed memory model where distinct processes, typically residing on Central Processing Units (CPUs), exchange data through explicit, two-sided messaging operations (Send/Receive). This model was architecturally compatible with systems where the computation and the network interface were controlled by the same processor. However, in modern GPU-accelerated clusters, the computation has migrated to the device (GPU), while the orchestration of communication has historically remained on the host (CPU). This dichotomy creates a “CPU-centric” communication workflow that is fundamentally at odds with the “GPU-centric” nature of modern computation. [3]

The implications of this mismatch are severe. In a typical legacy workflow, data residing in GPU memory must often be staged through host memory to be transmitted over the network, or at the very least, the control path must traverse the PCIe bus to trigger the Network Interface Card (NIC).

## 1.2. Theoretical Framework: The Evolution of Parallel Communication Models

---

Even with advancements like GPUDirect RDMA [4], which optimizes the data path, control remains with the CPU. This introduces significant latency, serialization overheads, and latency jitter, effectively preventing the GPU from operating as a fully autonomous compute agent. As simulation domains push toward strong scaling limits, where the problem size per GPU shrinks to minimize time-to-solution, the overhead of CPU-initiated communication becomes the dominant factor limiting performance. This is the modern manifestation of Amdahl’s Law applied to communication latency. [5]

To resolve this mismatch, the HPC community has increasingly turned toward the Partitioned Global Address Space (PGAS) model [6]. While traditional MPI relies on two-sided communication, requiring both a sender and a receiver to explicitly coordinate a handshake via their CPUs, PGAS employs one-sided communication (Put/Get), which is the key enabler for GPU autonomy. Within this landscape, NVSHMEM (NVIDIA Shared Memory) has emerged as a critical innovation. It is not merely a library but a runtime environment that implements the OpenSHMEM standard specifically optimized for the architectural realities of NVIDIA GPU clusters. By leveraging proprietary interconnect technologies such as NVLink and InfiniBand, NVSHMEM enables true “GPU-centric” communication. It allows CUDA kernels to initiate network transfers directly, interleaving computation and communication at the thread-warp level, thereby removing the CPU from the critical path of execution entirely.

On a high level, this chapter provides an exhaustive analysis of the NVSHMEM architecture, its divergence from traditional models like MPI and NCCL, and its implications for the future of parallel programming. We will dissect the memory consistency models required to support this paradigm, evaluate the performance gains observed in seminal works, and situate NVSHMEM within the broader landscape of related works, including UPC++ [7], GASNet-EX [8], and vendor-specific alternatives like ROC\_SHMEM [9].

## 1.2 Theoretical Framework: The Evolution of Parallel Communication Models

To understand the necessity and innovation of NVSHMEM, one must first deconstruct the theoretical frameworks that have governed parallel computing. The shift from two-sided to one-sided communication is not just a syntactic change; it represents a fundamental rethinking of how synchronization and data movement are decoupled.

### 1.2.1 The Limitations of Two-Sided Messaging in MPI

The Message Passing Interface (MPI) is built on the concept of Two-Sided communication. A data transfer requires active participation from both the sender (calling `MPI_Send`) and the receiver (calling `MPI_Recv`).

**Coupling of synchronization and data.** In MPI, the data movement is implicitly coupled with synchronization. The arrival of a message at the receiver signals both that the data is available and that the sender has reached a specific point in execution. While this model is robust, it enforces a "rendezvous" protocol. The receiver must know ahead of time that a message is coming, or it must constantly poll the network fabric.

**The unexpected message queue.** If a message arrives before the matching receive call is posted, the MPI runtime must buffer this message in an "Unexpected Message Queue." Managing this queue incurs significant software overhead, requiring memory allocation, copying, and list traversal. In the context of GPUs, where thousands of threads might generate messages simultaneously, this serialization on the receiver side becomes a catastrophic bottleneck. [10]

### 1.2.2 The Partitioned Global Address Space (PGAS) Model

The PGAS model fundamentally breaks the coupling inherent in MPI. It presents a unified address space to the programmer, logically partitioned among the processing elements (PEs). In NVSHMEM, a PE is an OS process that is mapped to a GPU context.

**One-sided communication.** PGAS relies on one-sided primitives: `put` (write to remote memory) and `get` (read from remote memory). The initiator specifies the source, destination, and location, and the operation completes without the explicit involvement of the target PE.

**Decoupling.** Because the target PE does not need to call a "Receive" function, the initiator can push data whenever it is ready. This allows for asynchronous progress and "zero-copy" protocols where data is written directly into the destination's final memory buffer, bypassing intermediate staging areas.

### 1.2.3 The "GPU-Centric" Paradigm Shift

The application of PGAS to GPUs, which we term "GPU-Centric Communication", addresses the specific performance concerns in communication.

**Launch latency.** Launching a CUDA kernel from the CPU involves a latency of several microseconds. If a simulation step requires "Compute → Communicate → Compute," in traditional MPI-based GPU applications, the CPU must launch a kernel, wait for it to finish, initiate MPI calls, wait for them to



finish, and launch the next kernel. This host-centric execution model exacerbates the impact of kernel launch latency at scale, motivating GPU-initiated communication models such as NVSHMEM. [11]

**Fine-grained parallelism.** GPUs execute using massive thread-level parallelism (TLP). Traditional MPI is coarse-grained (one process per GPU). NVSHMEM allows individual threads or warps to initiate communication, matching the granularity of the communication to the granularity of the computation.

## 1.3 NVSHMEM: A High-Level Architectural Overview

NVSHMEM operationalizes the PGAS model specifically for NVIDIA GPU clusters, addressing the “Communication Wall” by projecting a global address space directly onto the GPU’s memory hierarchy. Unlike traditional models where the CPU manages a distributed memory space, NVSHMEM creates a “symmetric heap” in device memory. This architecture establishes a symmetric memory model where objects exist at the same relative offset across all PEs. By registering this memory with the GPU’s memory management unit and the Network Interface Card (NIC), NVSHMEM enables direct addressing of remote data, allowing a GPU to read or write to a peer’s memory without OS intervention.

The core innovation of NVSHMEM lies in its dual-interface design (host and device interfaces), which exposes communication primitives directly to CUDA kernels. Importantly, the device interface enables “kernel-initiated communication.” This capability allows individual GPU threads or warps to issue one-sided put and get operations directly to the hardware interconnect. This decouples data movement from CPU synchronization, effectively removing the host processor from the critical path of execution. Consequently, applications can interleave computation and communication at a fine granularity, leveraging the GPU’s massive parallelism to hide network latency.

To achieve this, NVSHMEM abstracts the underlying transport layer, automatically selecting the optimal path, such as NVLink for intra-node communication or InfiniBand (via technologies like InfiniBand GPU Direct Async (IBGDA) [12]) for inter-node transfers. This hardware-software co-design ensures that the library creates a unified, high-performance fabric that treats the entire cluster as a single, coherent accelerator. Also, it is important to note that NVSHMEM has a relaxed memory ordering model, which improves performance but relies on explicit synchronization primitives for correctness. This architectural approach not only mitigates the overhead of legacy message-passing protocols but also provides the foundational primitives necessary for today’s “GPU-centric” paradigm.

## 1.4 Related Works and Comparative Analysis

To situate NVSHMEM within the broader landscape of high-performance computing, it is essential to compare it against existing paradigms. NVSHMEM is not a replacement for all communication needs but a specialized tool that fills a specific gap in the problem space.

### 1.4.1 MPI

The Message Passing Interface (MPI) remains the de facto standard for exascale applications, but its legacy as a CPU-centric protocol creates friction in GPU-native workflows.

MPI and NVSHMEM differ in both communication semantics and execution efficiency on GPUs. CUDA-aware MPI uses a two-sided send/receive model that requires CPU involvement for each communication event, introducing host-side latency and potential serialization even when operating on GPU-resident data. NVSHMEM instead adopts a one-sided put/get model that decouples synchronization from data movement and allows GPU threads to initiate communication directly, reducing rendezvous overheads and better matching the GPU execution model.

MPI exposes a private, distributed address space, requiring explicit packing/unpacking of messages, whereas NVSHMEM provides a partitioned global address space that enables direct access to remote data structures, simplifying irregular communication patterns. Latency hiding in MPI depends on explicit overlap of communication and computation using non-blocking operations, while NVSHMEM benefits from intrinsic latency hiding via GPU warp scheduling during remote memory accesses.

Existing research using stencil benchmarks [13] and molecular dynamics codes [14] has shown that while CUDA-aware MPI can efficiently move large contiguous blocks of data, it struggles with fine-grained, irregular communication. In strong scaling regimes where the message size shrinks, the fixed overhead of the MPI software stack (traversing the OS networking stack, even with bypass) becomes a bottleneck. Potluri et al. (2017) [13] demonstrated that NVSHMEM could saturate network bandwidth with significantly smaller message sizes than MPI on Pascal GPUs, primarily due to the elimination of the software stack overhead. Furthermore, MPI's bulk-synchronous nature forces a "lock-step" execution, whereas NVSHMEM supports asynchronous progress where some PEs can proceed while others are communicating [14].

Although NVSHMEM and MPI target similar application domains and can be viewed as competing communication models for distributed memory systems, NVSHMEM's runtime can still leverage MPI as a bootstrap and

control-plane mechanism. In this role, MPI functions not as the primary data-movement API, but as an existing, portable infrastructure for process management and metadata exchange, while NVSHMEM provides its own one-sided, GPU-centric data-plane abstraction.

In addition, NVSHMEM is explicitly designed to coexist with MPI to ease incremental adoption and portability of existing codes. In this mode, an application remains an MPI program (with `MPI_Init`/`MPI_Finalize` and MPI ranks as usual), and NVSHMEM is initialized on top of an existing `MPI_Comm`, typically `MPI_COMM_WORLD`, using `nvshmemx_init_attr`. Each process thus has both an MPI rank and an NVSHMEM PE identifier, allowing developers to introduce NVSHMEM operations (e.g., GPU-initiated one-sided communication) into an otherwise conventional MPI code without rewriting the entire communication layer. This cooperative setup supports hybrid programming patterns where MPI continues to handle aspects such as job launch, global collectives, or inter-job interoperability, while NVSHMEM is gradually employed for performance-critical GPU-resident data movement. As a result, NVSHMEM is not only an alternative to MPI but also a complementary extension that can be incrementally integrated into mature MPI applications to exploit GPU-centric, one-sided communication capabilities.

### 1.4.2 NCCL

NVIDIA Collective Communication Library (NCCL) is often confused with NVSHMEM as both are NVIDIA-specific communication libraries. However, they serve fundamentally different algorithmic patterns and occupy different niches in the software ecosystem.

NCCL is optimized purely for dense collective operations (`AllReduce`, `AllGather`, `Broadcast`). It creates static ring or tree topologies and is tuned to saturate bandwidth for large, contiguous transfers, typical in Deep Learning training (e.g., gradient aggregation). It is largely opaque; the user simply calls `ncclAllReduce` and the library handles the implementation, often utilizing NCCL group aggregated operations to maximize throughput.

In contrast, NVSHMEM is optimized for granularity. NVSHMEM exposes the global address space; it excels at point-to-point, fine-grained, and irregular communication (e.g., graph traversals, sparse matrix operations). While NVSHMEM can do collectives, its primary strength is enabling custom communication patterns that do not fit the rigid NCCL templates. For example, if a GPU needs to send data to a random subset of neighbors based on a runtime calculation (data-dependent communication), NVSHMEM is the only viable choice.

Recent versions suggest a convergence: NCCL v2.28 has introduced NCCL Device API [15] (Load/Store Accessible (LSA) APIs for intranode and GPU-

Initiated Networking (GIN) APIs for internode — more details in section 3.5.3) and begin to expose finer-grained control, while NVSHMEM is adding optimized collectives. However, the distinction remains: use NCCL for bulk synchronous deep learning training; use NVSHMEM for those machine learning or scientific simulation workloads with complex neighbor exchange patterns. (We note that TensorRT-LLM contains internal NVSHMEM-based IPC components in its runtime implementation, suggesting that NVSHMEM can serve as a building block for model-parallel inference frameworks, although this is not documented in public user-facing materials.)

### 1.4.3 Portable PGAS Libraries (UPC++, GASNet-EX)

The PGAS concept predates GPUs, with languages like UPC (Unified Parallel C) and Fortran Coarrays.

UPC++ [7] is a C++ template library built on top of GASNet-EX [8]. GASNet-EX provides the low-level communication substrate. While GASNet-EX has added support for “Memory Kinds” ([16]) to allow the CPU to RDMA data into and out of GPU memory, the control logic remains largely CPU-driven. The CPU must issue the put/get calls. This is a “Host-Initiated” model. [17]

Compared to NVSHMEM, The critical differentiator is the source of the operation. NVSHMEM is unique in allowing the GPU threads themselves to issue the communication. In UPC++, a thread on the host initiates a transfer between GPU A and GPU B. In NVSHMEM, a thread on GPU A initiates the transfer to GPU B. This makes NVSHMEM significantly more performant for workloads where the control flow decisions (what data to fetch) are made on the GPU based on intermediate computation results (e.g., traversing a graph stored in distributed GPU memory).

### 1.4.4 Vendor-Specific Implementations (ROC\_SHMEM, Intel SHMEM)

As the heterogeneous computing landscape diversifies, other vendors have introduced analogous libraries.

ROC\_SHMEM [9] is AMD’s implementation for the ROCm platform. It follows the OpenSHMEM standard closely and aims to provide similar device-side capabilities for AMD GPUs (MI250X, MI300).

Intel SHMEM [18] is Intel’s implementation for OneAPI and Xe GPUs. It supports SYCL and leverages the Level Zero runtime for low-level access.

**Interoperability.** While the concepts are identical, the libraries are currently distinct. However, the naming convention (`nvshmem_`, `roc_shmem_`) distinguishes them, potentially allowing a single hybrid application to link against multiple backend libraries, though this remains an area of active research for portability layers like Kokkos [19] or RAJA [20]. [21]

### 1.4.5 Summary

The following table summarizes the key architectural differences between the major communication libraries discussed in this report.

| Feature            | MPI (CUDA-Aware)              | NCCL                          | NVSHMEM                     | UPC++ / GASNet-EX           |
|--------------------|-------------------------------|-------------------------------|-----------------------------|-----------------------------|
| Primary Model      | Two-Sided (Message Passing)   | Collectives (Group Comm.)     | One-Sided (PGAS)            | One-Sided (PGAS)            |
| Initiation Source  | Host (CPU)                    | Host (CPU) / Device (Limited) | Device (GPU) or Host        | Host (CPU)                  |
| Primary Use Case   | Legacy HPC, Coarse-Grained    | Deep Learning Training        | Irregular HPC, Fine-Grained | Portable HPC                |
| Topology Awareness | Limited (Graph Communicators) | High (Ring/Tree detection)    | High (Team/PE mapping)      | High (GASNet conduits)      |
| Hardware Offload   | GPUDirect RDMA                | LSA / GIN                     | NVLink Load-Store / IBGDA   | RMA Offload (Network)       |
| Synchronization    | Implicit (Message Arrival)    | Implicit (Stream Ordered)     | Explicit (Fences/Quiet)     | Explicit (Futures/Promises) |
| Addressing         | Ranks & Tags                  | Communicators                 | Symmetric Heap Pointers     | Global Pointers             |

**Table 1.1:** Summary of the key architectural differences between the major communication libraries discussed in this section.

This comparative landscape highlights that while there is overlap, each tool is specialized. The successful exascale application of the future will likely be hybrid, employing MPI for process orchestration, NCCL for bulk dense transfers, and NVSHMEM for the fine-grained, latency-critical data movements that define the cutting edge of performance.

## 1.5 Overview and Contributions

The structure of the thesis are organized as follows.

In Chapter 2, we introduce NVSHMEM’s key concepts and architecture in more technical detail, spanning symmetric memory, device-side interfaces, transport layer abstraction, and memory model and consistency. We also present a comprehensive overview of NVSHMEM APIs, along with some example NVSHMEM programs.

In Chapter 3, we analyze NVSHMEM source code in depth. This is a deep dive into NVSHMEM’s key component implementation, including symmetric memory management, one-sided remote memory accesses, fine-grained synchronization approaches, collective operations, and memory ordering enforcement.

In Chapter 4, we present a set of benchmark results, and some insights into

NVSHMEM’s performance characteristics. We executed the full NVSHMEM test suite to analyze both point-to-point and collective operations, evaluating their efficiency across host-side and device-side interfaces in intra-node and inter-node environments. Furthermore, we conducted a comparative analysis against NCCL, testing its collectives as well as a side-by-side performance breakdown of a Jacobi solver implemented in both frameworks.

In Chapter 5, we move one level up and analyze NVSHMEM’s integration in higher-level libraries. We analyzed one example in detail: DeepEP, a communication library for Mixture-of-Experts (MoE) workloads.

Finally, Chapter 6 contains some discussions on NVSHMEM’s potential improvements and optimizations, and Chapter 7 concludes the thesis.

In summary, this thesis makes the following contributions.

- A systematic deconstruction and a comprehensive architectural analysis of NVSHMEM, including API references.
- An in-depth implementation study and white-box analysis of NVSHMEM source code to demystify its critical components.
- An extensive performance evaluation using the complete NVSHMEM test suite, characterizing latency and bandwidth behaviors across both intra-node (NVLink) and inter-node (InfiniBand) topologies; and a head-to-head comparison against NCCL, evaluating collective operation efficiency and the real-world performance of Jacobi solver implementations to represent typical scientific computing workloads.
- An analysis of application integration, specifically a detailed examination of DeepEP, a state-of-the-art communication library for Mixture-of-Experts (MoE) models.

---

# NVSHMEM Overview

---

## 2.1 Key Concepts and Terminology

NVSHMEM is a complex software-hardware co-design that projects a unified memory address space across a cluster of GPUs. It is built upon the OpenSHMEM specification (currently version 1.3 with extensions), but it diverges significantly in its implementation details to accommodate the massive parallelism and relaxed memory consistency models inherent to GPU hardware.

### 2.1.1 The Symmetric Memory Model

The foundational concept of NVSHMEM is symmetric memory. In a parallel job, each PE possesses its own local memory. NVSHMEM partitions this memory into two distinct classes: private memory and shared/symmetric memory.

**Private Data Objects.** These are standard variables allocated in the local memory of a PE (e.g., using `cudaMalloc` or stack variables). They are private to the PE and cannot be accessed by remote PEs via NVSHMEM routines. This follows the standard C memory model for thread-local or process-local storage.

**Symmetric Heap.** During initialization, NVSHMEM reserves a large, contiguous region of GPU device memory known as the symmetric heap. This memory is special: it is pinned (non-pageable) and registered with the GPU's memory management unit (MMU) and the network interface card (NIC). This registration is a prerequisite for remote direct memory access (RDMA), allowing the NIC to read/write this memory without OS intervention.

**Symmetric Data Objects.** These are the remotely accessible objects that form the Global Address Space. Crucially, the "symmetry" implies that if a

variable `X` is allocated in the symmetric heap on PE 0, a corresponding object `X` exists on PE 1, PE 2, and so forth, with the same name, type, and size.

**Symmetric Addressing.** Because of symmetry, the relative offset of an object within the heap is identical across all PEs. This allows NVSHMEM to implement a simplified addressing scheme. A PE can address remote memory using a tuple of `<symmetric_address, destination_PE>`. The library translates this into a physical address or a virtual address mapping depending on the transport.

### 2.1.2 The Dual-Interface Design: Host and Device

A distinguishing feature of NVSHMEM, compared to standard CPU-based OpenSHMEM implementations, is its dual-interface design. It provides APIs for both the host (CPU) and the device (GPU), allowing for a hybrid programming model that can adapt to different phases of an application.

#### Host-Side Interface

The host-side interface serves primarily for initialization, resource management, and coarse-grained orchestration.

- Initialization: `nvshmem_init()` sets up the communication context, allocates the symmetric heap, and establishes connections with peers.
- Memory Allocation: `nvshmem_malloc()` is used to allocate symmetric objects. Unlike `cudaMalloc`, which returns a pointer valid only on the local GPU, `nvshmem_malloc` returns a pointer that represents a global handle, valid for use in NVSHMEM communication routines.
- Stream Integration: NVSHMEM introduces stream-aware APIs (e.g., `nvshmemx_TYPENAME_put_on_stream`). These allow the host to queue communication operations into a CUDA stream. This is vital for hybrid workflows where the CPU manages the control flow but wants to pipeline communication alongside GPU kernel execution asynchronously. This bridges the gap between the CPU-centric and GPU-centric models.

#### Device-Side Interface (`__device__`)

The device-side interface is the transformative component of the architecture. It enables CUDA threads within a kernel to issue communication operations such as `put`, `get`, and `atomic_add` directly to remote GPUs. This capability is fundamental to kernel-initiated communication.

By moving the initiation of communication into the kernel, NVSHMEM allows for the following advantages.



**Fine-grained interleaving.** A warp can compute a partial result, push it to a neighbor’s memory, and immediately resume computation without a global kernel barrier or CPU intervention.

**Latency hiding.** The GPU’s massive thread-level parallelism (TLP) can be leveraged to hide network latency. While one warp waits for a remote memory access to complete, the warp scheduler can switch to other warps that are ready to execute compute instructions. [13]

**Reduced launch overhead.** Eliminating the need to split a simulation step into “Compute Kernel” → “Communication Phase” → “Compute Kernel” removes the latency associated with launching kernels and synchronizing streams via the CPU.

### 2.1.3 Transport Layer Abstraction and Hardware Mapping

NVSHMEM is not a monolithic protocol; it is an abstraction layer that transparently selects the optimal transport mechanism based on the topology of the cluster and the proximity of the target PE.

#### Intra-Node: The Power of NVLink

Within a single compute node (or a tightly coupled system like the NVIDIA GB200 NVL72 rack), GPUs are connected via NVLink, a high-bandwidth, low-latency interconnect that supports load/store semantics.

**Direct load/store.** When two GPUs are connected via NVLink or other P2P-mapping capable transports, NVSHMEM maps the symmetric heap of the remote GPU into the virtual address space of the local GPU using CUDA’s Unified Virtual Addressing (UVA) or similar driver mechanisms. This allows a CUDA thread to access remote data using standard C pointer dereferencing (e.g., `*ptr = value`). This bypasses the network stack entirely, treating remote GPU memory as a NUMA (Non-Uniform Memory Access) node. [22]

**nvshmem\_ptr.** The `nvshmem_ptr` function allows a PE to obtain a local pointer to a remote symmetric object if it is reachable via NVLink. If the target is reachable, the function returns a valid pointer; otherwise, it returns NULL, signaling the program to fall back to explicit network APIs. This allows developers to write “topology-aware” kernels that optimize for local neighbors: user programs can write an if-clause in the user code to take the NVLink path when `nvshmem_ptr` returns non-null and the NIC path otherwise.

#### Inter-Node: InfiniBand and IBGDA

For communication across nodes, NVSHMEM utilizes RDMA interconnects like InfiniBand or RDMA over Converged Ethernet (RoCE).

- GPUDirect RDMA [4]: Historically, NVSHMEM leveraged GPUDirect RDMA to allow the Network Interface Card (NIC) to read/write directly from GPU memory without CPU buffering.
- InfiniBand GPU Direct Async (IBGDA) [12]: A critical advancement in recent versions (NVSHMEM 2.6.0 and later versions) is the support for IBGDA. This technology allows the GPU to directly control the NIC. Instead of the CPU preparing command descriptors (work queue elements) for the NIC, the GPU writes these descriptors directly to the NIC's doorbell register. This is the hardware mechanism that enables true kernel-initiated communication over the network. More generally, IBGDA is an InfiniBand realization of the broader GPU Direct Async Kernel-Initiated (GDA-KI) communication model.
- CPU-Assisted Fallback [23]: In scenarios where hardware or administrative privileges prevent direct GPU-to-NIC control, NVSHMEM 3.0 introduced a CPU-assisted mode for IBGDA, ensuring portability while attempting to maintain asynchronous characteristics. This uses a proxy thread on the host to poll for requests from the GPU, though it incurs higher latency than native IBGDA. However, unlike legacy GPUDirect RDMA without IBGDA, where the CPU must explicitly initiate transfers (often requiring GPU synchronization), this fallback preserves the kernel-initiated programming model; The GPU "fires and forgets" commands to a shared memory buffer, allowing the application to maintain asynchronous logic and code portability even when hardware support for direct doorbell access is unavailable.

### 2.1.4 Memory Consistency and Ordering Models

One of the most complex aspects of NVSHMEM is its relaxed memory consistency model. To extract maximum performance, NVSHMEM does not enforce strict ordering of operations by default. This is a deliberate design choice that differentiates it from CPU-based shared memory models.

#### Weak Ordering

NVSHMEM operates under a weak ordering model.

- No Implicit Ordering: If a thread issues two consecutive writes, e.g. `nvshmem_put(A)` followed by `nvshmem_put(B)`, there is no guarantee that A will arrive at the destination before B. The network fabric may reorder packets, or different paths may be taken.
- Relaxed Fetch: Unlike OpenSHMEM, which requires blocking fetch operations (like `get`) to be ordered, NVSHMEM relaxes this. A subsequent instruction is not guaranteed to see the result of a previous

get immediately without synchronization. This avoids the massive performance penalty of inserting memory barriers after every instruction, allowing the GPU pipeline to remain full.

### Explicit Synchronization: Fences and Quiet

To manage these relaxed ordering semantics, NVSHMEM provides explicit synchronization primitives:

- `nvshmem_fence()`: Ensures ordering of delivery. Calling `fence()` between `put(A)` and `put(B)` guarantees that A is delivered before B. This is crucial for protocols where B acts as a flag indicating that A is valid.
- `nvshmem_quiet()`: Ensures global completion. It blocks the calling thread until all outstanding remote operations initiated by that thread have completed. This is used when a thread needs to ensure data is safely stored remotely before signaling another PE.

They rely on CUDA's `__threadfence_system()` if the system is direct ld/st accessible. However, `__threadfence_system()` does not guarantee completion of remote network transfers.

### Visibility Scopes

On complex hierarchical systems (e.g., NVLink + InfiniBand), visibility guarantees can vary.

**PE-PE Visibility.** Synchronization operations like `nvshmem_quiet` generally only guarantee that updates are visible to the specific PEs involved in the communication.

**Global Visibility.** On systems with only NVLink, visibility might be global due to the cache coherency mechanisms of the interconnect. However, on hybrid systems with both NVLink and InfiniBand, explicit barriers are required to ensure that updates are visible to the entire world. This nuance requires developers to be explicitly aware of the system topology to avoid race conditions.

## 2.2 API

We briefly describe a set of APIs available in NVSHMEM and their basic functionalities in this section. The definitions of the NVSHMEM API set can also be found in NVSHMEM's official documentation. A more detailed analysis of their internal implementations can be found in Chapter 3.

For all APIs listed below, if started with "`[_device_]`", this API is available as a dual interface, i.e. both host-side and device-side.

### 2.2.1 Setup / Exit

```

void nvshmem_init(void);
int nvshmemx_init_attr(unsigned int flags,
    ↪ nvshmemx_init_attr_t *attributes);

int nvshmemx_get_uniqueid(nvshmemx_uniqueid_t *id);
void nvshmemx_set_attr_uniqueid_args(int rank, int nranks,
    ↪ nvshmemx_uniqueid_t *id, nvshmemx_init_attr_t *attr);

```

The library setup API `nvshmem_init` allocates and initializes resources used by the NVSHMEM library. It is a collective operation that all PEs must call before any other NVSHMEM routine may be called. If NVSHMEM fails to initialize, the program will exit with a non-zero value.

Multiple calls to `nvshmem_init` are allowed, and must be called by the same set of processes as the initial call to `nvshmem_init`. At the end of the NVSHMEM program which it initialized, each call to `nvshmem_init` must be matched with a call to `nvshmem_finalize`.

In addition to `nvshmem_init`, the `nvshmemx_init_attr` function is provided to support the easy porting of MPI and OpenSHMEM programs to NVSHMEM; its `flags` parameter specifies bootstrap mode, and `attributes` contain initializer arguments per bootstrap, including MPI communicator handle and Unique ID arguments. NVSHMEM supports launcher agnostic bootstrapping, with support for MPI, PMI, SHMEM, and UID bootstrap modes, and users can also manually specify the bootstrap plugin. (Here, UID stands for Unique ID over network. After calling `nvshmemx_get_uniqueid` on a PE to acquire a unique ID, the user shall be responsible for distributing this unique ID to all PEs, e.g. through environment variable `NVSHMEM_BOOTSTRAP_UID_SESSION_ID`. After this unique ID is set up, all PEs call `nvshmemx_set_attr_uniqueid_args` before collectively calling `nvshmemx_init_attr`.)

The user can specify which remote transport to use with environment variable `NVSHMEM_REMOTE_TRANSPORT`. NVSHMEM supports P2P, InfiniBand Reliable Connection (IBRC) [24], InfiniBand GPUDirect Async (IBGDA) [4], etc. as the transport layer and defaults to IBRC. During the initialization process, NVSHMEM will dynamically link the transport layer library.

```

int nvshmemx_cumodule_init(CUmodule module);
int nvshmemx_hostlib_init_attr(unsigned int flags,
    ↪ nvshmemx_init_attr_t *attributes);

```

NVSHMEM allows dynamic linking of device-side libraries (with the `nvshmemx_cumodule_init` API), and in this case, the user should call a dedicated API `nvshmemx_hostlib_init_attr` to initialize the host-side library only.

```
void nvshmem_finalize(void);

[__device__] void nvshmem_global_exit(int status);
```

The exit function `nvshmem_finalize` is also a collective call and requires all PEs to participate, not just a subset of the PEs. It ends the NVSHMEM portion of a program previously initialized by `nvshmem_init` and releases all resources used by the NVSHMEM library.

NVSHMEM also supports force termination of the entire program by one PE: the `nvshmem_global_exit` API. This API stops the proxy thread and hands over to the bootstrap's global exit function (`PMI_Abort`, `MPI_Abort`, or no-op in UID mode).

### 2.2.2 Memory Management

```
void *nvshmem_malloc(size_t size);
void nvshmem_free(void *ptr);
void *nvshmem_align(size_t alignment, size_t size);
```

NVSHMEM provides a set of APIs for managing the symmetric heap: `nvshmem_malloc`, `nvshmem_free`, and `nvshmem_align`. The APIs allow one to dynamically allocate, deallocate and align symmetric data objects in the symmetric heap, respectively. These routines are collective, i.e. require participation by all PEs. The space is allocated from the symmetric heap (in contrast to the non-NVSHMEM general `malloc` in C, which allocates from the private heap).

When no action is performed, these routines return without performing a barrier. Otherwise, each of these routines includes at least one call to a barrier procedure: `nvshmem_malloc` and `nvshmem_align` call a barrier on exit; and `nvshmem_free` calls a barrier on entry.

NVSHMEM supports both dynamic and static symmetric heap allocation policies. Dynamic symmetric heap allocation is accomplished using the CUDA VMM APIs and is enabled by default. Setting `NVSHMEM_DISABLE_CUDA_VMM` will cause NVSHMEM to use a statically allocated symmetric heap. In this mode, the total size of the symmetric heap is determined at job startup. One can specify the size of the heap using the `NVSHMEM_SYMMETRIC_SIZE` environment variable.

```
int nvshmemx_buffer_register(void *addr, size_t length);
int nvshmemx_buffer_unregister(void *addr);
```

The buffer register APIs, `nvshmemx_buffer_register()` and variants, register the unregistered buffer with the remote transport (either UCX or IB) and with CUDA if the address supplied is unregistered host memory. Once

memory is registered with this function, it is possible to use that memory as the local operand in any host or device atomic memory operations or remote memory access operations (introduced below).

However, it is unsupported to use memory registered with this function as the remote operand. Sending a buffer that is spread over multiple registrations to a single NVSHMEM RMA operation is not supported. Registering NVSHMEM heap memory or CUDA managed memory with this API is not supported (the heap is already registered by default).

### 2.2.3 Team Management

The NVSHMEM Team is a logical set of PEs for communication. The NVSHMEM library predefines the world team `NVSHMEM_TEAM_WORLD` that contains all PEs in the NVSHMEM program; all communication and collective operations operate on the world team by default, i.e., when users call those API versions that do not specify a team argument, the library assumes they are operating on the world team. NVSHMEM also predefines `NVSHMEM_TEAM_SHARED` that corresponds to a team of PEs that share a memory domain and so symmetric heap objects on each PE are directly load/store accessible by all PEs in the team, and `NVSHMEMX_TEAM_NODE` that corresponds to the team that contains all PEs in the same node as the calling PE.

Team-based collective operations include all PEs in a valid team. Point-to-point communication can make use of team-relative PE numbering through PE number translation.

A team may not be used concurrently by multiple threads in the same PE for collective operations ([25]). The rationale is as follows. The collective APIs take team as an argument. If multiple CTAs invoke a collective on the same team simultaneously, then the NVSHMEM runtime will have no way to make a distinction between the two invocations. Besides, each team has internal resources associated with it (e.g., symmetric scratch space) that it uses to execute the collective operation. There is only one set of such resources. Hence at any given time only one collective invocation is allowed on a team.

```

[__device__] int nvshmem_team_my_pe(nvshmem_team_t team);
[__device__] int nvshmem_team_n_pes(nvshmem_team_t team);
int nvshmem_team_translate_pe(nvshmem_team_t src_team, int
↪ src_pe, nvshmem_team_t dest_team);

```

These are the basic routines to query the current PE number in a team, the total number of PEs in a team, and to translate PE numbers between teams.

```

int nvshmem_team_split_strided(nvshmem_team_t parent_team, int
↪ start, int stride, int size, const nvshmem_team_config_t
↪ *config, long config_mask, nvshmem_team_t *new_team);

```

```

int nvshmem_team_split_2d(nvshmem_team_t parent_team, int
↪ xrange, const nvshmem_team_config_t *xaxis_config, long
↪ xaxis_mask, nvshmem_team_t *xaxis_team, const
↪ nvshmem_team_config_t *yaxis_config, long yaxis_mask,
↪ nvshmem_team_t *yaxis_team);

int nvshmemx_team_init(nvshmem_team_t *team,
↪ nvshmem_team_config_t *config, long config_mask, int npes,
↪ int pe_idx_in_team);
int nvshmemx_team_get_uniqueid(nvshmemx_team_uniqueid_t
↪ *uniqueid);

void nvshmem_team_destroy(nvshmem_team_t team);

```

A user program can create new teams arbitrarily or by splitting an existing team. NVSHMEM supports two patterns to split teams: strided and 2D splitting (`nvshmem_team_split_strided()`, `nvshmem_team_split_2d()`). The latter divides a team in x- and y-axis directions, thus benefiting Cartesian decomposition and block communication. There's also a manual approach to create arbitrary new teams by calling `nvshmemx_team_get_uniqueid` and `nvshmemx_team_init`; the root PE should get and send a unique team ID to all PEs of the new team, and every PE should propose its index in the new team (`pe_idx_in_team`) respectively.

The user can also explicitly destroy a team with `nvshmem_team_destroy()`, and otherwise, teams remain valid for the duration of the NVSHMEM portion of an application.

### 2.2.4 One-sided Remote Memory Access

NVSHMEM provides remote memory access (RMA) routines to read or write remote symmetric objects. These operations are one-sided, meaning that the PE invoking an operation provides all communication parameters and the targeted PE is passive. One-sided operation decouples communication from synchronization.

NVSHMEM's RMA routines include both host-side and device-side versions, and the device-side versions can be called in thread/warp/block scope. Moreover, a single-element put/get API (`nvshmem_g`, `nvshmem_p`) is provided for low latency, and a specific method for copying strided data elements (`iput`/`iget` APIs) is provided as a shorthand.

There are 3 types of RMA APIs. The user can specify the type name of the symmetric object and call the typed APIs `TYPENAME_put`, where `TYPENAME` is a standard RMA type as defined in [26, Section *Remote Memory Access*]. Or they can specify the length of the object and call `putSIZE`, where `SIZE` is 8,

16, 32, 64, or 128. There are also untyped versions `putmem`. The source and dest parameters refer to the data objects that an operation reads from and writes to.

```

void
↪ nvshmem_{TYPENAME_put/putmem/put{SIZE}/TYPENAME_get/getmem/get{SIZE}}(
↪ TYPE/void *dest, const TYPE/void *source, size_t
↪ nelems/bytes, int pe);
void
↪ nvshmemx_{TYPENAME_put/putmem/put{SIZE}/TYPENAME_get/getmem/get{SIZE}}
↪ _on_stream(TYPE/void *dest, const TYPE/void *source, size_t
↪ nelems/bytes, int pe, cudaStream_t stream);
__device__ void
↪ nvshmem_{TYPENAME_put/putmem/put{SIZE}/TYPENAME_get/getmem/get{SIZE}}(
↪ TYPE/void *dest, const TYPE/void *source, size_t
↪ nelems/bytes, int pe);
__device__ void
↪ nvshmemx_{TYPENAME_put/putmem/put{SIZE}/TYPENAME_get/getmem/get{SIZE}}
↪ _{block/warp}(TYPE/void *dest, const TYPE/void *source,
↪ size_t nelems/bytes, int pe);

[__device__] void nvshmem_TYPENAME_p(TYPE *dest, TYPE value, int
↪ pe);
[__device__] TYPE nvshmem_TYPENAME_g(const TYPE *source, int
↪ pe);

void
↪ nvshmem_{TYPENAME_iput/iput{SIZE}/TYPENAME_iget/iget{SIZE}}(
↪ TYPE/void *dest, const TYPE/void *source, ptrdiff_t dst,
↪ ptrdiff_t sst, size_t nelems/bytes, int pe);
void
↪ nvshmemx_{TYPENAME_iput/iput{SIZE}/TYPENAME_iget/iget{SIZE}}
↪ _on_stream(TYPE/void *dest, const TYPE/void *source,
↪ ptrdiff_t dst, ptrdiff_t sst, size_t nelems/bytes, int pe,
↪ cudaStream_t stream);
__device__ void
↪ nvshmem_{TYPENAME_iput/iput{SIZE}/TYPENAME_iget/iget{SIZE}}(
↪ TYPE/void *dest, const TYPE/void *source, ptrdiff_t dst,
↪ ptrdiff_t sst, size_t nelems/bytes, int pe);
__device__ void
↪ nvshmemx_{TYPENAME_iput/iput{SIZE}/TYPENAME_iget/iget{SIZE}}
↪ _{block/warp}(TYPE/void *dest, const TYPE/void *source,
↪ ptrdiff_t dst, ptrdiff_t sst, size_t nelems/bytes, int pe);

```



SIZE = 8, 16, 32, 64, 128

Note that on-stream variants do not have a `__device__` version API.

### 2.2.5 Atomics

NVSHMEM's atomic routines guarantee that concurrent accesses by any of these routines, and optionally together with a wait or test operation from the target PE, to the same location and using the same datatype, will be exclusive. When atomic and non-atomic NVSHMEM operations are used to access the same location concurrently, or when NVSHMEM atomic operations and non-NVSHMEM operations (e.g., load and store operations) are used to access the same location concurrently, the behavior is undefined.

NVSHMEM's atomic memory operations (AMOs) are one-sided and only for symmetric objects. AMOs are only supported as device operations for remote transports (i.e. UCX, IBRC). Host side AMOs are only supported for NVLink connected PEs.

```
__device__ TYPE nvshmem_TYPENAME_atomic_fetch_op(TYPE *dest,
↪ TYPE value, int pe);
__device__ void nvshmem_TYPENAME_atomic_op(TYPE *dest, TYPE
↪ value, int pe);
__device__ TYPE nvshmem_TYPENAME_atomic_fetch_inc(TYPE *dest,
↪ int pe);
__device__ void nvshmem_TYPENAME_atomic_inc(TYPE *dest, int pe);
__device__ TYPE nvshmem_TYPENAME_atomic_compare_swap(TYPE *dest,
↪ TYPE cond, TYPE value, int pe);
__device__ TYPE nvshmem_TYPENAME_atomic_swap(TYPE *dest, TYPE
↪ value, int pe);
op = add, and, or, xor.
```

Here, TYPENAME is also a standard RMA type. The fetching routines return after the data has been fetched from the target PE and delivered to the calling PE. The non-fetching routines issue the atomic operation and may return before the operation executes on the target PE, and the quiet or barrier routines can be used to force completion for these non-fetching atomic routines; these will be introduced in the next subsection.

### 2.2.6 Memory Ordering

NVSHMEM's RMA APIs do not enforce ordering, and unblocking APIs also do not guarantee completion. This section lists NVSHMEM APIs that provide mechanisms to ensure ordering and/or delivery of completion on memory store, blocking, and nonblocking NVSHMEM routines.

```
[__device__] void nvshmem_fence(void);
```

The `nvshmem_fence` routine ensures ordering of delivery of operations on symmetric data objects: all operations on symmetric data objects issued *to a particular PE* prior to the call to `nvshmem_fence` are guaranteed to be delivered before any subsequent operations on symmetric data objects to the same PE. However, `nvshmem_fence` guarantees only order of delivery, not completion.

```
[__device__] void nvshmem_quiet(void);  
void nvshmemx_quiet_on_stream(cudaStream_t stream);
```

The `nvshmem_quiet` function ensures completion and visibility of all previous NVSHMEM memory operations issued by the calling PE. On systems with only NVLink, all operations on symmetric data objects are guaranteed to be complete and visible to all PEs when `nvshmem_quiet` returns; on systems with both NVLink and InfiniBand, visibility is only guaranteed at the destination PE. This quiet function has both CPU-side version and GPU-side version, which complete only communication operations issued from the CPU and the GPU, respectively. However, stream-based CPU-side quiet operation `nvshmemx_quiet_on_stream()` will also take effect on the GPU side; it executes a quiet operation on GPU in stream order.

In an NVSHMEM program with multithreaded PEs, the `nvshmem_quiet` routine can enforce memory store ordering only for the calling thread. This means the use of a synchronization operation provided by the threading package may be required. For example, when using POSIX Threads, the user may call the `pthread_barrier_wait` routine to ensure that all threads have issued their respective memory operations before a thread calls `nvshmem_quiet`, so that those operations are also guaranteed completion.

`nvshmem_quiet` does not have an effect on the ordering between memory accesses issued by the target PE. `nvshmem_wait_until`, `nvshmem_test`, `nvshmem_barrier`, `nvshmem_barrier_all` routines can be called by the target PE to guarantee ordering of its memory accesses — see next subsection.

Table 2.1 lists the operations affected by NVSHMEM memory ordering routines.

| Operations       | Fence          | Quiet |
|------------------|----------------|-------|
| Memory Store     | X              | X     |
| Blocking Put     | X              | X     |
| Blocking Get     |                |       |
| Blocking AMO     | X              | X     |
| Non-blocking Put | X              | X     |
| Non-blocking Get |                | X     |
| Non-blocking AMO | X <sup>1</sup> | X     |

**Table 2.1:** Operations affected by NVSHMEM Memory Ordering routines [26, Section *Memory Ordering*]

### 2.2.7 Synchronization

NVSHMEM has also provided `wait_until` and `test` routines for synchronization between two PEs based on the value of a symmetric data object. These point-to-point synchronization routines can be used to portably ensure that memory access operations observe remote updates in the order enforced by the initiator PE using the memory ordering routines `nvshmem_fence` and `nvshmem_quiet`.

```

void nvshmemx_{TYPENAME}_wait_until_on_stream(TYPE *ivar, int
↪ cmp, TYPE cmp_value, cudaStream_t stream)
__device__ void nvshmem_{TYPENAME}_wait_until(TYPE *ivar, int
↪ cmp, TYPE cmp_value)
__device__ void nvshmem_{TYPENAME}_wait_until_all(TYPE *ivars,
↪ size_t nelems, const int *status, int cmp, TYPE cmp_value)
__device__ size_t nvshmem_{TYPENAME}_wait_until_any(TYPE *ivars,
↪ size_t nelems, const int *status, int cmp, TYPE cmp_value)
__device__ size_t nvshmem_{TYPENAME}_wait_until_some(TYPE
↪ *ivars, size_t nelems, size_t *indices, const int *status,
↪ int cmp, TYPE cmp_value)
__device__ void nvshmem_{TYPENAME}_wait_until_all_vector(TYPE
↪ *ivars, size_t nelems, const int *status, int cmp, TYPE
↪ *cmp_values)
__device__ size_t nvshmem_{TYPENAME}_wait_until_any_vector(TYPE
↪ *ivars, size_t nelems, const int *status, int cmp, TYPE
↪ *cmp_values)
__device__ size_t nvshmem_{TYPENAME}_wait_until_some_vector(TYPE
↪ *ivars, size_t nelems, size_t *indices, const int *status,
↪ int cmp, TYPE *cmp_values)

```

<sup>1</sup>NVSHMEM fence routines does not guarantee order of delivery of values fetched by nonblocking AMO routines.

```

uint64_t nvshmemx_signal_wait_until_on_stream(uint64_t
↪ *sig_addr, int cmp, uint64_t cmp_val, cudaStream_t stream)
__device__ uint64_t nvshmem_signal_wait_until(uint64_t
↪ *sig_addr, int cmp, uint64_t cmp_val)

```

The `wait_until_` operations block until the value contained in the symmetric data object, `ivar`, at the calling PE satisfies the wait condition. The wait condition is specified by the comparison operator `cmp` and comparison value `cmp_value`. The optional status is a mask array where each element corresponds to the respective element in `ivars` and indicates whether the element is excluded from the wait set.

The `wait_until_any` operation returns the index of an element in the `ivars` array that satisfies the wait condition, or `SIZE_MAX` if the wait set is empty. The `wait_until_some`, upon return, fills in the `indices` array the indices of at least one element in the wait set that satisfied the wait condition during the call to `wait_until_some`, and returns the number of indices returned in the `indices` array.

The vector versions `wait_until_{all,any,some}_vector` compare to a vector of `cmp_values` instead of a single `cmp_value`. The signal version `signal_wait_until` compares a signal data object instead of general symmetric data objects.

```

__device__ int nvshmem_{TYPENAME}_test(TYPE *ivar, int cmp, TYPE
↪ cmp_value)
__device__ int nvshmem_{TYPENAME}_test_all(TYPE *ivars, size_t
↪ nelems, const int *status, int cmp, TYPE cmp_value)
__device__ size_t nvshmem_{TYPENAME}_test_any(TYPE *ivars,
↪ size_t nelems, const int *status, int cmp, TYPE cmp_value)
__device__ size_t nvshmem_{TYPENAME}_test_some(TYPE *ivars,
↪ size_t nelems, size_t *indices, const int *status, int cmp,
↪ TYPE cmp_value)

__device__ int nvshmem_{TYPENAME}_test_all_vector(TYPE *ivars,
↪ size_t nelems, const int *status, int cmp, TYPE *cmp_values)
__device__ size_t nvshmem_{TYPENAME}_test_any_vector(TYPE
↪ *ivars, size_t nelems, const int *status, int cmp, TYPE
↪ *cmp_values)
__device__ size_t nvshmem_{TYPENAME}_test_some_vector(TYPE
↪ *ivars, size_t nelems, size_t *indices, const int *status,
↪ int cmp, TYPE *cmp_values)

```

The test operations test the numeric comparison of the symmetric object pointed to by `ivar` with the value `cmp_value` according to the comparison operator `cmp`. Contrary to the `wait_until_` operations, the test operations

do not block and returns immediately the comparison result. For example, `test` and `test_all` return zero if ivars, or all entries in ivars, do not satisfy the test condition.

### 2.2.8 Collective

Collective routines are defined as coordinated communication or synchronization operations performed by a group of PEs. User can specify a team as the scope of participating PEs; the collective routines operate on all PEs by default if not specified. Note that PEs can enter or exit collectives at different times, and so accessing symmetric memory remotely may require additional synchronization.

The following collective communication APIs are defined; they are dual interface, and the device-side versions come in thread-, warp- (`_warp`), and threadblock-scoped (`_block`) variants. Note that the NVSHMEM `fcollect` operation is the PGAS analogue of an MPI `allgather`. Besides, the NVSHMEM memory management APIs are also collective and each imply one or more calls to `nvshmem_barrier_all`; team creation functions are also collective.

```
int nvshmem[x]_barrier[_on_stream](nvshmem_team_t team[,
    ↪ cudaStream_t stream]);
__device__ int nvshmem[x]_barrier[_warp/_block](nvshmem_team_t
    ↪ team);

int nvshmem[x]_barrier_all[_on_stream]([cudaStream_t stream]);
__device__ int nvshmem[x]_barrier_all[_warp/_block]();

int nvshmem[x]_{TYPENAME_alltoall/alltoallmem}[_on_stream](
    ↪ nvshmem_team_t team, TYPE/void *dest, const TYPE/void
    ↪ *source, size_t nelems[, cudaStream_t stream]);
__device__ int
    ↪ nvshmem[x]_{TYPENAME_alltoall/alltoallmem}[_warp/_block](
    ↪ nvshmem_team_t team, TYPE/void *dest, const TYPE/void
    ↪ *source, size_t nelems);

int nvshmem[x]_{TYPENAME_broadcast/broadcastmem}[_on_stream](
    ↪ nvshmem_team_t team, TYPE/void *dest, const TYPE/void
    ↪ *source, size_t nelems, int PE_root[, cudaStream_t stream]);
__device__ int
    ↪ nvshmem[x]_{TYPENAME_broadcast/broadcastmem}[_warp/_block](
    ↪ nvshmem_team_t team, TYPE/void *dest, const TYPE/void
    ↪ *source, size_t nelems, int PE_root);
```

```

int nvshmem[x]_{TYPENAME_fcollect/fcollectmem}[_on_stream](
    ↪ nvshmem_team_t team, TYPE/void *dest, const TYPE/void
    ↪ *source, size_t nelems[, cudaStream_t stream]);
__device__ int
    ↪ nvshmem[x]_{TYPENAME_fcollect/fcollectmem}[_warp/_block](
    ↪ nvshmem_team_t team, TYPE/void *dest, const TYPE/void
    ↪ *source, size_t nelems);

int nvshmem[x]_TYPENAME_{and, or, xor, max, min, sum,
    ↪ prod}_reduce[_on_stream](nvshmem_team_t team, TYPE/void
    ↪ *dest, const TYPE/void *source, size_t nreduce[,
    ↪ cudaStream_t stream]);
__device__ int nvshmem[x]_TYPENAME_{and, or, xor, max, min, sum,
    ↪ prod}_reduce[_warp/_block](nvshmem_team_t team, TYPE/void
    ↪ *dest, const TYPE/void *source, size_t nreduce);

int nvshmemx[x]_TYPENAME_{min, max,
    ↪ sum}_reducescatter[_on_stream](nvshmem_team_t team,
    ↪ TYPE/void *dest, const TYPE/void *source, size_t nreduce[,
    ↪ cudaStream_t stream]);
__device__ int nvshmemx[x]_TYPENAME_{min, max,
    ↪ sum}_reducescatter[_warp/_block](nvshmem_team_t team,
    ↪ TYPE/void *dest, const TYPE/void *source, size_t nreduce);

int nvshmem[x]_sync[_on_stream](nvshmem_team_t team[,
    ↪ cudaStream_t stream]);
__device__ int nvshmem[x]_sync[_warp/_block](nvshmem_team_t
    ↪ team);

int nvshmem[x]_sync_all[_on_stream]([cudaStream_t stream]);
__device__ int nvshmem[x]_sync_all[_warp/_block]();

```

### 2.2.9 Example NVSHMEM Program

In this section, we present and discuss some example NVSHMEM programs from the NVSHMEM documentation. [27]

#### Ring broadcast

```

__global__ void ring_bcast(int *data, size_t nelelem, int root,
    ↪ uint64_t *psync) {
    int mype = nvshmem_my_pe();
    int npes = nvshmem_n_pes();
    int peer = (mype + 1) % npes;

```

```

    if (myype == root)
        *psync = 1;

    nvshmem_signal_wait_until(psync, NVSHMEM_CMP_NE, 0);

    if (myype == npes-1) return;

    nvshmem_int_put(data, data, nelem, peer);
    nvshmem_fence();
    nvshmemx_signal_op(psync, 1, NVSHMEM_SIGNAL_SET, peer);

    *psync = 0;
}

```

In this example, PE 0 broadcasts a message by sending it to PE 1, which sends the message to PE 2 and so on.

This example demonstrates several NVSHMEM APIs, including the use of `nvshmem_fence` to order communication and `nvshmem_signal_wait_until` and `nvshmemx_signal_op` for point-to-point synchronization.

### Ring allreduce

```

__global__ void ring_reduce(int *dst, const int *src, size_t
↪ nreduce, uint64_t *signal, size_t chunk_size) {
    int myype = nvshmem_my_pe();
    int npes = nvshmem_n_pes();
    int peer = (myype + 1) % npes;

    int thread_id = threadIdx.x;
    int num_threads = blockDim.x;
    int num_blocks = gridDim.x;
    int block_idx = blockIdx.x;
    size_t elems_per_block = nreduce / num_blocks;

    // Change src, dst, nreduce, signal to what this block is
    ↪ going to process
    // Each CTA will work independently
    if (elems_per_block * (blockIdx.x + 1) > nreduce) return;
    src = src + block_idx * elems_per_block;
    dst = dst + block_idx * elems_per_block;
    nreduce = elems_per_block;
    signal = signal + block_idx;
}

```

---

```

size_t chunk_elems = chunk_size / sizeof(int);
size_t num_chunks = nreduce / chunk_elems;

// reduce phase
for (size_t chunk = 0; chunk < num_chunks; chunk++) {
    if (mype != 0) {
        if (thread_id == 0)
            ↪ nvshmem_signal_wait_until(signal,
            ↪ NVSHMEM_CMP_GE, chunk + 1);

        __syncthreads();
        for (size_t i = thread_id; i < chunk_elems; i +=
            ↪ num_threads) {
            dst[i] = dst[i] + src[i];
        }
        __syncthreads();
    }
    if (thread_id == 0)
        nvshmem_int_put_signal_nbi(dst, (mype == 0) ? src :
            ↪ dst, chunk_elems, signal, 1, NVSHMEM_SIGNAL_ADD,
            ↪ peer);
    src = src + chunk_elems;
    dst = dst + chunk_elems;
}

// Broadcast phase
dst = dst - num_chunks * chunk_elems;
if (thread_id == 0) {
    for (size_t chunk = 0; chunk < num_chunks; chunk++) {
        if (mype < npes - 1) { // Last pe already has the
            ↪ final result
            nvshmem_signal_wait_until(signal,
            ↪ NVSHMEM_CMP_GE, (mype == 0) ? chunk + 1 :
            ↪ num_chunks + chunk + 1);
        }
        if (mype < npes - 2)
            nvshmem_int_put_signal_nbi(dst, dst,
            ↪ chunk_elems, signal, 1, NVSHMEM_SIGNAL_ADD,
            ↪ peer);
        dst = dst + chunk_elems;
    }
    *signal = 0; // reset for next iteration
}
}

```



```
// launching: nvshmemx_collective_launch((const void  
↪ *)ring_reduce, gridDim, blockDim, args, 0, stream);
```

In this ring allreduce example, during the reduce phase, each PE waits for the incoming chunk from its left neighbor (PE0 starts with its own src chunk), accumulates it locally, and then forwards the updated chunk to its right neighbor. After the final chunk reaches the last PE (and the last PE passes it on to PE0 via the wrap), the broadcast phase sends the reduced chunks around the ring (starting from PE0) so every PE ends with the full reduced result.

The example highlights point-to-point data movement and synchronization using `nvshmem_int_put_signal_nbi` with `nvshmem_signal_wait_until` to coordinate chunk progress. Note that this kernel is intentionally more instructive than performance-tuned: it conservatively uses the single-thread `nvshmem_int_put_signal_nbi` path for clarity, rather than the block/warp-scoped `nvshmemx_*_put_signal_nbi_*` APIs that would better exploit GPU parallelism in a production implementation.

---

# NVSHMEM Source Code Analysis

---

This chapter analyzes NVSHMEM’s source code in detail. We go through each aspect of NVSHMEM functionality and analyze their source code implementations and design choices, covering bootstrap, collective kernel launch, memory management, one-sided remote memory access, peer synchronization primitives, collective communication, and memory ordering.

## 3.1 Initialization and Bootstrapping

This section describes how an NVSHMEM job transitions from an ordinary multi-process CUDA program into a globally coordinated, PGAS-capable system. The focus is the initialization sequence: process discovery, PE numbering, bootstrap abstraction, topology discovery, transport bring-up, and symmetric heap setup. We emphasize invariants and observable semantics so that readers can reason about correctness and performance without needing to read every implementation detail.

### Goals and invariants

- Strong process group: all processes (PEs) agree on group membership, total number of PEs (`npes`), and their own PE number (`mype`).
- PE mapping: each PE has a mapping to node, NUMA domain, and GPU device selection.
- Transport readiness: intra-node and inter-node endpoints are created and exchanged, with peer-access or RDMA enabled as applicable.
- Symmetric heap consistency: all PEs create a heap with compatible size and alignment, with sufficient metadata exchanged to compute remote addresses at constant cost.

- Global liveness check: a final barrier ensures every PE is ready before the first user-visible communication.

**Bootstrap provider abstraction.** NVSHMEM relies on a pluggable bootstrap layer to form the initial process group and exchange small control payloads before high-performance transports are ready. In the official NVSHMEM repo, supported providers include **MPI, PMI/PMI2/PMIx, SHMEM, UID, and user-specifiable bootstrap plugin.**

The user is responsible for selecting the bootstrap backend via the environment variable `NVSHMEM_BOOTSTRAP` (or by specifying the flag parameter for `nvshmemx_init_attr`). If the bootstrap provider is incorrect, NVSHMEM falls back to single-process mode and launches all threads in singletons.

During bootstrap, payload sizes are small; performance is less important than simplicity and robustness. Collective operations in this procedure are hence blocking and failure-intolerant.

The bootstrap provider is separate from runtime data-plane transports; it is torn down after the data plane is ready or retained for debugging.

**PE rank mapping and distribution identification.** In NVSHMEM, a processing element (PE) is an OS process that is mapped to one GPU; since version 2.4.1, NVSHMEM also provides limited support for multiple PEs per GPU. All PEs run the same user program, and collectively call `nvshmem_init` or `nvshmemx_init_attr` for initialization.

After a bootstrap provider is chosen, the job's size (in terms of number of PEs) `npes` and each PE's rank `mype` are fixed. This mapping is done via the launcher (SLURM, `mpirun`, `jsrun`, etc.) and environment variables (e.g., `CUDA_VISIBLE_DEVICES`); most commonly, PE-GPU assignment is done by local rank. NVSHMEM also checks whether there are the same number of PEs on every node.

Furthermore, this step also establishes PE distribution by checking the PEs' host hashes. If a round robin distribution or a block distribution is identified, multiple optimized communication algorithms can be employed.

**CUDA context bring-up.** Before any transport is created, NVSHMEM runtime ensures CUDA context readiness and discovers hardware VMM capabilities.

In detail, the CUDA bring-up procedure `nvshmemi_get_cucontext` selects device, creates or attaches CUDA context, sets up CUDA function table, and establishes the default internal stream (with the greatest priority) for host-side point-to-point and collective communication APIs.

NVSHMEM also queries CUDA attributes, including VMM support and GPUDirect RDMA capability with VMM.

**Symmetric heap initialization.** The symmetric heap is the foundation of NVSHMEM's PGAS model. Here, we only discuss the default vidmem VMM mode, where NVSHMEM implements the symmetric heap on top of CUDA's low-level virtual memory management (VMM) API. Alternative backends using system memory or pinned device memory can be selected via environment options.

Its initialization is done in two phases: first reserve the heap address space, and then setup the heap with remote transports. Support for NVLink SHARP (NVLS, an NVIDIA-proprietary in-network collective acceleration mechanism over NVLink/NVSwitch), and NCCL-based CPU collectives are initialized before proceeding to `reserve_heap()`.

In `reserve_heap()`, the heap's memory granularity is set to CUDA device allocation granularity `cuMemGetAllocationGranularity` (usually 2MB hugepages), capped by the internal constant of maximum IPC handle size `NVSHMEMI_MAX_HANDLE_LENGTH` (default 2 GB) so that the page granularity never exceeds the handle size the transports can register. Users can manually configure the allocation granularity by an environment variable `NVSHMEM_CUMEM_GRANULARITY` (default 512 MB), subject to the capped size. Users may also manually configure the symmetric heap size per PE by an environment variable `NVSHMEM_MAX_MEMORY_PER_GPU` (default 128 GB). Moreover, a first-fit memory allocator is also initialized in this step. More quantitative details are discussed in section 3.4.2.

Later, the `setup_symmetric_heap()` will be called after the transport layer bring-up finishes. It records the remote heap base of P2P-reachable PEs. To support constant-time address translation between PEs (we will explain it in detail in the memory management section, Section 3.4), this step also performs an allgather of peer heap bases.

After this step, we have a global heap table that allows any PE to compute remote addresses and access mapped memory without further control-plane traffic.

**Transport layer bring-up, topology discovery, and proxy thread setup.** With CUDA context and heaps in place, the runtime instantiates the data-plane.

For intranode P2P transports, NVSHMEM records the PCIe IDs/BDF strings of all local GPUs in the initialization sequence, and registers a callback function `nvshmemt_p2p_can_reach_peer` that determines P2P connectivity in the sequence of NVLink - PCIe - host-side fallback (details in Section 3.5.1).

For inter-node RDMA, NVSHMEM first loads and initializes the selected transport backend among direct InfiniBand-based transports (IBRC [24], and IBDEVX, a vendor-specific low-level InfiniBand transport based on NVIDIA Mellanox DevX interfaces), UCX [28], and libfabric [29]; this selection is done via environment variable NVSHMEM\_REMOTE\_TRANSPORT and defaults to IBRC.

Then, NVSHMEM walks the PCI and NUMA topology and selects one or more NICs per node: if NVSHMEM\_ENABLE\_NIC\_PE\_MAPPING is enabled, this allocation is done in a round-robin manner or under the user constraint NVSHMEM\_HCA\_LIST; otherwise (the default case), a PE is assigned to the NIC on the node that is closest to it by distance. After this step, we have a topology matrix that informs algorithm selection and endpoint provisioning.

Finally, it creates and connects endpoints (EPs) to peers. At this point, one-sided put/get and atomics can target any PE's heap.

Moreover, NVSHMEM spins up a host proxy thread per PE whenever any enabled transport requires host-side progress (RDMA completions, staged atomics, etc.). During init, `nvshmemi_proxy_init` allocates the proxy's command queues, launches the thread bound to the chosen CPU core, and hands it a view of the symmetric heap and the array of initialized transports so it can issue network operations on behalf of device kernels. Once running, the proxy polls the transports' completion queues and executes deferred RMA/atomic verbs, allowing GPU kernels to enqueue one-sided operations without blocking on host-side progress paths.

**Final barrier, and error handling.** A final barrier ensures that all PEs have reached a consistent ready state. After this barrier, the device-side runtime can safely issue RMA and collectives using the established descriptors. Note that built-in teams (e.g. NVSHMEM\_TEAM\_WORLD, NVSHMEM\_TEAM\_SHARED) are initialized in this step, including the setup of P2P-reachable PE lists.

NVSHMEM's initialization is failure-intolerant; partial success is rolled back on failure. If any PE fails a mandatory step, every other PE will hit a timeout on the final barrier, and then prints a descriptive message and aborts initialization.

A summary pseudocode of the initialization process is attached in appendix.

## 3.2 Team Management

The team-management code in NVSHMEM is largely about engineering a flexible, topology-aware abstraction around process groups. Internally, a team is a struct that stores the team's membership (`start`, `stride`, `size`, `my_pe`), configuration, and a bidirectional mapping between team ranks and global numbering of PEs ("world" PEs).

NVSHMEM pre-constructs a set of “well-known” built-in teams:

- `NVSHMEM_TEAM_WORLD`: set of all PEs in the NVSHMEM program, analogous to `MPI_COMM_WORLD`. Point-to-point and collective communication APIs that do not specify a team parameter use the world team by default.
- `NVSHMEM_TEAM_SHARED`: set of PEs that share a memory domain. More precisely, it contains all PEs that would mutually return non-null for the direct pointer call `nvshmem_ptr` with the local PE, and all symmetric objects on each PE are directly LD/ST accessible by all PEs in the team.
- `NVSHMEM_TEAM_NODE`: set of PEs in the same node as the calling PE.
- `NVSHMEMX_TEAM_SAME_MYPE_NODE`: set of the same-ranked process on each node. Groups together one PE from every node, the PE whose local node rank (`mype_node`) matches the current PE.
- `NVSHMEMI_TEAM_SAME_GPU`: the team that contains all PEs running on the exact same GPU (same host hash and CUDA device UUID). Using this team, collectives can be restricted to PEs sharing a GPU.
- `NVSHMEMI_TEAM_GPU_LEADERS`: a PE joins this team iff it is the first PE in its `NVSHMEMI_TEAM_SAME_GPU` team (`nvshmemi_team_same_gpu->start == mype`). This team has size equal to the number of GPUs and is used for GPU-leader coordination (e.g., limited multi-process GPU support). It is invisible to non-leader PEs, whose `NVSHMEMI_TEAM_GPU_LEADERS` pointer is set to `NULL`.

and it automatically derives internal subteams upon team creation, specifically the node-local or same-MPI-rank-on-node teams, which serve as prerequisites for other subsystems including topology-aware collectives, NVLS setup, and peer-to-peer optimizations.

It supports creating new teams via strided and 2D splits, including a special path for non-strided parents, and it runs a small allreduce across the team to ensure that collective operations like `team_split` return a consistent error code on all PEs. NVSHMEM also supports a manual approach to create arbitrary new teams by calling `nvshmemx_team_get_uniqueid` and `nvshmemx_team_init`.

There are optional hooks to attach NCCL communicators or NVLS resources to teams and to duplicate team resources across multiple cooperative thread arrays (CTAs).

**The psync buffers layout.** Every team has a psync buffer, which stands for “persistent synchronization buffer.” It is a user-allocated device-side symmetric memory array used by collective and some point-to-point operations to coordinate communication and synchronization across PEs. The psync

synchronization memory regions reside in global memory. They are strided to avoid memory contention and false sharing; when multiple teams update their psync variables, striding ensures they do not share cache lines. this is achieved through static offsets and team-specific addressing.

Within each team’s psync region, different operations (e.g., BCAST, REDUCE, ALLTOALL) use hardcoded offsets within their team’s psync region to minimize cross-operation contention within a team. The memory layout is as follows. Note that some operations are double-buffered to avoid needing barriers between uses.

| Section               | Size in longs                                                                                  |
|-----------------------|------------------------------------------------------------------------------------------------|
| SYNC Buffer 1 & 2     | NVSHMEMI_SYNC_SIZE                                                                             |
| REDUCE Buffer 1 & 2   | reduce_scratch_size / (2 * sizeof(long))<br>alternate using rdxn_count % 2                     |
| BCAST Buffer          | NVSHMEMI_BCAST_SYNC_SIZE (= 5 × NVSHMEMI_SYNC_SIZE)                                            |
| FCOLLECT Buffer       | get_fcollect_psync_len_per_team()                                                              |
| ALLTOALL Buffer 1 & 2 | NVSHMEMI_ALLTOALL_SYNC_SIZE (= 0.5 × NVSHMEMI_SYNC_SIZE)<br>alternate using alltoall_count % 2 |
| FCOLLECT.128 Buffer   | get_fcollect_ll128_psync_len_per_team()                                                        |

**Table 3.1:** Buffer layout and sizes (in longs)

where the *SYNC.SIZES* are hardcoded to scale linearly with the number of PEs. Note that the hardcoded values are aligned to 128-byte cache lines. The  $10 * SYNC\_SIZE$  (determined by empirical benchmarking) suggests heavier synchronization for broadcasts (possibly due to tree-based algorithms — might represent # of PEs × tree fanout: e.g., 4608 Summit nodes × 1.5 avg fanout).

```
/* Size in units of long */
#define SYNC_SIZE 27648 /*XXX:Number of GPUs on Summit;
↪ currently O(N), need to be O(1)*/
#define NVSHMEMI_SYNC_SIZE (2 * SYNC_SIZE)
#define NVSHMEMI_BCAST_SYNC_SIZE (10 * SYNC_SIZE)
#define NVSHMEMI_ALLTOALL_SYNC_SIZE SYNC_SIZE
```

And specifically, the LL128 sync size is calculated with a 128-byte granularity (matching GPU cache lines):

```
#define NVSHMEMI_FCOLLECT_LL128_CALC_PSYNC_SIZE(threshold, type)
((threshold + sizeof(uint64_t) - 1) / sizeof(uint64_t)) * sizeof(uint64_t)}
```

### 3.3 Collective Kernel Launch

The collective launch code bridges NVSHMEM’s global, multi-PE view of execution and CUDA’s per-device kernel launch model. The collective launch functions, `nvshmemx_collective_launch` and `nvshmemx_collective_launch`

`_query_gridsize`, wrap CUDA kernel launches with additional logic that enforces consistency and feasibility across all participating PEs. Internally, they use `cudaOccupancyMaxActiveBlocksPerMultiprocessor` together with the device's `multi_processor_count` to compute a grid size that is legal for each GPU, and it explicitly rejects configurations where some PEs cannot support the requested grid. This is crucial for correctness when the kernel itself will perform global NVSHMEM collectives or synchronization.

Another interesting aspect is how NVSHMEM integrates with CUDA's cooperative launch and stream/event mechanisms to orchestrate globally synchronized execution. During initialization (`nvshmemi_setup_collective_launch`), NVSHMEM queries whether the device supports `cudaDevAttrCooperativeLaunch` and creates a dedicated high-priority and non-blocking stream. The launch path then conditionally uses `cudaLaunchCooperativeKernel` if supported, falling back to `cudaLaunchKernel` otherwise, but in both cases it surrounds the launch with event records and `cudaStreamWaitEvent` calls to sequence the user's stream (a passed-in parameter) and the internal collective launch stream. This pattern effectively serializes user-visible work and NVSHMEM's internal collective kernel in a well-defined order on each GPU, while still allowing fine control over priority and asynchrony.

## 3.4 Memory Management

SHMEM follows a single program, multiple data (SPMD) model with a partitioned global address space (PGAS). The programming model exposes *symmetric* memory segments: each PE allocates an object of the same size and type at a symmetric address, and these objects are globally addressable (possibly residing in physically distributed memories).

### 3.4.1 Virtual memory management

By default, NVSHMEM implements the symmetric heap on top of CUDA's low-level virtual memory management (VMM) API; alternative backends using system memory or pinned device memory can be selected via environment options. With VMM enabled, NVSHMEM implements a two-tier memory management scheme that cleanly separates *virtual address reservation* from *physical memory allocation*. The symmetric heap's virtual address range is reserved eagerly during initialization, while physical pages are committed on demand as the application allocates from the heap. This design frees applications from having to compute or over-provision a fixed heap size before calling `nvshmem_init` [30].

**Initialization and virtual address (VA) reservation.** During `nvshmem_init`, NVSHMEM creates a symmetric heap abstraction on each PE. When CUDA VMM is enabled, the class `nvshmemi_symmetric_heap_vidmem_dynamic_vmm`



first determines the memory allocation granularity by calculating the maximum between the return value of calling `cuMemGetAllocationGranularity`, the runtime option `CUMEM_GRANULARITY`, and `NVSHMEMI_MAX_HANDLE_LENGTH`. It then chooses a per-PE heap size

$$\text{heap\_size\_} = \max(\text{MAX\_MEMORY\_PER\_GPU}, \text{internal overhead}),$$

rounded up to the chosen granularity. Here, the “internal overhead” is a small memory for internal bookkeeping structures and coalescing buffers for `g`, `put`, `get` operations. The environment variable `MAX_MEMORY_PER_GPU` defaults to 128 GiB [30], which sets an upper bound on the GPU memory that NVSHMEM may manage for symmetric allocations.

Next, each PE reserves a contiguous VA range of size

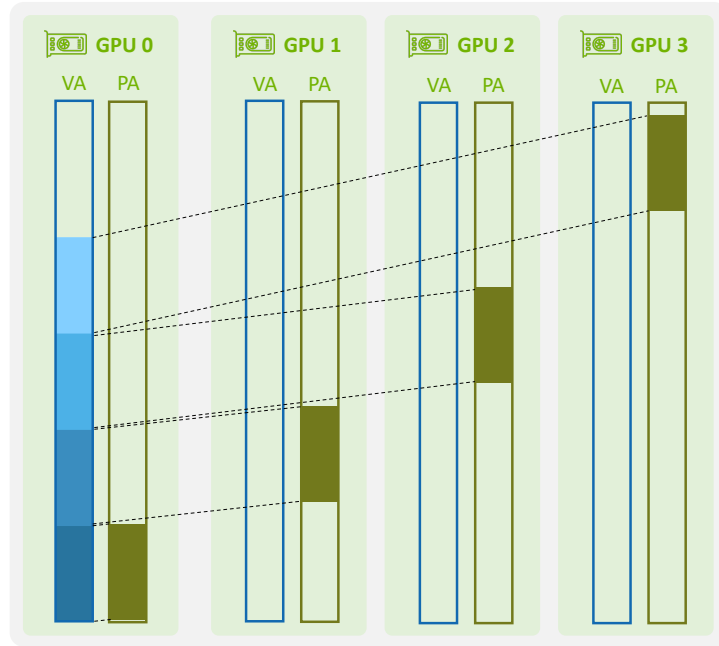
$$\text{p2p\_npes\_} \times \text{heap\_size\_}$$

using `cuMemAddressReserve()`. Here `p2p_npes_` is the number of P2P-reachable GPUs. The first `heap_size_` bytes of this range are used for the local symmetric heap; subsequent segments of the same length are reserved for mapping the symmetric heaps of peer PEs (Figure 3.1). This layout ensures that for each P2P-reachable PE there exists a fixed VA segment of length `heap_size_`, and that all of these segments are disjoint in the local GPU VA space. NVSHMEM then instantiates a host-side heap allocator `mspace` over the local portion of this reserved VA range, but does not yet commit any physical memory.

Lastly, after the transport layer is initialized, PEs allgather every peer’s local heap base address (their heap base in their own VA space) for remote memory access over NIC.

**On-demand physical memory.** NVSHMEM allocates physical GPU memory only when an allocation request cannot be served from already-backed VA. When a symmetric allocation request triggers growth of the heap, `allocate_physical_memory_to_heap` performs the following steps (after rounding the requested size to the CUDA allocation granularity):

1. creates a physical memory handle with `cuMemCreate()`;
2. maps a contiguous VA subrange of the local heap (`heap_base_ + current offset`) to this physical memory using `cuMemMap()`, and installs access permissions for local and remote GPUs via `cuMemSetAccess()`;
3. registers the new region with P2P transports (exports this physical memory handle on each P2P transport) so that peer GPUs can directly map it, and binds the region to NVLS resources for multicat collectives (when enabled);
4. inserts the region into `mspace` map, NVSHMEM’s memory allocator;



**Figure 3.1:** Symmetric heap VMM layout across P2P-reachable GPUs: each PE reserves a contiguous VA window, uses the first VA segment for its local heap, and keeps one equal-sized VA segment per peer for direct mapping, ensuring fixed, disjoint per-peer offsets in the local GPU VA space.

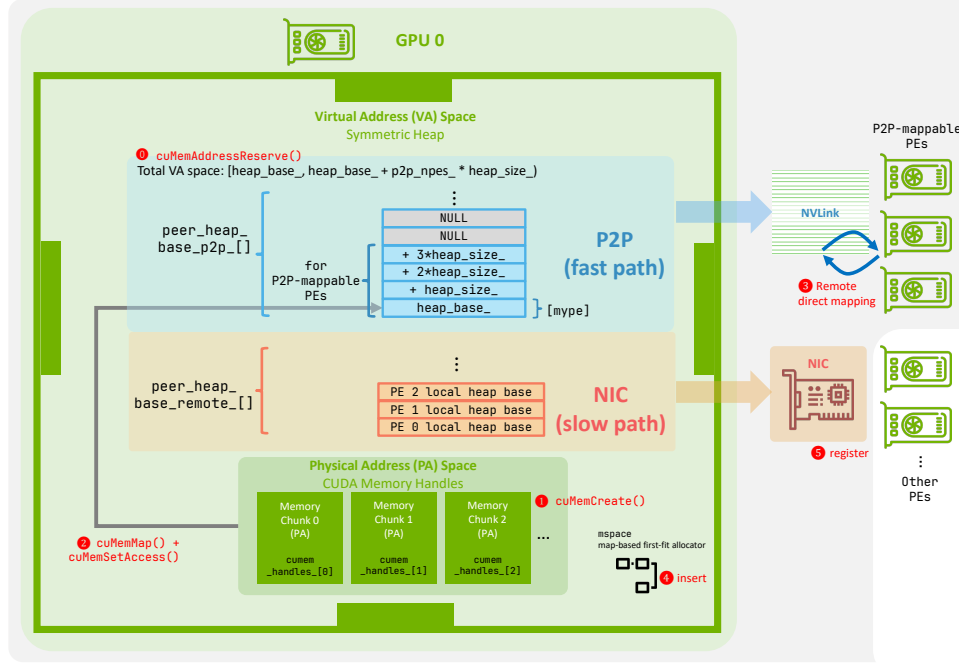
5. registers the region with network transports for remote one-sided operations with non-P2P-reachable peers.

The implementation records each physical allocation in a vector of CUDA memory handles (`cumem_handles_`), together with its offset and size, and broadcasts or gathers these handles so that remote PEs can import and map the same physical memory. After setting up the new region, NVSHMEM performs a barrier to ensure that all PEs have completed the VA and handle setup before any RMA or collective operation can touch the new part of the symmetric heap. Figure 3.2 illustrates this procedure and the resulting address spaces.

### 3.4.2 Allocation

On top of the reserved VA range, NVSHMEM uses a simple host-side allocator, implemented in `mspace` class (in `custom_malloc.cpp`), to manage the symmetric heap. The allocator maintains three `std::map<void*, size_t>` structures:

- a map from chunk start address to size for free chunks;
- a map from chunk end address to size for free chunks;



**Figure 3.2:** NVSHMEM device-memory symmetric heap with CUDA VMM. In each PE's VA window of size  $\text{heap\_size\_} \times \text{p2p\_npes\_}$ , the first segment is the local symmetric heap, each of the next  $(\text{p2p\_npes\_} - 1)$  segments maps to a P2P-reachable peer, and the remaining segments are unused (example: PE0 with peers 1–3). Remote peer heap are unmapped and are accessed via NIC; PEs allgather every peer's local heap base address (their heap base in their own VA space) during initialization, and use it directly for remote memory access over NIC. Physical backing is created lazily by creating new chunks and mapping into the local heap, exporting handles, and updating metadata when allocations grow the heap.

- a map from chunk start address to size for in-use chunks.

This design enables efficient forward and backward coalescing of neighboring free blocks: when a block is freed, the allocator checks whether the preceding or following chunk is free (using the two free maps) and merges them if possible.

**Allocation policy.** The method `mspace::allocate` implements a first-fit policy. It scans the free-chunk map in address order, selects the first block that is at least as large as the requested size, and either splits or consumes the entire block:

- if the chosen block is larger than the request, the allocator returns the front portion and reinserts the remainder as a smaller free block;
- if the block size matches exactly, the block is removed from the free maps.

All sizes are aligned to `NVSHMEMI_MALLOC_ALIGNMENT`. The dual-map book-

keeping makes both splitting and coalescing straightforward. When NVSHMEM serves allocations from this mspace, it simply returns the resulting symmetric address to the user.

**Determinism and complexity.** The allocator’s operations are conceptually  $O(n)$  in the number of free chunks, because it linearly scans the free map. This is a deliberate trade-off: NVSHMEM targets HPC applications in which symmetric allocations are typically concentrated during initialization or infrequent setup phases, and the performance-critical phases are dominated by communication, not dynamic memory management. The simple, deterministic behavior of the allocator is easier to reason about in a PGAS context than more aggressive, adaptive strategies, and avoids many of the subtle fragmentation and tuning issues of general-purpose allocators.

Table 3.2 contrasts this NVSHMEM allocator with mainstream CPU allocators such as `jemalloc`, `tcmalloc`, and `ptmalloc`. In particular, the NVSHMEM allocator uses a single global heap per mspace instance, without per-thread arenas or elaborate binning by size class.

| Feature               | NVSHMEM Simple Allocator                                                     | jemalloc / tcmalloc / ptmalloc                                                                              |
|-----------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Data Structures       | Simple <code>std::map&lt;void*, size_t&gt;</code> for free and in-use chunks | Rich hierarchies: arenas, size classes, radix trees, bins, freelists                                        |
| Allocation Strategy   | First-fit with linear scan of free chunks                                    | Segregated fits within size classes; cache-aware fast paths                                                 |
| Chunk Management      | Explicit splitting on allocation; eager coalescing on free                   | Delayed coalescing, slab allocation, and background maintenance                                             |
| Thread Awareness      | Single global heap per mspace                                                | Multiple <i>arenas</i> for scalable multi-threading                                                         |
| Performance           | Simple, predictable behavior but up to $O(n)$ search                         | Optimized for amortized $O(1)$ allocation on common paths                                                   |
| Fragmentation Control | Basic splitting and coalescing only                                          | Sophisticated heuristics to limit fragmentation                                                             |
| Customizability       | Fixed policy (no user-visible tuning)                                        | Rich configuration knobs, hooks, and profiling support (e.g., <code>jemalloc</code> has extensive tunables) |
| Concurrency Control   | Protected by higher-level NVSHMEM serialization assumptions                  | Lock-free or low-lock fast paths using per-thread caches                                                    |

**Table 3.2:** Comparison of the NVSHMEM heap allocator with general-purpose CPU allocators. In general-purpose allocators, an *arena* is a per-thread or per-core heap used to reduce lock contention and improve scalability [31].

**Threading and reallocation.** The allocator itself is not internally thread-safe; instead, NVSHMEM relies on its thread support level. The reference implementation is built for `NVSHMEM_THREAD_SERIALIZED`, where the application is responsible for ensuring that NVSHMEM routines (and thus heap operations) are not invoked concurrently from multiple threads. This matches the common deployment model where each MPI rank (PE) is largely single-threaded

with respect to NVSHMEM calls.

The allocator also implements `mspace::reallocate`, which may move a block and uses `cudaMemcpy` with device-to-device copies when necessary. However, NVSHMEM does not expose a user-level `realloc` interface; public APIs focus on `nvshmem_malloc`, `nvshmem_calloc`, and `nvshmem_free`. This is consistent with the design philosophy that symmetric memory layouts should be largely static once established.

A build-time option `NVSHMEM_USE_DL_MALLOC` enables the use of Doug Lea's `dlmalloc` in place of the simple `mspace` allocator. This option is disabled by default in the official builds, reflecting a preference for the custom allocator's predictable behavior and tighter integration with NVSHMEM's symmetric heap abstractions.

### 3.4.3 Mapping and remote address computation

As noted above, NVSHMEM reserves a VA range large enough to cover the memory of all P2P-reachable GPUs on the node and then lays out each peer's symmetric heap at a fixed offset within that range. This layout allows NVSHMEM to implement remote addressing as simple base-plus-offset arithmetic, without maintaining distributed metadata for each individual object.

**P2P mapping.** When new physical memory is added to the heap, or when a previously unmapped portion of the heap must be made accessible to peers, NVSHMEM invokes its mapping routines. The high-level sequence is:

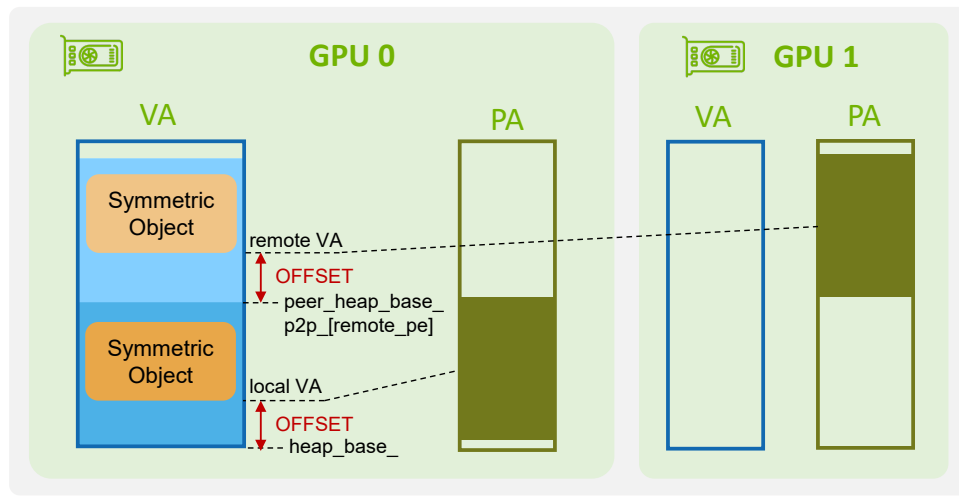
1. For each transport with P2P mapping capability, export the local CUDA memory handle using `cuMemExportToShareableHandle` (wrapped in `export_memory`).
2. Allgather these handles across PEs so that each process learns the handles for every other PE.
3. For each transport, import and map the peer memory handles into the locally reserved VA segments using CUDA VMM's `cudaMemMap`.
4. For non-P2P network transports, register the same region via the appropriate NIC or RDMA registration interface.

**Address computation.** Once peer heaps are mapped, the symmetric addressing scheme becomes particularly simple. Let `heap_base_` be the base address of the local symmetric heap, and let `peer_heap_base_p2p_[pe]` be the base address at which PE `pe`'s symmetric heap is mapped in the local VA space. Given a symmetric pointer `dest_local` on the local PE (for example, returned by `nvshmem_malloc`), the corresponding address in PE `remote_pe`'s

heap is computed as:

$$\text{dest\_remote} = \text{peer\_heap\_base\_p2p\_}[\text{remote\_pe}] + (\text{dest\_local} - \text{heap\_base\_}) \quad (3.1)$$

In other words, NVSHMEM uses the offset of the object within the local heap and applies the same offset to the base address of the remote PE's heap segment (Figure 3.3). This design avoids per-object metadata and enables constant-time computation of remote addresses using only a PE-indexed table of heap bases.



**Figure 3.3:** Symmetric address computation. With peer heaps mapped into fixed-length VA segments, a symmetric pointer on the local PE is translated to a remote address by *preserving its heap offset* and adding the remote peer's heap base. This yields constant-time, metadata-free remote address computation using only the per-PE heap-base table.

**Consistency after heap growth.** Whenever physical memory is added to the symmetric heap, NVSHMEM not only performs the CUDA VMM operations and handle exchange described above, but also updates internal device-side state to keep GPU kernels consistent with the new heap layout. Host-side operations such as allocation, deallocation, and heap growth are followed by `nvshmemi_update_device_state()`, which refreshes device-resident metadata structures. Since heap growth and deallocation are infrequent relative to communication operations, this explicit synchronization cost is acceptable and keeps the device-side view of the heap in sync with the host.

The mapping function employs tolerant error handling. If mapping fails for a transport, that transport's mapping capability is disabled and the function continues with other transports. This provides fault tolerance for heterogeneous GPU configurations.

### 3.4.4 Direct pointers

The VMM-based design described above enables NVSHMEM to expose *direct* pointers to symmetric data resident on remote GPUs: once a peer heap is mapped into the local VA space, the local GPU can issue ordinary loads and stores to the corresponding *remote virtual addresses*, which are transparently translated by CUDA VMM to the appropriate remote physical memory. In subsequent sections we discuss how NVSHMEM leverages this property to implement one-sided RMA and collective operations using direct, pointer-based access patterns on the GPU.

## 3.5 One-sided Remote Memory Access

NVSHMEM implements one-sided RMA operations (put, get, atomic, and signal operations) through two execution paths, selected dynamically based on peer connectivity and memory mapping capabilities:

- a *fast path* that issues direct loads/stores or device-to-device copies; and
- a *slow path* that uses either device-initiated network transfers (IBGDA) or a host proxy thread, used when direct mapping is not available.

Both device- and host-initiated APIs ultimately resolve to one of these paths.

### 3.5.1 Fast path: direct GPU memory access

The fast path is selected when the target PE's symmetric heap is P2P-mapped into the local GPU address space, i.e. when the target PE's heap has a valid entry in the device-side table `peer_heap_base_p2p_`. In this case, the remote address is computed following the pointer arithmetic (Formula 3.1), and the RMA operation reduces to a direct device-side memory access on the computed remote address. This corresponds to GPUDirect P2P / NVLink-style communication and bypasses host-side proxies and NICs entirely. Scalar put and get operations compile to a single device store or load on the direct remote pointer, while bulk operations invoke a threadgroup-wide device `memcpy` to or from that address.

**P2P connectivity and fast-path eligibility.** A crucial precondition for the device-side fast path is that the pair of PEs be considered P2P-connected. This decision is made in the P2P transport's host logic, primarily in `nvshmemt_p2p_can_reach_peer` (`p2p.cpp`). The runtime performs the following checks, with earlier checks take precedence:

- **Multi-node NVLink (MNNVL) [32] connectivity.** If the runtime determines that the peer PE is connected via the MNNVL fabric (using `is_nvl_connected_pe(pe)`), NVSHMEM enables full P2P mapping with load, store, and atomic capabilities. This does not necessarily

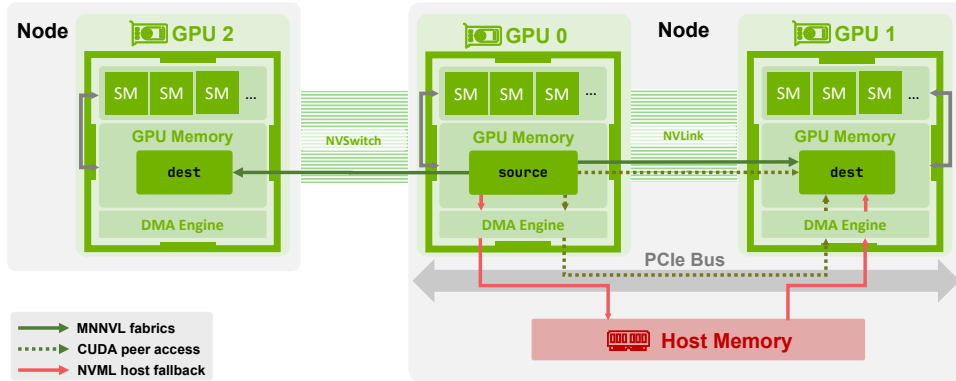
recognize legacy single-node NVLink connections (unless the peer is in the MNNVL fabric clique list); this case can be handled by subsequent checks.

- **Same-host check.** If the local and remote PEs have different hostHash values, they reside on different hosts; in that case P2P is disabled and no fast-path mapping is attempted.
- **Direct PCIe visibility.** If the peer GPU can then be matched to a local CUDA device by PCIe bus/device/function (by scanning the local list of devices), NVSHMEM records the peer CUDA device ID and proceeds.
- **CUDA peer access [33].** For peers that are directly visible and not the same device, NVSHMEM uses `cudaDeviceCanAccessPeer` to check P2P connectivity. This function determines whether the remote peer is P2P-mappable on the local peer depending on the PCIe and/or NVLink topology. If accessible, it sets load/store mapping capabilities, and then uses `cudaDeviceGetP2PAttribute(cudaDevP2PAttrNativeAtomicSupported)` to decide whether GPU-initiated atomics should be enabled.
- **NVIDIA Management Library (NVML) fallback.** If the peer is not directly visible (for example, hidden by `CUDA_VISIBLE_DEVICES`) and either CUDA VMM is disabled or the CUDA driver version is at least 12.0, NVSHMEM falls back to NVML, which utilizes pinned host memory. It queries `nvmlDeviceGetP2PStatus` for read, write, and atomic capabilities between the local and remote devices and sets the mapping flags accordingly.
- **VMM-specific restriction.** If the peer is not directly visible and the heap uses VMM on an older CUDA driver (pre-12.0), NVSHMEM cannot reliably export memory handles. In that case, it prints a warning and leaves P2P disabled for those peers, recommending that users consider disabling CUDA VMM to potentially recover performance.

Only when one of the successful P2P paths above is taken does NVSHMEM map the peer heap and populate `peer_heap_base_p2p_[pe]`, enabling the device-side fast path for that PE. This is represented in Figure 3.4.

**Device-side RMA on the fast path.** For scalar puts and gets, the helpers `nvshmemi_p` and `nvshmemi_g` first check that the target PE has a mapped heap base. After the direct pointer arithmetic, the RMA operation becomes a simple device-side load/store on the remote address. Similarly, the bulk interfaces (e.g., `nvshmemi_put_threadgroup` and `nvshmemi_get_threadgroup`) perform a threadgroup-wide `memcpy` to or from the computed remote address when `peer_heap_base_p2p_[pe]` is available.





**Figure 3.4:** Device-side RMA fast path. When the remote PE's symmetric heap is mapped into the local GPU's virtual address space, NVSHMEM computes the remote address by adding the local heap offset to the peer's heap base and issues direct loads/stores or device-to-device copies.

**Host-side RMA on the fast path.** For host-side RMA, the mechanics differ because the host is orchestrating CUDA copies (and network operations in slow path case) rather than issuing GPU loads/stores directly. The public host APIs (e.g., `nvshmem_put`, `nvshmemx_put_on_stream`) delegate to a common helper `nvshmemi_prepare_and_post_rma` (`putget.cpp`), which encodes the operation. For a given `pe`, NVSHMEM first checks that PE's heap is mapped into the local process, and goes into `nvshmemi_prepare_and_post_mapped_rma`, the P2P-optimized path. This path uses CUDA memcpy operations (`cudaMemcpyAsync`, or `cudaMemcpy2DAsync` for strided regions) between local and remote device pointers (obtained by the direct pointer arithmetic), optionally preceded by `cudaEventRecord` and `cudaStreamWaitEvent` when the operation is non-blocking.

For device-local data within the symmetric heap, NVSHMEM may further optimize using specialized helpers such as `nvshmemi_p2p_rma_optimized`. It uses explicit `cudaMemcpyDeviceToDevice`, `cudaMemcpyHostToDevice`, or `cudaMemcpyDeviceToHost` kinds depending on the operation, instead of `cudaMemcpyDefault` in the general `nvshmemi_p2p_rma_registered`, which lets it skip some runtime inference. Furthermore, `nvshmemi_p2p_rma_optimized` specializes the copy behavior based on stream/NBI flags, and it has a specialized single-word p/g path that uses `HostToDevice`/`DeviceToHost` depending on verb.

### 3.5.2 Slow path: IBGDA and proxy execution

The slow path is triggered when the target PE is not P2P-mapped, i.e. when none of the fast-path eligibility checks above succeed. For example, inter-node NIC communication, or with PEs that are not CUDA P2P-accessible.

For the slow path, NVSHMEM adopts IBGDA if supported, where the

GPU directly controls the NIC; or uses CPU proxy thread to execute this RMA request. Both cases are zero-copy, i.e. data does not go through an intermediate buffer.

**Network transport and connection management.** Slow-path RMA relies on RDMA transports, which raises the question of how NVSHMEM manages network connections at scale, given that NVSHMEM is generally meant for kernels where latencies can become critical.

Currently, NVSHMEM maintains all-to-all connectivity between PEs through RC queue pairs (QPs) (RC stands for Reliable Connection); according to private communication with NVIDIA, this all-to-all connection has not become a bottleneck in NVSHMEM's use-cases so far. NVSHMEM also supports DC QPs (stands for Dynamic Connection) with IBGDA transport. A DC QP can switch the connection to different endpoints at runtime on demand. So a single DC QP can serve multiple endpoints, thus eliminating the need to maintain all-to-all RDMA connections; although this comes at a cost since every time it switches to a different endpoint a handshake has to be performed.

Note that the latest version of NVSHMEM has introduced QP-specific communication APIs to users [26, Section *Queue Pair (QP) Specific APIs*], allowing finer control over RDMA QPs.

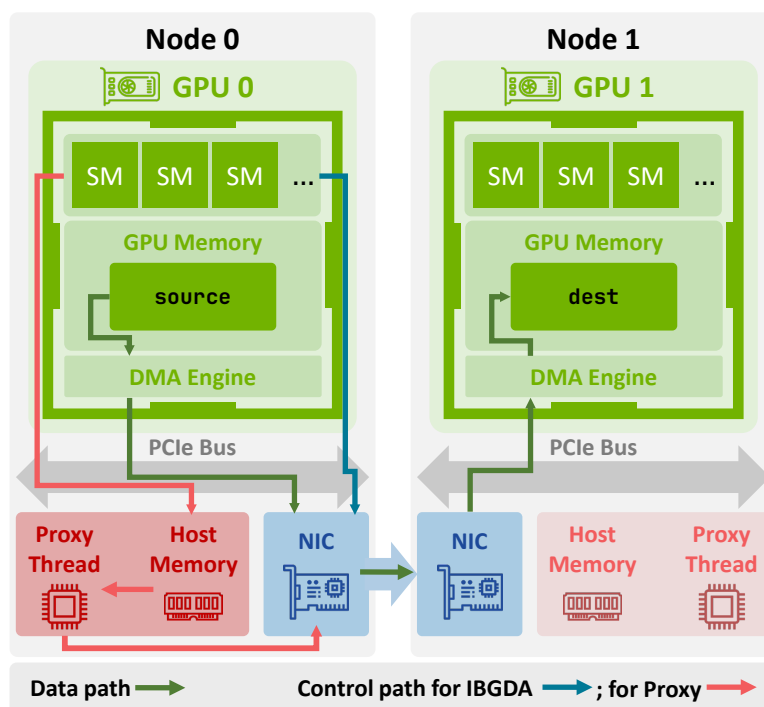
**Device-side RMA on the slow path.** Device-side helpers call into the generic transfer functions in `transfer_device.cu`:

- `nvshmemi_transfer_rma_p` and `nvshmemi_transfer_rma_g` for scalar put/get;
- `nvshmemi_transfer_rma` and `nvshmemi_transfer_rma_nbi` for bulk blocking and non-blocking operations;
- `nvshmemi_transfer_put_signal` and `nvshmemi_transfer_amo_*` for signal and atomic operations.

Each RMA call first checks whether IBGDA is initialized on the device. If IBGDA is available, the operation is offloaded to device-initiated network routines such as `nvshmemi_ibgda_rma_*`, which construct and post RDMA operations directly from GPU code to the NIC. Otherwise, the operation is encoded as a work request and written into a proxy buffer in host-pinned memory, to be executed by a dedicated host proxy thread via functions like `nvshmemi_proxy_rma_p`, `nvshmemi_proxy_rma_g`, and `nvshmemi_proxy_rma_nbi`. Here, a *work request* is a compact descriptor that records all parameters of a logical RMA operation (source and destination addresses, length, PE, and operation type). The proxy or IBGDA code interprets this descriptor and translates it into one or more low-level RDMA operations on the underlying network.

Thus, the slow path is still GPU-initiated at the API level, but completion is driven either by the NIC (IBGDA) or by a host-side proxy that consumes descriptors from a pinned-memory queue.

Figure 3.5 sketches this slow path.



**Figure 3.5:** Device-side RMA slow path. When direct mapping is unavailable, NVSHMEM routes operations either through IBGDA (device-initiated network transfers) or through a host proxy thread that reads RMA descriptors from a pinned command buffer and issues network operations on behalf of the GPU.

**Blocking vs. non-blocking device operations.** On the fast path (peer heap mapped), both blocking and non-blocking device RMA operations use exactly the same memory access pattern: a ld/st or threadgroup-wide device memcpy from the local buffer to the remote VA. This is because, for the fast path (direct memory copy), the operation is always synchronous from the perspective of the issuing thread, so “blocking” and “non-blocking” are indistinguishable, as it is not possible to delegate this memory operation to another worker thread to make the operation non-blocking.

On the slow path (no P2P mapping), blocking calls use `nvshmemi_transfer_rma` or `nvshmemi_transfer_put_signal`, which for IBGDA issue RMA descriptors and then enforce completion (including appropriate fences) at the given SCOPE, while non-blocking calls use `nvshmemi_transfer_rma_nbi` to enqueue work to IBGDA or the proxy without waiting.

**Host-side RMA on the slow path.** The public host RMA APIs (e.g., `nvshmem_put`, `nvshmemx_put_on_stream`) delegate to a common helper `nvshmemi_prepare_and_post_rma` (`putget.cpp`), which encodes the operation and selects the appropriate transport.

If the peer heap is not mapped (unmapped case), NVSHMEM treats the operation as a remote network transfer. Before it handles the transfer, it first enforces several restrictions: remote strided operations, `g` operations, and `put_signal` on off-stream operations are currently not supported for remote transports and will result in an internal error if attempted.

*Off-stream vs. on-stream.* For unmapped peers, the behavior further depends on whether the operation is off-stream (host-initiated) or on a CUDA stream:

- Off-stream (host-initiated) RMA: NVSHMEM selects a remote transport (IBRC, UCX, or libfabric in the current implementation) with RMA capability for the target PE. For scalar `p` operations, it constructs local and remote memory descriptors (including a remote memory handle obtained via `nvshmemi_get_remote_mem_handle`) and calls the transport's `host_ops.rma` entry point (e.g., `ibv_post_send` in the IBRC transport) using a host-resident queue pair (QP). For bulk puts/gets, it uses `nvshmemi_process_multisend_rma` to post the appropriate RDMA operations.
- On-stream (CUDA stream) RMA: When the API variant targets a CUDA stream, offloading is performed through the proxy infrastructure. The helper encodes the RMA request into a proxy command array and calls `nvshmemi_proxy_rma_launcher` on the given stream. The proxy thread then drives the underlying network operations, similarly to the device-side slow path but initiated from host code.

In both cases, the selection of the actual transport for a given PE is made via `nvshmemi_state->selected_transport_for_rma[pe]`, which is set during NVSHMEM's transport layer initialization procedure according to topology.

*Limitations for remote transports.* The host implementation enforces several limitations when using remote transports (as opposed to P2P-mapped peers):

- remote strided RMA (non-unit `lstride` or `rstride`) is not supported;
- host-side `g` operations to unmapped peers are not implemented;
- `put_signal` to unmapped peers is only supported on-stream, where the operation is encoded and executed via the proxy.

These restrictions reflect the fact that the current transports and proxy protocol are optimized around contiguous puts/gets and device-side signaling, while more complex access patterns and host-side signaling remain confined to the P2P-mapped (fast-path) case.

### 3.5.3 Comparison with NCCL’s device API

Since version 2.28, NCCL has also introduced device-side APIs [34]. NCCL’s device-side APIs are largely similar to NVSHMEM in terms of implementation: both can run forward with GPU-driven progress, and both have a fast-path/slow-path split based on cluster connectivity (though running on different backends). See Table 3.3 for details.

They diverge in their primary abstraction and runtime control flow. NCCL is collective-centric: its host-side communicator setup, algorithm selection, and transport configuration are the organizing structure, and the device-side API exists to feed and compose collective progress (e.g., signaling/counter mechanisms coupled to collective engines and backend pathways). NVSHMEM is PGAS-centric: the symmetric heap and per-PE state are the organizing structure, and the device-side API is the primary programming interface for one-sided operations, with ordering and completion semantics grounded in the PGAS memory model. Despite sharing GDA-KI capability and similar primitive sets, NCCL and NVSHMEM differ in where the device API “fits” in the stack: NCCL’s device calls are in service of collective orchestration, while NVSHMEM’s device calls are the core PGAS interface.

| Dimension                       | NCCL Device API                                                                                                                                                                                                                        | NVSHMEM Device API                                                                                                                                                                                      |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Programming model               | Extension of NCCL with device-initiated communication that fits into NCCL’s collective runtime and communicator model.                                                                                                                 | OpenSHMEM-style PGAS one-sided model with device-callable put/get/atomics.                                                                                                                              |
| Intranode                       | <b>Load/Store Accessible (LSA)</b> : device-initiated load/store over CUDA P2P (NVLink/PCIe).                                                                                                                                          | <b>Direct pointer ld/st or cuMem*</b> : when <code>nvshmem_ptr</code> returns a valid directly-accessible pointer. Goes through MNNVL fabrics, CUDA P2P (NVLink/PCIe), or NVML host memory fallback.    |
| Multicast support               | <b>Multimem</b> APIs: NVLink SHARP hardware multicast.                                                                                                                                                                                 | <b>NVLS</b> ( <code>nvshmemi_mcast.*</code> ) capability utilized for multicast in NVSHMEM internals.                                                                                                   |
| Internode and transport backend | <b>GPU-Initiated Networking (GIN)</b> : device-initiated networking over RDMA. Can utilize <b>GDA-KI</b> when supported, a direct GPU-to-NIC transport via DOCA GPUNetIO; relies on <b>CPU proxy</b> otherwise.                        | Direct mapping via <b>MNNVL</b> possible. Otherwise, <b>IBGDA</b> (if supported) GPU-initiated NIC control (which is also part of GDA-KI), or <b>CPU proxy</b> — zero-copy RDMA datapath in both cases. |
| API hierarchy                   | Distinct modules for different connectivity scopes: LSA / Multimem / GIN. GIN uses a three-layer design: host APIs (comm setup + window registration), device APIs, and pluggable backends with dual backend semantics (GDA-KI/Proxy). | Host-side init and symmetric heap allocation, plus device-side single one-sided communication API; connectivity handled by runtime.                                                                     |
| Memory abstraction              | <b>Collective symmetric memory window</b> managed by NCCL; accessed via NCCL handles/offsets.                                                                                                                                          | <b>User-visible symmetric heap/objects</b> allocated via NVSHMEM APIs.                                                                                                                                  |

**Table 3.3:** Summary of NCCL vs. NVSHMEM device-side APIs.

## 3.6 Point-to-Point Synchronization

NVSHMEM's family of device-side point-to-point synchronization primitives operate on locations in symmetric memory. This consists of blocking `nvshmem_wait_*` routines (including `nvshmem_signal_wait_until`) and non-blocking `nvshmem_test_*` routines. Conceptually, these APIs let GPU threads block or poll on completion conditions encoded in memory locations, enabling fine-grained coordination between PEs without resorting to global collectives.

All of these functions are device-only in the current implementation and can be used on both signal variables (used by NVSHMEM put-with-signal operations) and generic integer variables.

### 3.6.1 `nvshmem_signal_wait_until`

We start with the primitive `nvshmem_signal_wait_until(uint64_t *sig_addr, int cmp, uint64_t cmp_val)`, the canonical blocking wait on a 64-bit signal variable. Its public device implementation is extremely thin:

```
NVSHMEMI_DEVICE_PREFIX NVSHMEMI_DEVICE_ALWAYS_INLINE uint64_t
↪ nvshmem_signal_wait_until(uint64_t *sig_addr, int cmp,
↪ uint64_t cmp_val) {
    nvshmemi_wait_until<uint64_t>(sig_addr, cmp, cmp_val);
    return *sig_addr;
}
```

Here `nvshmemi_wait_until<T>` implements a busy-wait loop on a device-visible address, dispatching to one of several comparison-specific helpers based on the `cmp` parameter value (`NVSHMEM_CMP_EQ`, or `_GE`, etc.). For example, the equality (`_EQ`) case boils down to:

```
template <typename T>
__device__ NVSHMEMI_DEVICE_ALWAYS_INLINE void
nvshmemi_wait_until_equals(volatile T *addr, T val, int caller)
↪ {
    #ifdef NVSHMEM_TIMEOUT_DEVICE_POLLING
        long long int start;
        asm volatile("mov.u64 %0, %%globaltimer;" : "=l"(start));
    #endif
    while (*addr != val) {
        #ifdef NVSHMEM_TIMEOUT_DEVICE_POLLING
            nvshmemi_check_timeout_and_log<T>(start, caller,
            ↪ (uintptr_t)addr, *addr, val);
        #endif
    }
}
```

```
    }  
}
```

If `NVSHMEM_TIMEOUT_DEVICE_POLLING` is enabled at build time, the polling loop periodically checks the GPU global timer and records diagnostic information in a device-resident timeout structure if the wait exceeds a large cycle threshold. This supports deadlock detection and debugging of long waits.

In practice, `nvshmem_signal_wait_until` is used in conjunction with signal-producing operations such as `nvshmem_put_signal_nbi`: a sender updates data and then performs a signal operation; the receiver spins on the signal variable until the desired value or condition is observed, and then proceeds.

### 3.6.2 `nvshmem_wait_*` on generic variables

Beyond signal variables, NVSHMEM exposes a rich set of wait primitives on generic integer types. For each supported “wait type” (`int`, `long`, `long long`, `size_t`, `ptrdiff_t`), NVSHMEM defines the following device APIs:

- scalar wait: `nvshmem_<type>_wait_until(ivar, cmp, cmp_value)`,
- set-based waits: `nvshmem_wait_until_all`, `nvshmem_wait_until_any`, `nvshmem_wait_until_some`,
- vector variants where each element has its own comparison value: `nvshmem_wait_until_{all,any,some}_vector`.

All of these ultimately call the generic `nvshmemi_wait_until<T>` implementation from the previous subsection, but they differ in how they organize the wait set:

- Scalar wait (e.g., `nvshmem_int_wait_until`) simply calls `nvshmemi_wait_until` on a single location, and then calls `nvshmemi_transfer_syncapi_update_mem()`, which informs the transfer layer that a synchronization API has completed and may trigger additional cache or view updates so that the waiting thread observes a consistent system state.
- `wait_until_all`: iterates over an array of variables and, for each element that is active in the optional status mask, performs a blocking `nvshmemi_wait_until` until the condition is satisfied. If at least one location was actually waited on, it calls `nvshmemi_transfer_syncapi_update_mem()` once at the end.
- `wait_until_any`: repeatedly scans the array of variables in a rotated order, testing the condition via `nvshmemi_test`. The starting index of the scan is also “rotated” using a shared `test_wait_any_start_idx_ptr` index to balance contention: `unsigned long long start_idx = atomicAdd(nvshmemi_device_state.d.test_wait_any_start_idx_ptr, 1);`

The first index that satisfies the condition is returned; if no location in the wait set satisfies the predicate and the wait set is empty, the function returns `SIZE_MAX`.

- `wait_until_some`: first performs a round of non-blocking tests on the entire array; if zero locations are satisfied but the wait set is non-empty, it falls back to a blocking `wait_until_any`. The indices of all satisfied locations are returned to the caller in an output array.
- Vector variants (`wait_until_all_vector`, `wait_until_any_vector`, `wait_until_some_vector`) generalize these patterns to the case where each watched location has its own comparison value `cmp_values[i]`.

These APIs are the building blocks for higher-level protocols (e.g., tree-based broadcasts) that need to block on a set of flags or counters and are carefully designed to minimize redundant flushing: `nvshmemi_transfer_syncapi_update_mem()` is only invoked when at least one condition is satisfied or at least one location was actually waited on.

### 3.6.3 `nvshmem_test_*`: non-blocking predicates

Complementing the blocking waits, NVSHMEM provides non-blocking test primitives that check whether a condition is currently satisfied without spinning. Their declarations mirror the wait family:

- scalar test: `nvshmem_<type>_test(ivar, cmp, cmp_value)`,
- multi-variable tests: `nvshmem_test_all`, `nvshmem_test_any`, `nvshmem_test_some`,
- vector forms: `nvshmem_test_{all,any,some}_vector`.

At the core, all of these rely on the small helper `nvshmemi_test<T>` in `nvshmemi_common_device.cuh`, dispatching to one of several simple compare-and-return branches based on the `cmp` parameter value:

```
template <typename T>
__device__ NVSHMEMI_DEVICE_ALWAYS_INLINE int
nvshmemi_test(volatile T *ivar, int cmp, T cmp_value) {
    int return_value = 0;
    if (NVSHMEM_CMP_GE == cmp) {
        if (*ivar >= cmp_value) return_value = 1;
    } else if (NVSHMEM_CMP_EQ == cmp) {
        if (*ivar == cmp_value) return_value = 1;
    } else if (NVSHMEM_CMP_NE == cmp) {
        if (*ivar != cmp_value) return_value = 1;
    } else if (NVSHMEM_CMP_GT == cmp) {
        if (*ivar > cmp_value) return_value = 1;
    } else if (NVSHMEM_CMP_LT == cmp) {
```



```
        if (*ivar < cmp_value) return_value = 1;
    } else if (NVSHMEM_CMP_LE == cmp) {
        if (*ivar <= cmp_value) return_value = 1;
    }
    return return_value;
}
```

The scalar device API simply wraps this and, on success, triggers a synchronization update:

```
#define NVSHMEM_TEST(Name, Type) \
    NVSHMEMI_DEVICE_PREFIX NVSHMEMI_DEVICE_ALWAYS_INLINE int \
    nvshmem_##Name##_test( \
        Type *ivar, int cmp, Type cmp_value) { \
        int return_value = nvshmemi_test<Type>(ivar, cmp, cmp_value); \
        if (return_value == 1) nvshmemi_transfer_syncapi_update_mem(); \
        return return_value; \
    }
```

The multi-variable tests also follow the same patterns as the wait family:

- `test_all` returns 1 only if all active locations satisfy the predicate; otherwise 0.
- `test_any` returns the index of the first satisfying location, or `SIZE_MAX` if none do.
- `test_some` fills an index array with all satisfying locations and returns the count.
- Vector forms accept per-element comparison values rather than a single `cmp_value`.

As with waits, `nvshmemi_transfer_syncapi_update_mem()` is called only when at least one condition is satisfied, acting as a hook for the transfer layer to update or flush any state associated with synchronization APIs.

### 3.6.4 Semantics and usage

Together, the wait, test, and signal primitives provide a flexible point-to-point synchronization toolbox on the GPU:

- **Signaling and notification** using `nvshmem_signal_wait_until` on a 64-bit signal variable, typically updated via put-with-signal operations.
- **Flag-based protocols** where one PE updates integer flags or counters in symmetric memory and other PEs block (`wait_*`) or poll (`test_*`) until the desired state is reached.

- **Set-based coordination** across arrays of flags (for example, waiting until all children in a tree-based broadcast have completed, or any of a set of incoming channels has data).
- **RMA-with-signal variation** where data movement and notification are coupled: `put_signal` perform the transfer and then atomically update a remote 64-bit signal, enabling one-sided “send+notify” or “pull+notify” protocols. An example of this is presented above, the ring allreduce kernel in section 2.2.9.

All of these operations are purely device-side: they busy-wait in GPU code on device-visible locations. Correctness with respect to remote writes and atomics is ensured by the interplay between:

- the memory-ordering primitives (`nvshmem_quiet`, `nvshmem_fence`) described in the previous section;
- the transport-layer hooks invoked by `nvshmemi_transfer_syncapi_update_mem()`; and
- optional environment settings such as `ASSERT_ATOMICS_SYNC` that control whether additional flushes are performed on waits at the target.

In this way, NVSHMEM exposes SHMEM-style wait/test semantics to GPU code, while hiding the complexity of polling, timeouts, and transport-specific consistency mechanisms inside its device and transfer layers.

## 3.7 Collective Communication

Collective communication is essential for scalable GPU programs. NVSHMEM implements collective operations by calling its own point-to-point primitives under the hood (albeit with only 1 single CTA, which introduces certain performance constraints - more details in Chapter 4). This section examines these collective primitives in detail. This includes five data collectives: `alltoall`, `bcast`, `fcollect` (allgather), `allreduce`, and `reducescatter`, and two synchronization collectives `barrier`, `sync`.

We only discuss device-side collectives because each of NVSHMEM’s host-side collective either calls into NCCL, or is simply an on-stream version that eventually calls into these device-side collectives. When initiating an on-stream collective operation from the host side, the number of threads in a thread block is determined by calling `cudaOccupancyMaxPotentialBlockSize` on the data type, capped by the number of data elements, and there is only 1 thread block.

For each collective operation, NVSHMEM adaptively selects the algorithm to run based on several factors, including data size, topology, hardware support,

etc. Note that users can always specify the following environment variables and override NVSHMEM’s collective algorithm choice (omitting prefix NVSHMEM\_): `BCAST_ALGO`, `FCOLLECT_ALGO`, `REDUCE_ALGO`, `REDUCESCATTER_ALGO`. Users can also control NVSHMEM’s automatic algorithm selection process with these environment variables (details below): `FCOLLECT_LL_THRESHOLD`, `FCOLLECT_LL128_THRESHOLD`, `REDUCE_NVLS_THRESHOLD`. If the user tries to force an NVLS-based algorithm but the hardware prerequisites are not met, NVSHMEM “legalizes” the request back to legacy non-NVLS algorithms.

An overview of the complexity of all collective algorithms is shown as follows:

| $n$ = data size, $P$ = number of PEs |                              |                                          |                                                     |
|--------------------------------------|------------------------------|------------------------------------------|-----------------------------------------------------|
| NVSHMEM Primitive                    | Algorithm                    | Communication Volume                     | Latency                                             |
| Broadcast                            | Bruteforce put-to-all        | $O(nP)$                                  | $O(P)$                                              |
|                                      | $k$ -ary flat tree           | $O(nP)$                                  | $O(\log_k P)$                                       |
|                                      | Topological-aware tree       | $O(nP)$                                  | $O(\log_k P_{remote} + \log_k P_{intra})$           |
| Alltoall                             | P2P / general                | $O(nP^2)$                                | $O(P)$                                              |
| fcollect / Allgather                 | NVLS one-shot, NVLS LL       | $O(nP^2)$                                | $O(1)$                                              |
|                                      | generic all-push / LL        | $O(nP^2)$                                | $O(P)$                                              |
| Allreduce                            | NVLS one-shot                | $O(nP^2)$                                | $O(1)$                                              |
|                                      | NVLS two-shot                | $O(nP)$                                  | $O(1)$                                              |
|                                      | $k$ -ary recursive exchange  | $O(n \times P \times k \times \log_k P)$ | $O(\log_k P)$                                       |
|                                      | hierarchical fcollect        | $O(nP^2)$                                | $O(1)$ for NVLS-based<br>fcollect, $O(P)$ otherwise |
|                                      | (segmented) linear allreduce | $O(nP^2)$                                | $O(P)$                                              |
| Reduce-scatter                       | NVLS one-shot                | $O(nP^2)$                                | $O(1)$                                              |
|                                      | generic all-push             | $O(nP^2)$                                | $O(P)$                                              |
| Barrier                              | Dissemination                | $O(P \times k \times \log_k P)$          | $O(\log_k P)$                                       |

**Table 3.4:** Communication volume and latency of NVSHMEM collectives.

**The LL (low latency) and LL128 data format.** NVSHMEM has a dedicated data format LL (stands for low latency), and a bandwidth-efficient variant LL128, to avoid unpacking and repacking at each step of the collective algorithms, ensuring efficient communication especially in tree-based communication patterns. The LL protocol does not use explicit synchronization mechanisms such as memory barriers. Instead, it packs data in a specific format where each data element is paired with a synchronization flag:

$$[data_1, flag_1, data_2, flag_2, data_3, flag_3, data_4, flag_4, \dots]$$

The sender side packs 2 data elements + 2 flags into a single 16-byte atomic write. This has also brought a prerequisite that single-element size must be 4 bytes or more.

For the LL128 version, synchronization flags are placed after every 120 bytes of data rather than 4 bytes in regular LL ([120 bytes data + 8 bytes flag]), to better utilize bandwidth under the scenarios where 128-byte atomic write is available. [35]

Next, we will go through every collective operation and first explain how NVSHMEM selects the algorithm for it, then describe and analyze each algorithm in detail.

### 3.7.1 Broadcast

NVSHMEM's broadcast operation supports two class of algorithms:

- **bruteforce one-to-all algorithm:** one root PE directly pushes its data to all other PEs in a flat manner, without using LL; and
- **tree-based broadcast algorithms:** organize the PEs in tree topology, and send the data in two phases, internode before intranode. This includes both LL-based and non-LL tree algorithms.

**Algorithm selection** follows this decision sequence:

- **Message size and type:** If the broadcast operation's psync buffer is too small for the number of elements, or if the size of each element is less than 32 bits, or the number of elements is odd, the bruteforce algorithm is selected. Otherwise, if the total message size exceeds 16 KiB (an empirical threshold), the non-LL flat tree algorithm is selected. This is because the LL format relies on cache line coherency and cannot support small types.
- **Topology:** If none of the above conditions apply, NVSHMEM considers the system topology. If the PEs have a block-shaped distribution, and there are more than 32 PEs, the LL topology-aware tree algorithm is selected; otherwise, NVSHMEM selects the non-topology-aware LL algorithm.

Next, we go over each algorithm in more detail:

**Bruteforce (one-to-all) algorithm:** The simplest algorithm of broadcast is a bruteforce one-to-all. The root PE simply loops over every other PE and sends the data via direct push. All PEs then synchronize with a barrier. This algorithm is used for large data size where psync space is insufficient to run a tree-based broadcast. A fast path via direct pointer memcpy `nvshmem_ptr` is employed if device connectivity supports direct load and store; otherwise, uses `nvshmemi_put_nbi_threadgroup` by default.

**Hierarchical tree algorithm:** Two-level tree, first across nodes, then within nodes. First, the root PE does an inter-node all-to-all data transfer; every thread handles one child, to spread work across threads for distributed sending on bandwidth-limited remote networks. Then, every node does intra-node data transfer, hence completing the broadcast operation; at this level, every thread handles every child, which can utilize full local memory bandwidth by coordinating threads.

A reset on broadcast's pSync space is required if the latter is depleted.

This algorithm uses LL protocol for efficient transfers. In a broadcast tree, the usage of LL data format is as follows:

- LL Preparation Phase: Root calls `packLL` (source  $\rightarrow$  shared buffer); Non-root PEs call `recvLL` (dest  $\leftarrow$  shared buffer)
- Tree Communication Phase: All PEs call `put_nbi_threadgroup` (shared buffer  $\rightarrow$  children)
- Final Phase: Non-root PEs unpack the LL data from shared buffer to dest.

Here, the “shared buffer” is the pSync buffer initialized in the Team Management section. When its space for broadcast is used up, a barrier is called on the team of the broadcast to synchronize the LL send/recv progress across PEs, and then the buffer is reset and reused.

During each transfer, the current PE uses a non-blocking put operation to transfer the LL-packed data from its pSync buffer to every child PE's pSync buffer at the same symmetric address, whether across node or within node – for the latter, the direct pointer access is adopted internally.

This algorithm is topology-aware, optimized for block PE distributions.

**Topology-unaware tree algorithm:** A  $k$ -ary (binary by default) tree broadcast, not topology-aware. This means that the data is no longer transferred in two phases, inter-node before intra-node. Instead, every node will loop over every child, and calls a non-blocking put on remote ones before intranode ones. Whether a child is remote is determined by a NULL check on the return value of the direct pointer call `nvshmemi_ptr`, and the correctness is guaranteed by the internal ordering of put operations, adding to overheads.

This algorithm still uses the LL protocol for efficient transfers.

**Flat tree, non-LL:** Similar to the previous algorithm, but uses a non-LL approach. For types/sizes not supported by LL. This non-LL approach is based on `nvshmemi_put_signal_threadgroup`; it writes a signal into each remote child's pSync buffer after data movement to that child is done, to handle both synchronization and data movement. Finally, one thread in the threadgroup issues a quiet operation to guarantee correctness.

In short, in tree-based broadcast algorithms, the root PE sends data to its children in a  $k$ -ary tree; each child, in turn, forwards the data to its own children, etc. The tree structure (flat  $k$ -ary vs hierarchical) is determined by environment and team topology. For LL variants, special packing and signaling is used for efficiency.

**Complexity analysis.** In the brute-force put-to-all algorithm, the root sends to every PE and then issues a barrier, so per-PE work is  $O(n)$  for the root

and  $O(1)$  for others, with total network volume  $\Theta(P * n)$  and latency  $\Theta(P)$  (linear fanout).

The flat tree algorithm has a  $k$ -ary tree fanout, so each PE forwards to at most  $k$  children. The number of steps is  $\Theta(\log_k P)$  and each step transmits  $n$  elements, yielding total volume  $\Theta(P * n)$  but latency  $\Theta(\log_k P)$ .

For the hierarchical tree algorithm, we have a two-level tree (inter-node then intra-node), so latency becomes  $\Theta(\log_k P_{node} + \log_k P_{intra})$  while total volume stays  $\Theta(P * n)$ . This only changes constants and hop structure, not the asymptotics.

### 3.7.2 Alltoall

The `alltoall` collective in NVSHMEM implements a dense, flat all-to-all communication pattern: every PE sends a distinct message to every other PE in the team.

**Algorithm selection.** NVSHMEM selects between two algorithms based on the job’s connectivity:

- **P2P-only all-push via GPU LD/ST + remote atomics:** when the job’s connectivity supports direct LD/ST operations (`job_connectivity ≤ NVSHMEMI_JOB_GPU_LDST_REMOTE_ATOMICS`), NVSHMEM uses a pure P2P LD/ST “all-push” algorithm.
- **General network/proxy-based all-push:** otherwise, it uses an approach optimized for mixed P2P and remote transports and for different message sizes.

**P2P-only all-push variant.** In the P2P-only case, each PE iterates over all peers and uses `nvshmemi_ptr` to obtain a direct pointer into the peer’s dest buffer, then performs a threadgroup-wide memcpy via `nvshmemi_memcpy_threadgroup<SCOPE>`. This realizes a true all-push pattern over the NVSHMEM-mapped heaps: every PE writes its own contribution into the appropriate slot of every peer’s receive buffer. A final `nvshmemi_barrier_threadgroup<SCOPE>` enforces completion and ordering.

**General all-push variant: message-size tuning.** The more general `nvshmemi_alltoall_allpush_threadgroup` also follows an all-push pattern (each PE sends to all others) but carefully chooses the **level of intra-GPU parallelism** based on (i) message size, (ii) team size, and (iii) whether the target PE is P2P-mapped:

- Let `msgsize = nelems × sizeof(T)`, and define thresholds `NVSHMEMI_ALLTOALL_SMALL_MSGSIZE = 16` bytes and `NVSHMEMI_ALLTOALL_MEDIUM_MSGSIZE = 16384` bytes, which is an empirical value from NVSHMEM’s previous experiments.

- For each destination PE, the code checks `peer_heap_base_p2p_[next_rank]` to determine whether that PE is P2P-mapped (local / direct LD/ST) or must be reached via the general RMA engine (remote network or proxy).

The data-transfer phase then proceeds in three regimes:

- **Remote targets and very small messages.** For any target where `peer_base_addr == NULL` (i.e., no P2P mapping), as well as for P2P-mapped targets with `msgsize ≤ 16` bytes, the algorithm uses a single-thread scope RMA primitive:
  - remote targets: `nvshmemi_transfer_put_signal<NVSHMEMI_THREADGROUP_THREAD>`, which performs a put+signal on behalf of the issuing thread;
  - small P2P-mapped targets: `nvshmemi_put_nbi_threadgroup<T, NVSHMEMI_THREADGROUP_THREAD>`, which issues a non-blocking put from a single thread.

Conceptually, this approach “breaks rank” with the rest of the thread-group, and confines expensive, high-overhead RMA calls to a single thread per target, which is beneficial when messages are so small that additional parallelism would not amortize the overhead.

- **Medium messages with spare warps (block scope).** When the collective is invoked at BLOCK scope, and the CUDA block has more threads than the team size (so some warps are “free”), and the message size is in the medium range  $16 < \text{msgsize} \leq 16384$  bytes, NVSHMEM reassigns the unused warps to help with data transfer. Specifically:
  - it computes `first_unused_warp = ceil(PE_size / warpSize)` and `num_warps = groupSize / warpSize`;
  - warps with index  $\geq \text{first\_unused\_warp}$  iterate over destination PEs, and for each P2P-mapped peer PE they call `nvshmemi_put_nbi_threadgroup<T, NVSHMEMI_THREADGROUP_WARP>`.

This design exploits otherwise idle warps to increase throughput for medium-sized local transfers, while still avoiding excessive per-message overheads.

- **Large messages or non-spare warps.** For all remaining local P2P-mapped targets with `msgsize > 16` bytes (this also includes the warps with index  $\leq \text{first\_unused\_warp}$ , i.e. the non-spare warps, in the free-warp optimization case), the implementation uses the original SCOPE-level parallelism: `nvshmemi_put_nbi_threadgroup<T, SCOPE>` is called for each P2P-mapped peer. Thus, for large messages it fully leverages the available threadgroup (warp or block) bandwidth.

In summary, the data-transfer phase dynamically chooses among thread-, warp-, and full-SCOPE-level parallelism to balance per-message overheads against achievable bandwidth, while still performing a dense all-push among PEs.

**Synchronization and completion phase.** After all data transfers have been issued, `nvshmemi_alltoall_allpush_threadgroup` enters a synchronization phase to ensure global completion and visibility:

1. A threadgroup-wide synchronization (`nvshmemi_threadgroup_sync<SCOPE>`) ensures that all puts for this phase have been issued.
2. The first thread increments a per-collective work counter `pwrk` using `atomicAdd` and then executes `__threadfence_system()`. This fence publishes its writes and the updated counter to all GPUs and the host.
3. After another threadgroup sync, each thread iterates over all PEs and, for P2P-mapped peers, performs a signal operation via `nvshmemi_signal_op` on `psync` to notify them that its alltoall contributions are ready.
4. After another threadgroup sync, each PE then waits for matching notifications from all other PEs by calling `nvshmemi_wait_until_greater_than_equals` on the `psync` locations corresponding to each peer, until the observed value reaches `*pwrk`, the per-collective work counter.
5. Finally, a transport-aware quiet is issued:
  - in the special BLOCK-scope case with spare warps, a single warp (the first unused warp) calls warp-scope `nvshmemi_transfer_quiet<NVSHMEMI_THREADGROUP_WARP>(false)` to amortize the cost;
  - otherwise, the routine calls the quiet operation with the original calling scope `nvshmemi_transfer_quiet<SCOPE>(false)`.

A last `nvshmemi_threadgroup_sync<SCOPE>` and an increment of `teami->alltoall_count` complete the collective.

Overall, NVSHMEM's alltoall implementation is a direct, flat all-to-all in terms of communication pattern, but internally it combines:

- algorithm selection based on connectivity (pure P2P vs. general transports),
- message-size-aware choice of thread vs. warp vs. block parallelism and special utilization of spare warps, and
- a structured fence-signal-wait-quiet sequence for global completion,

to achieve higher throughput while controlling synchronization and RMA overheads.



**Complexity analysis.** In both algorithms, let  $P$  be the total number of PEs, every PE injects  $O(P)$  transfers and the network sees  $O(nP^2)$  byte volume for an  $n$ -byte payload. When P2P direct access is not available, the total cost is  $P$  remote writes plus  $P$  signal operations and  $P$  waits per PE, but the asymptotic complexity remains the same. Latency-wise, the critical path is one global phase; the latency scales with the per-PE fanout, i.e.,  $\Theta(P)$ , in the sense that each PE must complete  $P - 1$  remote ops and  $P - 1$  waits before the collective finishes.

### 3.7.3 fcollect (Allgather)

The NVSHMEM fcollect operation is the PGAS analogue of an MPI allgather.

**Algorithm selection.** Internally, NVSHMEM precomputes helper flags such as whether LL packing is legal for the configuration and selects an algorithm based on:

- message size in bytes (`nelems * sizeof(T)`, where  $T$  denotes the fcollect datatype);
- datatype properties (e.g., whether  $T$  is 16-bit or at least 32-bit wide);
- divisibility constraints on `nelems` and team size needed by LL layouts;
- availability of NVLink SHARP (NVLS) resources (`nvls_rsc_base_ptr`  $\neq$  NULL) and P2P connectivity.

The selection logic is:

- NVSHMEM prioritizes LL8 when the payload fits within the `FCOLLECT_LL_THRESHOLD` (default 2048 bytes), falls back to LL128 for slightly larger transfers within `FCOLLECT_LL128_THRESHOLD` (default 0 — disabled by default), and only then considers NVLS-specific paths.
- Among the NVLS algorithms, NVSHMEM selects NVLS LL when block scope plus even,  $\geq 32$ -bit chunks allow efficient multicast packing, and plain NVLS one-shot when the payload is 4-byte aligned.
- Otherwise, fall back to a generic P2P one-shot all-push algorithm that works on arbitrary topologies.

User override modes are “legalized” the same way: LL requests without a legal LL configuration drift back to general oneshot, while NVLS modes downgrade to P2P if NVLS scratch is missing or the payload sizes don’t satisfy the alignment requirements.

**All-push one-shot algorithms.** When `FCOLLECT_ONESHOT` is selected, NVSHMEM implements fcollect as an all-push operation: each PE writes its con-

tribution directly into the appropriate slice of dest on every other PE. Two variants are used:

- For jobs where GPUs have direct P2P access (`job_connectivity <= GPU_LDST_REMOTE_ATOMICS`), `fcollect` is realized as a set of direct GPU loads/stores (or device-to-device copies) into P2P-mapped symmetric heaps via `nvshmemi_memcpy_threadgroup`.
- Otherwise, a topology-agnostic fallback is adopted, which uses NVSHMEM RMA (potentially via IBGDA or the proxy) to push data to remote PEs via `nvshmemi_put_nbi_threadgroup` when P2P mapping is unavailable.

This all-push strategy has communication complexity  $O(P)$  per PE for a team of  $P$  PEs, but it is simple and robust, and benefits from the same fast-path mechanisms described earlier for one-sided RMA.

**NVLS-based multicast algorithms.** On systems with NVLS support, `fcollect` can exploit hardware multicast to reduce both latency and software overhead:

- The NVLS one-shot algorithm uses `nvshmemi_mc_ptr` to obtain a multicast-capable address corresponding to the destination buffer, and then calls `nvshmemi_mcast_memcpy_threadgroup` to replicate each PE's contribution across all GPUs in the team in a single pass, followed by a barrier.
- The NVLS LL algorithm combines LL-style packing with NVLS multicast, encoding data into LL blocks in scratch space and then using NVLink SHARP's `multimem` instructions to perform vectorized multicast stores. This is particularly effective for block-scoped collectives on *Hopper-class GPUs* (`__CUDA_ARCH__ >= 900`) where NVLS resources are exposed and all GPUs in the team share an NVLS domain.

These NVLS paths reduce the number of logical messages per PE and offload a large portion of the data movement to the fabric, making `fcollect` effectively bandwidth-bound rather than latency-bound for large messages.

**Low-latency LL8 and LL128 algorithms.** The LL8 and LL128 algorithms target smaller messages within `NVSHMEM_FCOLLECT_LL(128)_THRESHOLD`. They use a logically all-push pattern but operate on compact LL encodings:

- each PE packs its contribution into the corresponding per-team scratch pSync region using `nvshmemi_packLL` or `nvshmemi_packLL128`;
- non-blocking puts (`nvshmemi_put_nbi_threadgroup`) push these packed blocks to all remote PEs (or in a node-safe configuration, only to those that lack direct shared-memory access);
- finally, each PE unpacks the received LL blocks into its dest buffer using `nvshmemi_recvLL` or the corresponding LL128 unpack, distributing work across warps to exploit intra-GPU parallelism.

The LL algorithms trade a modest amount of extra packing/unpacking work for fewer network round trips and better cache behavior, yielding reduced latency for short messages compared to the generic all-push algorithm.

**Complexity analysis.** `fcollect`'s complexity depends on whether NVLS is present. We use  $P$  to denote the total number of PEs and  $n$  to denote the input data size. In NVLS one-shot and NVLS LL algorithms, each PE performs a single multicast write (after an  $O(n)$  LL packing) and then executes a barrier; the per-PE work is  $O(nP)$  and aggregate data injection is  $O(nP^2)$ , and the critical path is  $\Theta(1)$  rounds with latency dominated by one multicast and the barrier. In other algorithms, every PE has to iterate over all peers (latency  $O(P)$ ), so the per-PE work is  $O(nP)$ ; each chunk gets delivered to every peer, so the total data volume is  $O(nP^2)$ .

### 3.7.4 Allreduce

The allreduce collective in NVSHMEM is implemented with several algorithms, including NVLS-based algorithms and legacy software-based ones. They are selected at runtime based on hardware capabilities, data type, reduce operation, and message size.

#### Algorithm selection:

- Message size and hardware support (NVLS support for fast collectives). NVLS is supported if: the hardware supports, and the reduction is a sum on floating-point numbers, and the message length is 4-byte aligned. On this basis, if: the reduction is out-of-place, and the reduction workload is not a single-element half/bfloat (to avoid ld\_reduce vs store races), and the number of data elements do not exceed `NVSHMEM_REDUCE_NVLS_THRESHOLD` (default 2048, environment variable), NVSHMEM prioritizes NVLS one-shot algorithm; otherwise, NVSHMEM tries to select the NVLS two-shot algorithm, but rejects two-shot if it's a small half/bfloat workload whose number of elements was too small or indivisible by the number of PEs and would leave some PEs idle; if both one-shot and two-shot are rejected, or if NVLS is unsupported altogether, it goes into the next step to select among legacy non-NVLS algorithms.
- Build option. If the NVSHMEM instance is built with option `NVSHMEM_GPU_COLL_USE_LDST`, the function immediately dispatches to a direct peer-load reduce.
- Data type and reduce operation (e.g., sum, min, max, etc.). World-team reductions try in order a hierarchical `fcollect`-based reduce (requires block scope, even element counts, element counts not exceeding `NVSHMEM_FCOLLECT_LL_THRESHOLD` (default 2048),  $\geq 32$ -bit elements, and sufficient scratch space), a  $k$ -ary recursive exchange (re-

quires enough scratch space for the configured  $k$ ), or a segmented put-based reduce as the fallback.

**NVLS One-Shot and Two-Shot algorithms.** The two-shot NVLS algorithm splits the reduction into two phases, reduce-scatter and allgather, tailored to the NVLS multicast add hardware. It is for larger data size when the number of elements exceeds `reduce_nvls_threshold` (default 2048). The data buffer is partitioned evenly across the team, with the last GPU taking the remainder, so that every element is assigned to exactly one PE. Each GPU invokes `nvshmemi_add_reduce_mcast_threadroup` with `ONESHOT=0`, meaning its source and destination pointers are converted to multicast-capable addresses; the NVLS fabric performs an atomic add reduce into the destination slice while simultaneously distributing the reduced chunk to every participant. After the push round completes, a threadgroup barrier ensures all segments have been accumulated.

The one-shot algorithm trades for even lower latency on smaller messages, at the cost of redundant computation. Instead of partitioning the data, every GPU feeds the full `nreduce` elements into an NVLS multicast add with `ONESHOT=1`, so the hardware reduces and writes the entire data in a single pass. The downside is redundant computation since every GPU performs the full reduction, but for small payloads the single hardware round trip outweighs the extra arithmetic; hence the runtime only selects this mode when the message size is below `NVSHMEM_REDUCE_NVLS_THRESHOLD`.

**Recursive exchange (recexch) algorithm.** The team is recursively split into groups, and each group exchanges partial results in steps and phases. Each PE sends and receives data from a subset of other PEs in each phase, reducing the total communication volume and steps.

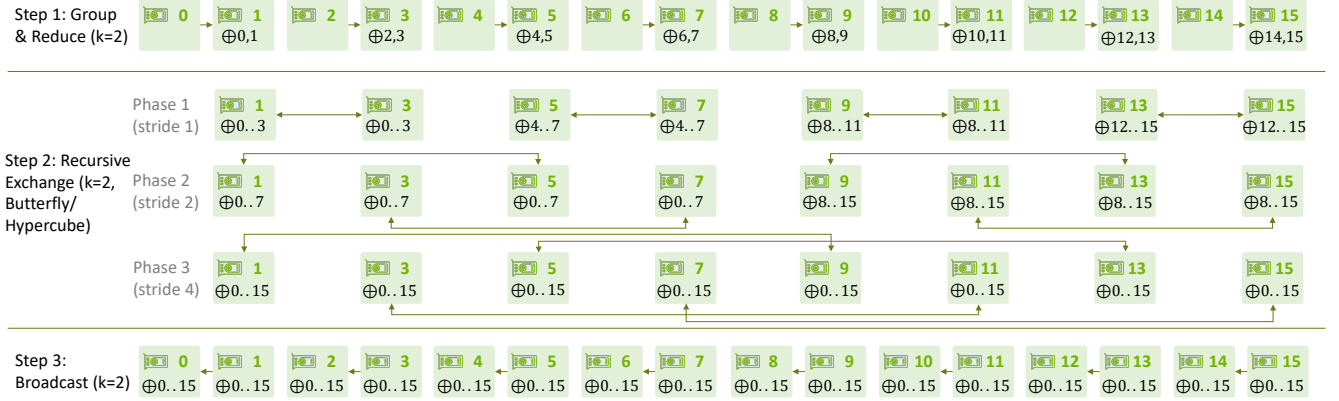
In detail, in the first step, PEs are divided into groups of  $k$  (2 by default) (or less for remainders), and the last PE in each group receives data from the  $k - 1$  other PEs and performs local reduction. Only this last PE in each group will participate in the next step (the *participating* PEs).

In the second step, NVSHMEM uses base- $k$  digit representation to determine communication patterns: in each phase, every *participating* PE communicates with its  $k - 1$  *participating* neighbors by varying one digit of its base- $k$  representation (or equivalently, in increasing power-of- $k$  strides); the number of phases is logarithmic in this step.

In the final step, the final results are distributed back to all PEs; the *participating* PEs send their final results to (and signals) non-*participating* PEs.

Notably, the second step has a communication topology of the " $k$ -ary butterfly" or " $k$ -ary hypercube" pattern. This is illustrated in Figure 3.6.

### 3.7. Collective Communication



**Figure 3.6:** The " $k$ -ary butterfly" or " $k$ -ary hypercube" pattern of allreduce recursive exchange. Here,  $\oplus$  represents the reduce operation, and  $\oplus_{i..j}$  stands for  $i \oplus i+1 \oplus \dots \oplus j$ . In this example figure,  $k = 2$  and there are 16 PEs in total.

The reduction procedure utilizes the pWrk buffer for data exchange and the pSync buffer for signaling. It uses a carefully designed memory layout in pWrk:

$$\begin{aligned}
 &[0 \text{ to } (k-1)n_{\text{reduce}} - 1] && \text{-- Step 1 receive buffers} \\
 &[(k-1)n_{\text{reduce}} \text{ to } k \text{ step2\_nphases } n_{\text{reduce}} - 1] && \text{-- Step 2 receive buffers} \\
 &[k \text{ step2\_nphases } n_{\text{reduce}} \text{ to end}] && \text{-- Step 2 send buffers}
 \end{aligned}$$

**Segmented/Linear allreduce.** For unsupported cases, i.e. when the scratch space for reduce is not enough for the pWrk buffer of the above algorithm, or if the team is not the world team, a simple linear algorithm is used. Each PE sends its data to all other PEs, and performs a local reduction. The user can use a build option NVSHMEM\_GPU\_COLL\_USE\_LDST to specify its zero-copy variant that utilizes direct pointers. May use segmentation if the scratch space is even smaller than the total data size: breaking data into segments and performing a linear reduction for each.

**Hierarchical fcollect.** This algorithm is divided into intranode and internode stages. First, every node-level subteam performs a fcollect (allgather) to gather each PE's data chunk into contiguous scratch, and then do a single-thread linear reduction across the gathered slices locally so every node produces a fully reduced tile. Next, GPUs that share an NVSwitch port (the same\_mype\_node team) run another fcollect to aggregate those node-level results and reuse the same tile-wise reduction to combine them and produce the final result.

**Complexity analysis.** Let  $n$  be the reduce data size and  $P$  be the total number of PEs. NVLS one-shot issues exactly one NVLS multicast (latency  $O(1)$ ) on the whole data range, so each GPU performs  $O(nP)$  work and injects  $\Theta(nP)$  bytes regardless of team size, since the fabric fans the data out, making the operation bandwidth-bound; and that's  $O(nP^2)$  communication volume in total. Likewise, in NVLS two-shot, each GPU performs  $O(n)$  work in  $O(1)$  latency, and the total communication volume is  $O(nP)$ , which is the theoretical lower bound of allreduce. For the non-NVLS path, the recursive exchange runs  $O(\log_k P)$  rounds (latency) where each of the  $P$  PEs exchanges  $n$  bytes with up to  $k - 1$  neighbors, giving  $\Theta(P \times n \times k \times \log_k P)$  total injection; hierarchical `fcollect` performs two  $O(nP)$  gather/reduce phases (node and same-mype node), so the aggregate bandwidth is still  $\Theta(nP^2)$  but the local work is batched per tile, and the latency is the same order as the underlying `fcollect`, which is  $O(1)$  for NVLS-based `fcollect` and  $O(P)$  otherwise; and the segmented fallback iteratively sends chunks that fit in the scratch buffer to every other PE, yielding the classic  $\Theta(nP^2)$  byte communication volume and  $O(P)$  latency complexity. Thus, absent NVLS hardware allreduce scales like other software allreduces, but NVLS modes collapse the message count to one multicast per PE.

### 3.7.5 Reduce-scatter

**Algorithm selection.** NVSHMEM reduce-scatter has a software P2P one-shot all-push algorithm and an NVLS-based one-shot algorithm. It first checks whether NVLS multicast reduction is legal for this reduce operation: the operation must be NVLS-capable (non-float/double types for MIN/MAX, or any data type for SUM/AND/XOR/OR), the datatype must be "canonical" ( $\geq 32$ -bit) or a special half/bfloat SUM case with even team size and element count  $\geq 2$ , the message must be 4-byte aligned, and the team must expose NVLS resources (`nvls_rsc_base_ptr`).

NVSHMEM tries to select NVLS one-shot when possible, or otherwise fall back to the software allpush path.

**Software one-shot P2P all-push algorithm.** In the non-NVLS path, NVSHMEM implements a direct "all-push" reducescatter. Each PE conceptually owns a contiguous slice of length `nreduce` in the final result, and its input buffer is a concatenation of slices destined for every PE in the team. The algorithm proceeds in two phases:

1. *All-push to scratch.* Each team maintains a symmetric scratch buffer `pWrk`, which is a `pSync` buffer of the team that is reserved for reduce operations. It is logically partitioned as `[PE0 block][PE1 block] ... [PEP-1 block]`, one block per PE, each of size `nreduce`. For a team of size  $P$  and calling PE index `my_pe`:

- for every other PE index `peer_pe_idx`, it sends (via the non-blocking put `put_nbi` at `SCOPE`) the `nreduce` elements of its source that belong to `peer_pe_idx`'s final slice to the block at offset `my_pe` in the *peer*'s scratch;
- after this phase, at PE  $p$ , the block `pWrk[p]` contains all remote contributions from other PEs for slice  $p$ , while some contributions may still reside in the local source buffer.

A threadgroup-wide barrier `nvshmemi_barrier_threadgroup<SCOPE>(team)` then ensures that all PEs have completed their puts before reduction begins.

2. *Local reduction.* Each PE then performs element-wise reductions over its own data:

- for the first reduction step, the code chooses `op1` and `op2` carefully from `source` and `pWrk` so as to avoid unnecessary copies (the exact choice depends on `nvshmemi_team_my_pe(team)`);
- for subsequent steps, it repeatedly reduces either from `source` or from the appropriate `pWrk` blocks into a running result buffer `dest`;
- the element-wise reduction itself is implemented by the linear reduction procedure `gpu_linear_reduce_threadgroup<TYPE,OP,SCOPE>` (described in the previous allreduce section), which applies the selected reduction operator (SUM, MIN, MAX, AND, OR, XOR, etc.) cooperatively across the threadgroup.

A final `nvshmemi_threadgroup_sync<SCOPE>()` ensures that all threads see the completed reduction in `dest`.

The scratch buffer must be large enough to hold  $P$  blocks of size `nreduce * sizeof(TYPE)`. The implementation explicitly checks that

$$nreduce \cdot \text{sizeof}(\text{TYPE}) \cdot P \leq 0.5 \times \text{NVSHMEM\_REDUCE\_SCRATCH\_SIZE},$$

and aborts with an assertion if this condition is violated. Thus, insufficient scratch space causes the collective to error out and the user must increase `NVSHMEM_REDUCE_SCRATCH_SIZE`.

**NVLS one-shot algorithm.** In the NVLS algorithm:

1. Each PE computes the appropriate `source_offset` into `source` for its reduce-scatter slice as `nreduce * nvshmem_team_my_pe(team)`.
2. Using `nvshmemi_mc_ptr`, it obtains a multicast-capable pointer `src_ptr` into the unified NVLS address space corresponding to `source + source_offset`.

3. A threadgroup-wide local reduce/multicast `nvshmemi_local_reduce_mcast_threadgroup`, is then invoked to perform a hardware-supported reduction via NVSwitch’s `ld.red` primitive, writing the reduced result directly into `dest`.

Because NVLS reductions are atomic at the switch level and the final result is written locally, no additional threadfence is required at the end of the NVLS `reducescatter`. A concluding `nvshmemi_sync_threadgroup<SCOPE>(team)` ensures that all threads in the threadgroup see the final values in `dest` before proceeding.

In both the P2P all-push and NVLS one-shot variants, NVSHMEM’s `reducescatter` trades more aggressive one-shot communication (each PE performs only a single wave of sends) for simpler synchronization: once the all-push and local reduction are complete, no remote state remains, and global completion reduces to intra-GPU threadgroup synchronization.

**Complexity analysis.** The complexity of reduce-scatter algorithms largely resembles that of allreduce. (1) NVLS one-shot: each PE issues a single multicast ( $O(1)$  latency) reduction load into its local `dest` slice and the fabric handles replication, so per-PE work is  $O(nP)$ ; aggregate injection is  $\Theta(nP^2)$  bytes (one reduced stream per PE). (2) Software allpush: each PE sends its segment to every other PE ( $P-1$  puts,  $O(P)$  latency), then performs  $P-1$  local reductions over  $n$  elements. That yields per-PE message count  $\Theta(P)$ , local compute  $\Theta(nP)$ , and total network volume  $\Theta(nP^2)$  bytes, i.e., the classic all-to-all exchange cost.

### 3.7.6 Barrier and sync

`nvshmem_sync` and `nvshmem_barrier` both use a **dissemination algorithm** to implement team-wide arrival synchronization across PEs. This is implemented in the threadgroup synchronization routine `nvshmemi_sync_algo_threadgroup`. It chooses a dissemination radix  $k$  and then runs a  $k$ -ary dissemination barrier among the PEs in the team.

*Radix selection and specialization.* The dissemination radix  $k$  controls how many neighbors each PE communicates with per phase in the dissemination barrier. NVSHMEM derives  $k$  from an environment-controlled parameter `barrier_tg_dissem_kval` (clamped to the team size and to a minimum of 2). For BLOCK-scope collectives on teams whose GPUs are fully P2P-connected (`are_gpus_p2p_connected`), the implementation further boosts  $k$  to the full team size, effectively collapsing the barrier into a very small number of phases.

For commonly used power-of-two radices  $k \in \{2, 4, 8, 16, 32\}$ , NVSHMEM dispatches to a set of statically specialized templates



`sync_dissem_pow2_threadgroup<k,logk,SCOPE>`. These variants can be aggressively optimized by the compiler, unrolling loops and simplifying shift operations. For other values of  $k$  (or arbitrary team layouts), the code falls back to the generic `sync_dissem_threadgroup<SCOPE>` implementation, which handles both strided and non-strided team configurations.

*Dissemination across PEs and threadgroups.* Both the specialized and generic variants implement a dissemination barrier in logarithmic phases, much like the allreduce recursive exchange (Figure 3.6):

- The team maintains a per-barrier counter `sync_counter` and a shared array `pSync` of length `size`, indexed by PE.
- In each phase, the active PEs are conceptually arranged in a ring of length `size`. Each PE sends a notification (using `nvshmemi_signal_for_barrier`, internally a remote store/atomic) to up to  $k - 1$  forward neighbors at increasing power-of- $k$  strides, and spin-waits (via `nvshmemi_wait_until_greater_than_equals`) for corresponding notifications from  $k - 1$  backward neighbors.
- Within a given SCOPE (warp, warpgroup, or block), threads cooperate: the thread with index `myIdx` in the threadgroup handles neighbors `j = myIdx+1, myIdx+1+groupSize, ...,` and a local `nvshmemi_threadgroup_sync<SCOPE>()` is called between phases so that the threadgroup progresses in lockstep.
- The number of phases is on the order of  $\lceil \log_k(\text{size}) \rceil$ ; in each phase, each PE sends and receives at most  $k - 1$  point-to-point notifications.

At the end of the dissemination, a designated thread (typically `myIdx == 0` within the threadgroup) increments the shared counter `sync_counter[0]`, which serves as the barrier epoch, and a final threadgroup synchronization ensures that all threads observe the updated counter before leaving the routine. The overall time complexity of the algorithm is therefore

$$O((k - 1) \log_k P)$$

point-to-point notifications per PE for a team of  $P$  PEs, which reduces to the familiar  $O(\log P)$  behavior for constant  $k$ .

*Relation to full barriers and memory ordering.* The function `nvshmemi_sync_algo_threadgroup` is called in both `nvshmem_sync` and `nvshmem_barrier`:

- `nvshmemi_sync_threadgroup` simply performs an intra-threadgroup sync followed by `nvshmemi_sync_algo_threadgroup`, providing an arrival-only team synchronization.
- `nvshmemi_barrier_threadgroup` adds memory-ordering semantics: before calling `nvshmemi_sync_algo_threadgroup`, it issues either

`nvshmemi_transfer_quiet` (for IBGDA/proxy-based jobs) or a `__threadfence_system()`, and after the dissemination, it optionally calls `nvshmemi_transfer_enforce_consistency_at_target`. These additional steps ensure that outstanding RMA operations are globally visible and that remote targets observe a consistent state.

In summary, `nvshmemi_sync_algo_threadgroup` provides an efficient, dissemination-style arrival barrier over arbitrary NVSHMEM teams, exploiting P2P connectivity and threadgroup cooperation to achieve  $O(\log_k P)$  synchronization steps while leaving memory fencing to higher-level barrier routines.

### 3.8 Memory Ordering

NVSHMEM's memory ordering operation, the `nvshmem_quiet` function, has both a host version and a device version (which complete only communication operations issued from the CPU and the GPU, respectively), along with an on-stream version that allows completing GPU-initiated communication from the CPU side.

The host-side quiet operation calls `cudaStreamSynchronize` on all the active internal CUDA streams in host-side state, and then calls transport-level quiet operations on every remote transport.

The on-stream version `nvshmemx_quiet_on_stream(cudaStream_t cstrm)` first synchronizes all internal streams with the user-provided stream `cstrm`, ensuring all prior NVSHMEM operations are visible on the user stream. After that, if the user stream is capturing a CUDA graph, it will launch 1 threadblock of `nvshmemi_quiet`, the device-side quiet operation (see below); otherwise, NVSHMEM launches one for every connected remote transport, or does nothing if there is none.

The device-side quiet operation has a control flow depending on the GPU's communication capability.

**Proxy or remote atomics.** If the GPU only supports proxy or if the atomics have to go through NIC (job connectivity  $\geq$  `NVSHMEMI_JOB_GPU_LDST_REMOTE_ATOMICS`), the slow path via `nvshmemi_transfer_quiet()` is invoked, handing over control to the transfer layer. The `nvshmemi_transfer_quiet()` function provides a specialized transfer-layer mechanism to enforce ordering and completion of GPU-initiated transfers. It flushes internal completion queues or counters (implements producer-consumer patterns with head/tail pointers), or issues a `membar.sys` instruction if the `use_membar` parameter is set, to wait for remote memory operations' completion.

**Direct ld/st.** Otherwise, the GPU supports direct loads and stores, without any communication happening over NIC. In this case, NVSHMEM only needs to call the CUDA memory fence function `__threadfence_system()`

plus a `__syncwarp()` or `__syncthreads()` as the fast path to implement the `nvshmem_quiet` semantic:

```
if (!myIdx)
    __threadfence_system();
nvshmemi_threadgroup_sync<SCOPE>();
```

The `__threadfence_system()` ensures memory visibility within the scope of P2P-mappable peers. The `__syncwarp()` or `__syncthreads()` then propagates data visibility among the warp or threadblock.

---

# Benchmarking

---

### 4.1 Simple Benchmarking

NVSHMEM comes with its own set of perftests. In this chapter, we present a set of benchmark results for NVSHMEM v3.3.9. All experiments were conducted on the Alps supercomputing cluster at the Swiss National Supercomputing Center (CSCS), using 16 nodes equipped with NVIDIA Grace Hopper Superchips (GH200). Each node has 4 GH200 GPUs fully connected with 150 GB/s per-direction NVLink intra-node interconnect, and a 25 GB/s per-direction Cray Slingshot network for internode communication. The cluster topology comprises 8 chassis per cabinet, 7 blades per chassis, and 2 nodes per blade. Although a chassis can contain up to 8 blades, Alps' GH200 chassis are intentionally underpopulated so that CSCS can increase the amount of power delivered to each GPU without exceeding the power limit of each cabinet.

#### 4.1.1 P2P

##### Benchmarking Infrastructure

In this section, we present results of unidirectional point-to-point (P2P) operations, including:

- `nvshmem_put` operations;
- `nvshmem_get` operations;
- `nvshmem_p`, the single-element put operations;
- `nvshmem_g`, the single-element get operations;
- Direct LD/ST operations, `nvshmem_ptr` direct pointer-based.

Specifically, in the bandwidth tests, the benchmarked APIs are:

- `nvshmem_uint32_g` and `nvshmem_uint32_p`, the single-element get/put operation on 32-bit integers, called repeatedly to cover the total message size;
- `nvshmemx_double_get_nbi_block` and `nvshmemx_double_put_nbi_block`, the non-blocking threadblock-scope bulk get/put operation, invoked once per iteration for the entire data range;
- and the (single-element) store operation using direct symmetric pointers obtained from `nvshmem_ptr`, called repeatedly to cover the whole data size:

```
// symmetric address of peer's data_d object
remote_d = (double *)nvshmem_ptr((void *)data_d, !mype);

// loop to cover the whole data size
for (size_t idx...) *(remote_d + idx) = *(data_d + idx);
```

While in the latency tests, they are: `nvshmem_int_g` and `nvshmem_int_p`, called repeatedly to cover the total message size; and `nvshmemx_int_get_nbi` and `nvshmemx_int_put_nbi`, the non-blocking thread-scope get/put operation, invoked once per iteration for the entire data range.

These benchmarks evaluate the performance of NVSHMEM one-sided communication primitives in terms of throughput and latency. Each test can sweep over total message sizes (ranging from 4 B to 4 MB by default), threads per block (TPB) (e.g., 32 → 1024, fixed data per thread), or number of CTAs (e.g., 1 → 32, fixed TPB and data per thread). For each message size / TPB / CTA, a kernel is launched that calls the target API for 5 untimed warm-up iterations followed by another kernel launch with 10 timed iterations, separated by global synchronization via `nvshmem_barrier_all()` to ensure completion and synchronization across PEs.

Latency is measured using CUDA events on the default stream, and bandwidth is reported in GB/s as:

$$BW = \frac{\text{Message Size}}{\text{Average Time per Iteration}}$$

A simplified pseudocode of the benchmark loop is shown below:

```
for each parameter value p in {message size, TPB, CTA} set
{
  reset_device_state(counter_d);
  // Warmup
  kernel<<<BLOCKS, THREADS>>>(data, ..., p, skip_iter);
```

```

// Timed iterations
cudaEventRecord(start); // default stream
kernel<<<BLOCKS, THREADS>>>(data, ..., p, iter);
cudaEventRecord(stop);
cudaEventSynchronize(stop);

// Compute latency or bandwidth
latency_ms = get_cuda_event_elapsed_time(start, stop) / iter;
bw = size * N_ITER / latency_ms / (B_TO_GB / MS_TO_S);

nvshmem_barrier_all(); // Synchronize all PEs
}

```

Each benchmark kernel repeatedly calls the NVSHMEM primitive under test to transfer data of the specified size between corresponding PEs. The `nvshmem_barrier_all()` ensures global synchronization between timing windows, eliminating inter-iteration interference.

Note that this pseudocode is for device-side test. Host-side tests loop outside the kernel (and call the kernel for a number of times, rather than loop inside one kernel). But we skip the host-side tests for now, because every host-side test is simply a wrapper that calls into NCCL.

Inside each bandwidth (bw) kernel, every iteration enforces both intra-CTA and inter-CTA synchronization to ensure correctness and consistent measurement. Within each CTA, threads are synchronized using `__syncthreads()`, ensuring progress of per-thread NVSHMEM operations. Across CTAs, an atomic counter in device memory is used to implement a global fence: once all blocks have completed an iteration, the last block triggers an update to a shared signal variable that other blocks poll before continuing; a `__threadfence()` guarantees that all data writes are visible to other CTAs before this signal is set. This mechanism enforces a global fence between iterations, ensuring that all threads in all CTAs advance in lockstep and preventing inter-iteration interference.

The pseudocode of this per-iteration synchronization in bw kernels:

```

__global__ void bw_kernel(...) {
    for (int i = 0; i < iter; i++) {
        nvshmemx_double_get_nbi_block(...);

        // Global barrier (both intra- and inter-CTA)
        __syncthreads();
        if (!tid) {
            __threadfence();
            counter = atomicInc((unsigned int
                ↪ *)counter_d, UINT_MAX);
        }
    }
}

```

```

        if (counter == (gridDim.x * (i + 1) - 1)) {
            *(counter_d + 1) += 1;
        }
        while (*(counter_d + 1) != i + 1)
            ;
    }
    __syncthreads();

}
// Final synchronization and quiet
if (blockIdx.x == 0 && !tid) nvshmem_quiet();
}

```

While inside latency kernels, each kernel issues an `nvshmem_quiet()` call per iteration to ensure *completion* of the target API operation; the only exception is the `nvshmem_g` test, since `nvshmem_g` is a blocking primitive that inherently waits for completion.

Example pseudocode for the latency kernel:

```

__global__ void latency_kern(int *data_d, int len, int pe, int
↪ iter) {
    int i, tid, peer;
    peer = !pe;
    tid = threadIdx.x;

    for (i = 0; i < iter; i++) {
        nvshmemx_int_get_nbi_###threadgroup(data_d, data_d, len,
↪ peer);

        __syncthreads();
        if (!tid) nvshmem_quiet();
        __syncthreads();
    }
}

```

This per-iteration `quiet()` operation is to serialize RMA progress and measure end-to-end visibility latency per operation. Specifically, each `get/put` is immediately followed by `quiet` to force completion of the target API before the next iteration, eliminating overlap and pipeline effects that would otherwise understate true per-operation latency. By contrast, bandwidth tests defer `quiet` until the end of the kernel so that many nonblocking operations can remain in flight concurrently, exploiting device and interconnect concurrency to approach saturation throughput; lightweight intra-kernel fences suffice to coordinate producers, and a single `quiet` at the end ensures global completion before reporting results. This usage of `quiet()` operation

matches NVSHMEM's completion semantics, enabling accurate latency measurement when applied per-iteration and high utilization when applied only at the end. NVSHMEM uses non-blocking APIs followed by an explicit `nvshmem_quiet()` rather than blocking APIs here.

In addition to the single-direction latency tests, we also have ping-pong latency benchmarks that measure full round-trip time between two PEs. It uses a symmetric flag variable for synchronization. Each iteration involves one PE sending data via `nvshmem_put_nbi()` and signaling completion using `nvshmemx_signal_op()`, while the peer waits using `nvshmem_wait_until()` before replying. This measures the effective latency of bidirectional communication. Its core device loop can be summarized as:

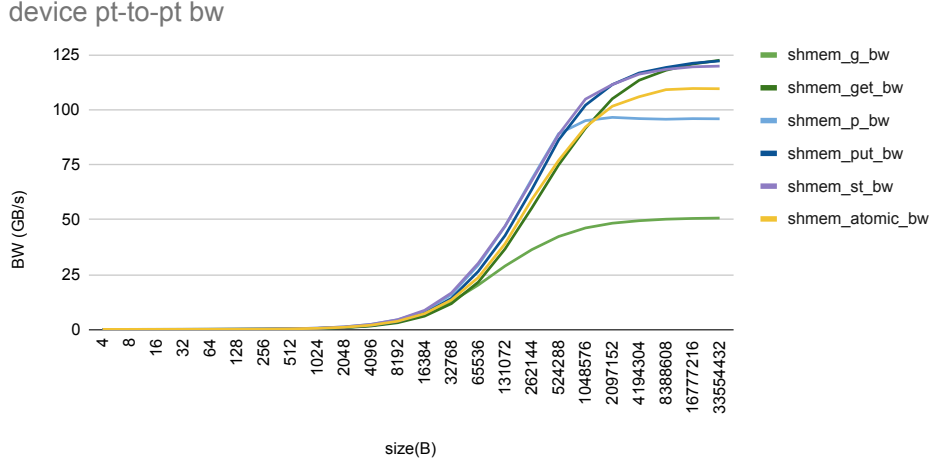
```
for (i = 0; i < iter; i++) {
    if (pe == 0) {
        nvshmem_put_nbi(remote, local, len, peer);
        nvshmem_fence();
        nvshmemx_signal_op(flag, i+1, NVSHMEM_SIGNAL_SET, peer);
        nvshmem_wait_until(flag, NVSHMEM_CMP_EQ, i+1);
    } else {
        nvshmem_wait_until(flag, NVSHMEM_CMP_EQ, i+1);
        nvshmem_put_nbi(remote, local, len, peer);
        nvshmem_fence();
        nvshmemx_signal_op(flag, i+1, NVSHMEM_SIGNAL_SET, peer);
    }
}
nvshmem_quiet();
```

### Intra-node

**Bandwidth test - message-size sweep.** We loop over data sizes ranging from 4 B to 32 MB and measure one-way bandwidth, using 32 cooperative thread arrays (CTAs) with 256 threads per block (TPB), representing the default configuration in NVSHMEM perftests.

Figure 4.1 demonstrates that `nvshmem_g` achieves a maximal bandwidth half of, and `nvshmem_p` reaches a slightly lower maximal bandwidth in comparison to, the other P2P operations. This performance gap arises from the overheads introduced by frequent single-element calls, as well as the inherent costs associated with read-based communication protocols. The `nvshmem_g` operation issues numerous fine-grained load transactions, each of which incurs synchronization and address-resolution overheads. Due to its single-element nature, `nvshmem_g` lacks aggregation opportunities, limiting its effective bandwidth. The relatively lower performance of `nvshmem_p` follows the same reasoning, though its write semantics typically introduce fewer stalls than reads.





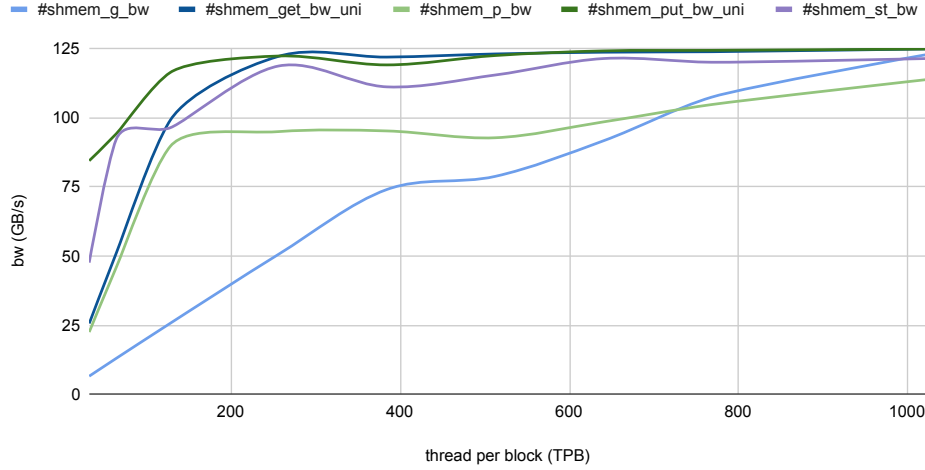
**Figure 4.1:** Device point-to-point unidirectional bandwidth by message size for various NVSHMEM operations. The figure demonstrates bandwidth scaling with message size 4B to 4MB under 32 CTAs and TPB=256 for six P2P operations. `st`, `put`, and `get` achieve the highest peak bandwidth  $\sim 115$  GB/s, while `atomic` reaches 105 GB/s and `p` peaks at 95 GB/s. The `g` operation shows significantly lower performance, plateauing at 50 GB/s, demonstrating the overhead of single-element operations and read protocol limitations. All operations show low bandwidth for small message sizes  $< 1\text{KB}$  and saturate at larger sizes  $> 1\text{MB}$ .

**TPB-sweep.** We have also experimented by varying the number of threads per CTA (or threads per block, TPB) within the range of 32 to 1024, while keeping the number of CTAs fixed at 32 (the NVSHMEM perfctest default) and maintaining a constant data size per thread of 4KB and 4B respectively. As shown in Figure 4.2, under data sizes of 4KB per thread, the bandwidth of `nvshmem_g` continues to increase gradually without reaching a clear saturation point, even at the maximum TPB of 1024, whereas the other 4 APIs saturate at TPB = 128 or 256. Notably, at TPB = 1024, `nvshmem_g` achieves a bandwidth comparable to the peak bandwidth of the other APIs, reaching approximately 125 GB/s. For smaller messages (4B per thread), none of the 5 APIs reach bandwidth saturation, as seen in Figure 4.3a.

This TPB-sweep reveals that `nvshmem_g` is highly sensitive to thread-level parallelism; the continued growth in bandwidth with increasing TPB indicates that more concurrent threads are required to hide the latency of individual `nvshmem_g` operations and to fill the NVLink communication pipeline. In contrast, bulk operations such as `put` and `get` quickly reach their peak performance since they already utilize large transfer granularity per thread. The lack of saturation for 4-byte data sizes suggests that all APIs remain dominated by per-call launch and synchronization overheads, and that small messages are insufficient to amortize protocol costs across threads. Overall, the analysis highlights that performance scaling with TPB

device pt-to-pt bw by TPB

4KB/thread data size. 32 CTA.



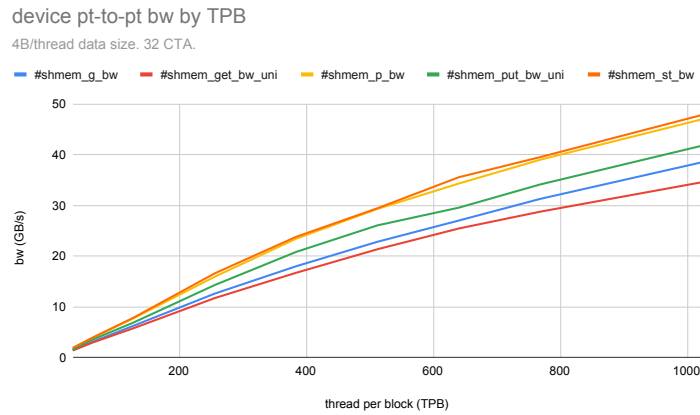
**Figure 4.2:** Device point-to-point bandwidth by threads per block (TPB) for various NVSHMEM pt-to-pt operations. The graph shows bandwidth performance of five P2P operations (`nvshmem.put`, `nvshmem.get`, `nvshmem.p`, `nvshmem.g`, and direct stores to `nvshmem.ptr`) with 4KB data per thread and 32 CTAs. The `g` operation shows continuous bandwidth increase without saturation, reaching 120 GB/s at TPB=1024, while other operations saturate around TPB=128-256 but converge to similar bandwidth ( $\sim 125$  GB/s) at higher TPB values. The `put` operation achieves the highest and most stable performance across the TPB range.

is operation-dependent, with `nvshmem.g` benefiting most from increased intra-CTA concurrency.

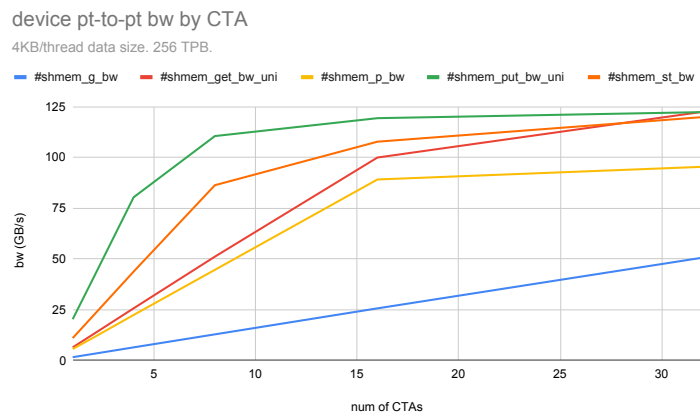
**CTA-sweep.** Similarly, a CTA-sweep from 1 to 32 results in a rising bandwidth across all 5 APIs, as Figure 4.3b and 4.3c show; even at 32 CTAs, the achieved bandwidth does not yet saturate. This observation suggests that the available GPU resources and NVLink endpoints were not fully utilized within the tested range. Increasing the number of CTAs effectively distributes communication across more streaming multiprocessors (SMs), thereby enhancing throughput through better overlap of memory and communication. However, the absence of clear saturation even at 32 CTAs implies that inter-warp communication latency and NVLink round-trip time still represent limiting factors. Larger CTA counts or multi-GPU concurrency could be explored in future versions to identify the true saturation point of the interconnect.

In practical GPU applications, however, it is often undesirable to allocate such a large fraction of SMs exclusively to communication. Production workloads typically prioritize reserving most SMs for computation while dedicating only a small subset for data movement. For instance, DeepEP [36], the Mixture-of-Experts (MoE) communication library developed by DeepSeek, demonstrates

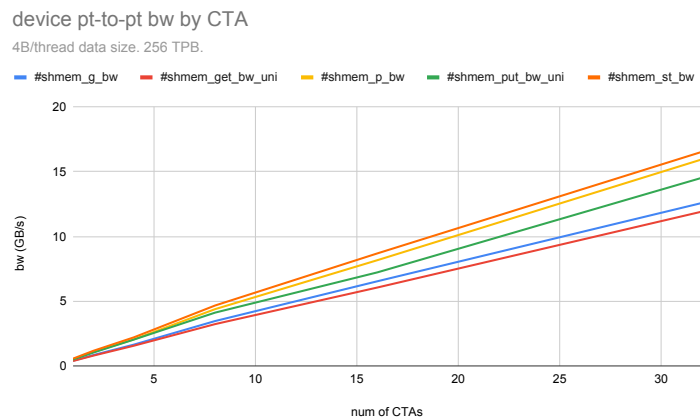
## 4.1. Simple Benchmarking



**(a)** TPB-sweep: device pt-to-pt bandwidth at 4 B/thread (32 CTAs) rises roughly linearly with TPB for all five APIs, with no saturation up to 1024 threads, indicating overhead-dominated small messages.



**(b)** CTA-sweep at 4 KB/thread, 256 TPB: bulk operations quickly approach 120–130 GB/s by 8–16 CTAs, while `nvshmem_g` continues to climb and remains well below peak, showing its higher sensitivity to intra-CTA parallelism.



**(c)** CTA-sweep at 4 B/thread, 256 TPB: bandwidth increases steadily with CTA count for all APIs and does not saturate by 32 CTAs, suggesting per-call/latency costs dominate for tiny messages.

Figure 4.3

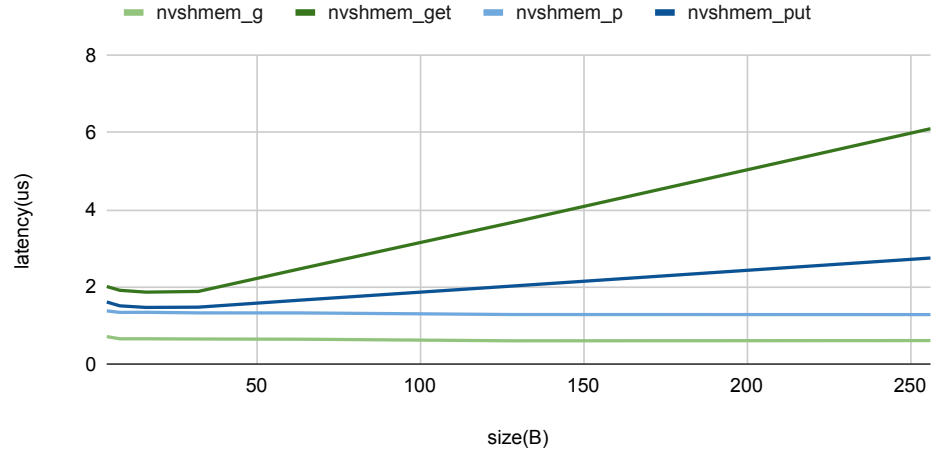
an efficient balance between compute and communication: it dedicates only around 20 SMs to handle inter-expert data exchange. Moreover, each expert communicates with at most 4 peers, which aligns well with their system’s interconnect topology — 160 GB/s intranode NVLink bandwidth and 50 GB/s internode IB bandwidth, i.e. 3.2x intranode/internode bandwidth ratio. This design achieves near-optimal overlap between compute and communication without fully saturating NVLink, highlighting that theoretical saturation is not necessarily the practical goal. Similarly, in our experiments, a configuration of 32 CTAs may already overcommit compute resources to communication tasks on GH200 systems; in many real workloads, fewer CTAs would likely yield a better balance between compute occupancy and communication efficiency. This trade-off highlights that achieving theoretical saturation is less critical in practice than maintaining a balanced ratio between computation and communication, ensuring that data movement overlaps efficiently with ongoing compute kernels.

**Latency test - message-size sweep.** From a latency perspective, while the single-element operations `nvshmem_g` and `nvshmem_p` do not offer bandwidth advantages, they demonstrate significantly lower latencies than their bulk counterparts, `nvshmem_get` and `nvshmem_put`, for data sizes up to 8KB. Figure 4.4 highlights this difference for payloads up to 256 bytes, where the latency gap is most visible. This trend is expected: the lightweight semantics of `g` and `p`, involving minimal synchronization and smaller payloads, make them more efficient for fine-grained data exchange. Conversely, `get` and `put` introduce additional setup overheads to enable bulk transfers and pipelining, which becomes beneficial only at larger data sizes. Hence, `g` and `p` operations are particularly suitable for latency-critical workloads where small, irregular data movements dominate.

When sweeping through data size per thread, latency first stays flat and later starts to bump up linearly. We tested `get`, `put` operations under all scopes (thread, warp, block) (Figure 4.5). For very small per-thread sizes up to 64–256 B, the latency floor ( $\sim 2 \mu\text{s}$ ) is dominated by per-call startup and synchronization overheads; increasing the size in that region changes the latency very little. Also, write is conducted in cache lines, so no difference for small data size. Beyond an inflection point of  $\sim 256$  Bytes / thread, latency grows roughly proportional to payload, reflecting dominant transfer time. Block scope shows the steepest latency increase once payload exceeds the per-thread inflection, because they aggregate more work per call and so their setup and serialization costs are larger. In short: small messages are startup-limited, while larger messages are bandwidth-limited. Consequently, small-message workloads benefit from `g/p`’s lower startup costs, whereas throughput-oriented large transfers should use bulk APIs to maximize sustained bandwidth.

## latency comparison of g, p, get, put

Single-thread scope. For small data up to 256B.



**Figure 4.4:** Latency comparison of single-element operations (g, p) versus bulk operations (get, put) for small data sizes up to 256B. The graph demonstrates that `nvshmem_g` and `nvshmem_p` maintain significantly lower and more stable latencies compared to their bulk counterparts `nvshmem_get` and `nvshmem_put` for data sizes up to 256B. `nvshmem_g` shows the lowest latency (0.5-0.6 us) with minimal size dependency, and `nvshmem_p` also maintains low latency (1.2-1.3 us). In contrast, `nvshmem_put` shows gradual latency increase (1.5-2.7 us), and `nvshmem_get` exhibits the highest latency with steep size-dependent increase (2.0-6.0 us), highlighting the overhead of bulk operations for small data transfers.

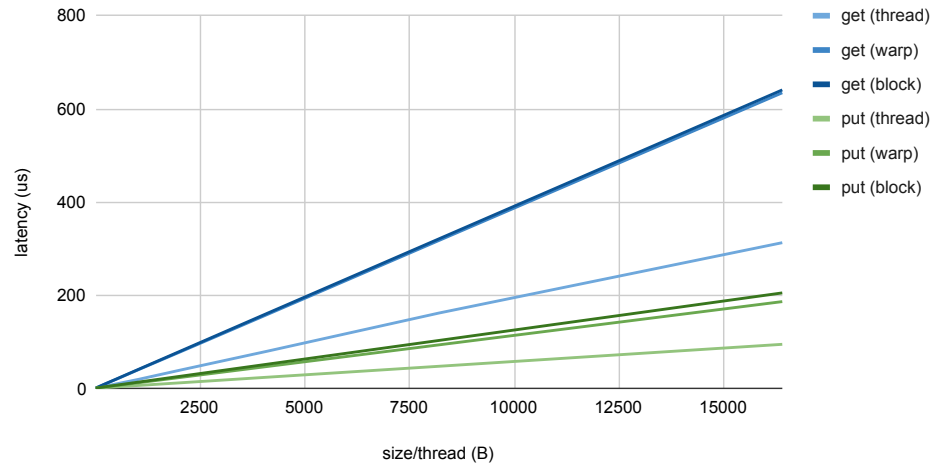
**TPB-sweep.** A TPB sweep in the latency tests shows an almost flat trend for `nvshmem_get` and `nvshmem_put`, whereas `nvshmem_g` and `nvshmem_p` begin to experience 25–100 % higher latency once  $\text{TPB} > 256$ . The ping-pong latency for `nvshmem_p` is at its minimal at  $\text{TPB} = 512$ . This is shown in Figure 4.6. The flat latency curve for bulk operations implies that their startup costs dominate and are largely unaffected by thread-level parallelism. In contrast, the increasing latency for g and p at high TPB can be attributed to contention within the same CTA — specifically, serialized memory accesses and limited availability of outstanding requests, as calculated above for the bandwidth tests. The optimal point of the ping-pong latency test observed at  $\text{TPB} = 512$  likely represents a balance between concurrency and contention. Beyond that, the benefits of additional threads are outweighed by scheduling and synchronization overheads.

Latency experiments were all conducted using a single CTA, hence no CTA-sweep was performed for these tests.

Overall, these results demonstrate that NVSHMEM’s fine-grained primitives (g/p) and bulk transfer APIs (get/put/st) exhibit distinct performance scal-

## get/put latency by size/thread

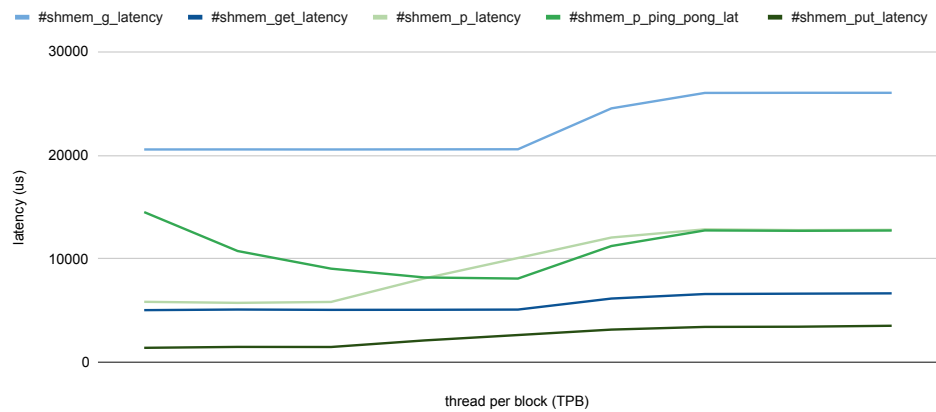
32-CTA, 256-TPB.



**Figure 4.5:** Latency of NVSHMEM get and put operations versus payload size per thread for thread-, warp-, and block-scope calls, measured with 32 CTAs and 256 threads per block. Latency grows roughly linearly with size; put is consistently faster than get, and block-level calls are the slowest.

## device pt-to-pt latency is largely irrelevant to TPB

4KB/thread data size. 32 CTA.



**Figure 4.6:** Device point-to-point latency versus threads per block (TPB) for various NVSHMEM operations with 4KB/thread data size and 32 CTAs. The graph shows that get and put maintain nearly flat latency profiles across the TPB range, demonstrating their insensitivity to thread count variations. However, g and p experience 25-100% latency increases when TPB exceeds 256, with g showing the most dramatic increase from ~20,500 us to ~26,000 us. The p ping-pong latency reaches its minimum value of 8,000 us at TPB=512 before increasing again, while put maintains the lowest overall latency of 1,500-4,000 us across all TPB values.

ing behaviors. While `get`, `put`, and `st` quickly reach saturation at moderate thread and CTA counts, fine-grained operations such as `nvshmem_g` and `nvshmem_p` deliver excellent latency characteristics for small messages but demand substantial thread-level parallelism to approach the throughput of bulk transfer primitives. This distinction suggests that the choice of API should be guided by workload characteristics:

**Performance takeaway.** Latency-sensitive applications should prefer single-element operations `g/p`, which is perfect for fine-grained, irregular data exchange patterns. (Also, avoid thread over-subscription for scalar APIs.) In contrast, throughput-oriented workloads benefit from bulk operations `get/put` that efficiently saturate NVLink bandwidth.

#### Inter-node

| op               | atomic_inc | get_bw_uni | put_bw_uni | p_bw   | g_bw   |
|------------------|------------|------------|------------|--------|--------|
| <b>bw (GB/s)</b> | 0.0010     | 23.7583    | 23.6468    | 0.0005 | 0.1121 |

**Table 4.1:** Performance metrics of shmem operations. Data size = 2097152 B, and threadgroup scope is their respective default: thread for `p` and `g`, and block for `put`, `get`.

**Message-size sweep.** In the internode configuration, a huge gap emerges between the fine-grained operations `nvshmem_g`, `nvshmem_p` and their bulk counterparts `nvshmem_get`, `nvshmem_put`. While `get` and `put` achieve up to 25 GB/s, approaching the theoretical peak of the Cray Slingshot interconnect, `g` and `p` exhibit throughput orders of magnitude lower, operating only in the MB/s range.

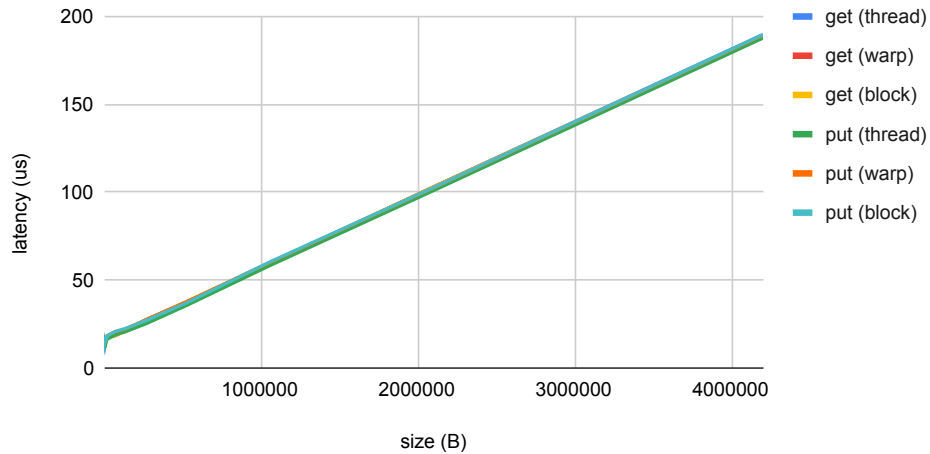
This dramatic gap can be primarily attributed to the high costs of issuing frequent, small payloads across the network when using CPU proxy. Each single-element call incurs communication protocol setup, remote addressing, and completion signaling costs, which dominate the total transfer time. Moreover, on our cluster where InfiniBand (and hence IBGDA) is not available, `g` and `p` go through the CPU proxy. NVSHMEM by default set CPU proxy to single channel (`CHANNEL_COUNT = 1`), prone to serialization under frequent small requests. Also, the proxy thread is set to process only 32 requests in a single iteration of the progress loop; increasing `NVSHMEM_PROXY_REQUEST_BATCH_MAX` to 512 or more increases `g` and `p`'s bandwidth by about 30%, but this is still orders of magnitude lower than bulk operations `get` and `put`.

In contrast, bulk operations amortize these costs over much larger data transfers, effectively saturating available bandwidth.

Latency measurements (Figure 4.7) reveal an additional distinction from the intranode case. Across all thread scopes (thread-, warp-, and block-level APIs), latency scales linearly with the *total message size*, rather than with

## Internode get/put latency by data size

32-CTA, 256-TPB.



**Figure 4.7:** Internode get/put latency by data size for different thread scopes (32 CTAs, 256 TPB). The graph demonstrates a completely linear correlation between latency and total data size, independent of thread scope (thread, warp, or block) and operation type (get vs. put). All six data series (get/put operations across thread, warp, and block scopes) show near-perfect overlap, with latency increasing linearly from 15-20 us for small data sizes to 185-190 us at 4MB. This linear relationship contrasts with intranode setups where latency scales with data size per thread, suggesting that internode networking overheads dominate the latency characteristics, making the total data volume the primary factor rather than the granularity of thread-level operations.

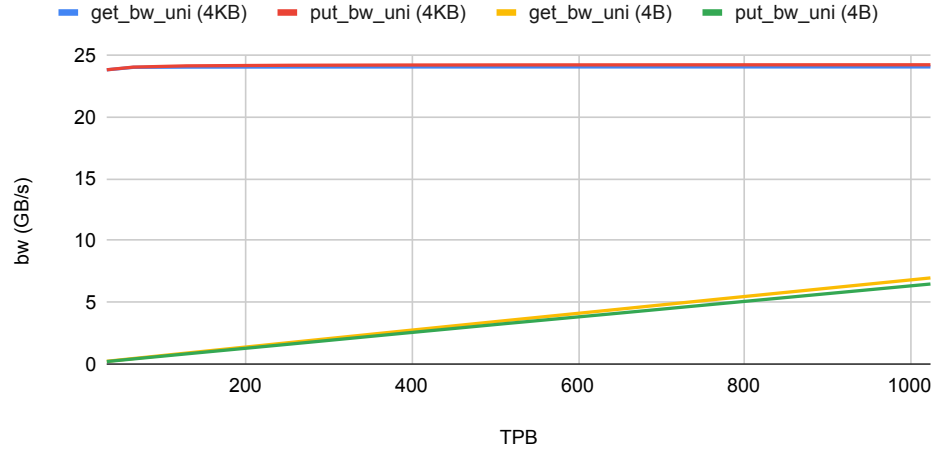
per-thread payload as seen in the intranode NVLink-based experiments. This behavior indicates that internode communication is governed by message-level transport costs, dominated by network injection, routing, and end-to-end acknowledgment delays, rather than GPU memory access latency. In other words, once a message leaves the node, GPU-level parallelism contributes little to hiding the communication latency.

**TPB and CTA sweep.** Thread-per-block (TPB) sweeps further reinforce this conclusion. Latency remains nearly constant across TPB configurations, showing that additional intra-block threads cannot compensate for network-induced delay. In bandwidth measurements, as Figure 4.8 shows, 4 B per-thread payloads fail to reach saturation even at a maximum TPB of 1024, while 4 KB per-thread transfers reach the 25 GB/s limit as early as TPB = 32. CTA-sweep experiments similarly show that increasing the number of CTAs beyond 4 yields no further bandwidth improvement, regardless of per-thread data size; this is shown in Figure 4.9. This plateau reflects the hardware-imposed network bottleneck: only a handful of concurrent communication streams are sufficient to fully utilize the available interconnect bandwidth, after which additional GPU concurrency provides diminishing returns.



## internode pt-to-pt bw to TPB

CTA=32. data/thread = 4KB or 4B.



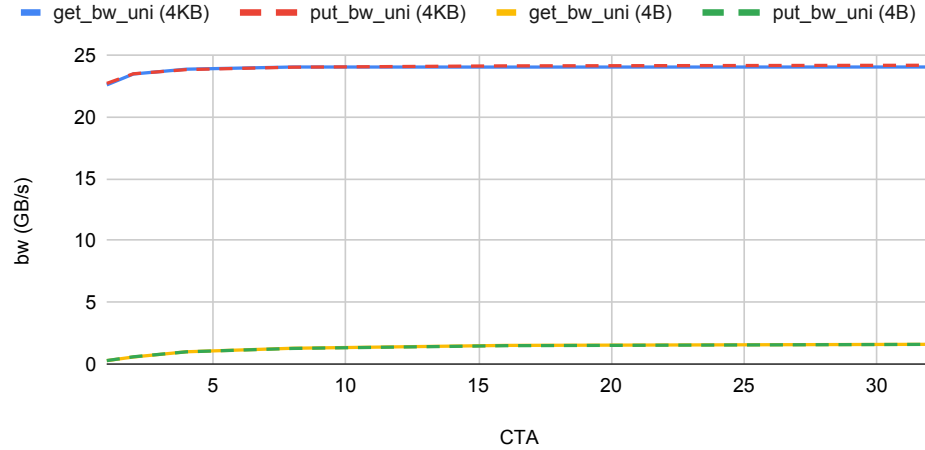
**Figure 4.8:** Internode point-to-point bandwidth versus threads per block (TPB) for different data sizes per thread (CTA=32). The graph shows that for 4KB data per thread, both get and put operations achieve rapid saturation at ~24 GB/s starting from TPB=32, maintaining this performance across the entire TPB range up to 1000. For 4B data per thread, both operations exhibit linear bandwidth increase without saturation, reaching 6-6.5 GB/s at TPB=1000. The 4KB operations demonstrate over 3x higher bandwidth than 4B operations at their respective peaks. These observations confirm that bandwidth tests do not reach saturation with 4B data per thread, while 4KB operations achieve saturation at approximately 25 GB/s starting from TPB=32, indicating that only a limited number of TPBs are necessary to reach the maximum bandwidth capacity set by internode networking.

To summarize, these results highlight a clear shift in the dominant performance factor when moving from NVLink to internode communication. While intranode performance is bounded by memory access parallelism and SM scheduling, internode performance is limited by the fixed per-message cost of the network stack. Optimizing NVSHMEM for internode use would require mechanisms such as message coalescing, batched command queues, or RDMA-based aggregation to bridge the large performance gap between fine-grained and bulk APIs. In the meantime, from a programmer's perspective:

**Performance takeaway.** Internode point-to-point transfers reach peak bandwidth with low GPU parallelism. Single-thread scope APIs are sufficient and recommended.

## internode pt-to-pt bw to CTA

TPB=256. data/thread = 4KB or 4B.



**Figure 4.9:** Internode point-to-point bandwidth versus number of CTAs for different data sizes per thread (TPB=256). The graph demonstrates that for 4KB data per thread, both get and put operations achieve rapid bandwidth saturation at  $\sim 24$  GB/s starting from CTA=5-10, with minimal performance variation across the CTA range up to 32. For 4B data per thread, both operations show gradual bandwidth increase without reaching saturation, achieving only  $\sim 1.5$  GB/s at CTA=32. The 4KB operations achieve over 15x higher bandwidth than 4B operations, and get/put operations perform nearly identically regardless of data size. These results indicate that bandwidth growth halts after CTA=4 for both data sizes, suggesting that only a limited number of CTAs are necessary to reach the maximum bandwidth capacity set by internode networking.

### 4.1.2 Collective

#### Benchmarking Infrastructure

In this section, we present the results of collective operations. We have benchmarked five NVSHMEM data collectives for bandwidth: `alltoall`, `bcast`, `fcollect` (allgather), `allreduce`, and `reducescatter`.

For these benchmarks, we no longer call explicit barriers in the test kernels; instead, the kernels only loop and call the target collective APIs. Consequently, we added one `nvshmem_barrier_all()` between the warmup iterations and the normal iterations, to ensure that the warmup iterations are complete.

```
__global__ void test_coll_kernel_##THREADGROUP(team, iter) {
    for (int i = 0; i < iter; i++) {
        nvshmem(x)_coll_##THREADGROUP(team);
        // limited to single thread/warp/block
    }
}
```

---

```

for each parameter value p in {message size, TPB, CTA} set
{
    reset_device_state(counter_d);
    // Warmup
    kernel<<<BLOCKS, THREADS>>>(data, ..., p, skip_iter);

    // Synchronize between warmup and normal phase
    nvshmem_barrier_all();

    // Timed iterations
    cudaEventRecord(start);
    kernel<<<BLOCKS, THREADS>>>(data, ..., p, iter);
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);

    // Compute latency and bandwidth
    latency_ms = get_cuda_event_elapsed_time(start, stop) /
        ↪ iter;
    algbw = size * N_ITER / latency_ms / (B_TO_GB / MS_TO_S);
    busbw = algbw * factor(coll);

    nvshmem_barrier_all();
}

```

As shown in the end of the pseudocode above, we're now differentiating between algorithm bandwidth (algbw) and bus bandwidth (busbw) in this section.

$$\begin{aligned}
 algbw &= \frac{\text{Message Size}}{\text{Average Time per Iteration}} \\
 busbw &= algbw \times \text{factor}
 \end{aligned}$$

where the bandwidth factor varies by target collective operation:

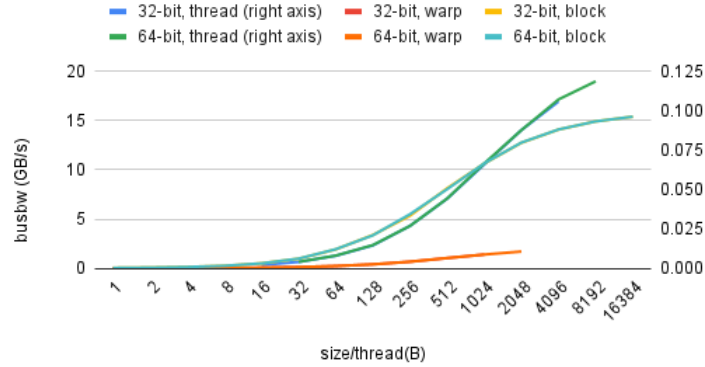
#### Intra-node

**Bandwidth.** Across all collectives, bandwidth scales linearly with increasing data size until saturation, as shown in Figure 4.10. Despite their similar scaling pattern, their asymptotic bandwidths vary significantly: 16 GB/s for `alltoall`, `bcast`, and `fcollect`; 8 GB/s for `reducescatter`; and only about 1 GB/s for `allreduce`.

These differences primarily arise from the distinct communication patterns and data movement semantics of each operation; the communication patterns

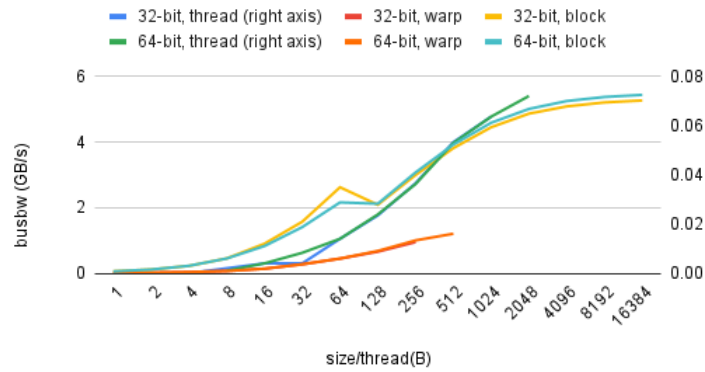
## 4.1. Simple Benchmarking

alltoall



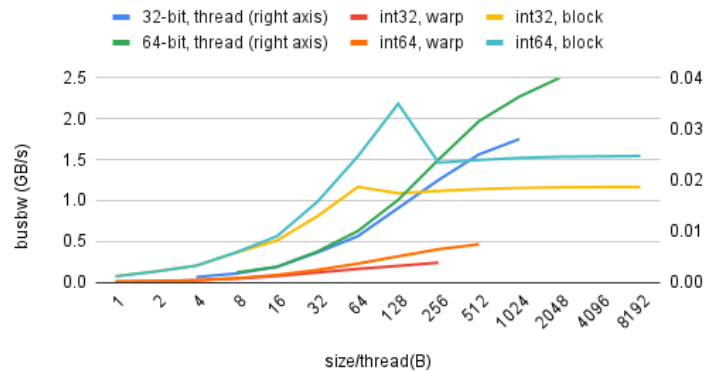
(a)

bcast



(b)

allreduce



(c)

**Figure 4.10:** The bandwidth for (a) alltoall, (b) bcast, and (c) allreduce relative to size per thread. Their bandwidth shows a linear increase until each hits their peak bandwidth. These maximum bandwidths are significantly lower than NVLink's theoretical capability, primarily <sup>87</sup> due to the default 1-CTA approach used in NVSHMEM. It's important to note a drop in bcast bandwidth when the size per thread transitions from 64 to 128 bytes; our analysis indicates that bcast internally alters its algorithm at this point. A similar pattern is observed for allreduce.

## 4.1. Simple Benchmarking

**Table 4.2:** Bandwidth scaling factors for different collective communication job types.

| Job Type(s)                                                                                                                                                                        | Factor Formula                     | Meaning / Notes                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|-------------------------------------------------------------------------------------------|
| reduction,<br>reduction_on_stream,<br>device_reduction                                                                                                                             | $\frac{2 \times (npes - 1)}{npes}$ | Accounts for both send and receive operations in reduction; higher cost per process.      |
| broadcast,<br>broadcast_on_stream,<br>bcast_device                                                                                                                                 | 1                                  | One-to-many communication; uses raw algorithmic bandwidth directly.                       |
| alltoall,<br>alltoall_on_stream,<br>alltoall_device,<br>fcollect,<br>fcollect_on_stream,<br>fcollect_device,<br>reducescatter,<br>reducescatter_on_stream,<br>device_reducescatter | $\frac{(npes - 1)}{npes}$          | Many-to-many communication pattern; bandwidth scaled by number of communication partners. |
| Any other job name                                                                                                                                                                 | —                                  | Prints a warning and sets both algBw and busBw to 1.                                      |

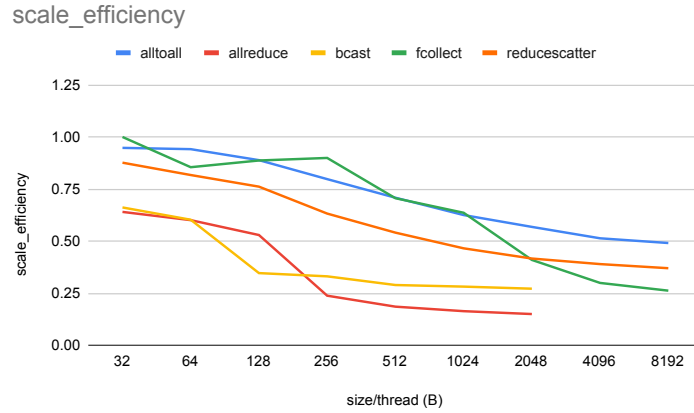
used by allreduce (e.g., segmented data exchange and linear reduction) align less naturally with NVSHMEM’s GPU execution model than the hierarchical or tree-based exchanges used in other collectives. More importantly, internal kernel configuration also make a difference. All current NVSHMEM collectives use a single CTA for execution, preventing effective utilization of all streaming multiprocessors (SMs). This is the primary reason why their maximal bandwidths are far below NVLink’s capacity.

Our test results show that the bandwidth curves saturate near 4 KB per thread under the default configuration (1 CTA  $\times$  256 TPB). Beyond this size, additional concurrency via more CTAs or larger TPBs would be required to exploit the remaining SM and interconnect capacity.

**Scalability** can be quantified by the metric

$$\text{scale\_efficiency} = \frac{\text{block\_bw}}{256 \times \text{thread\_bw}}$$

i.e. the ratio of block-scope bandwidth to thread-scope, and a value near 1.00 indicates ideal scaling from thread to block scope. As shown in Figure 4.11, for all collectives, *scale\_efficiency* declines monotonically as data size increases. At 32 B per thread, block-scope parallelism (256 threads) can reach as high as 256x theoretical speedup (or  $\sim 180x$  for bcast and allreduce) or 100% *scale\_efficiency*. But as message size grows to 8 KB per thread, the



**Figure 4.11:** Block-scaled collectives start out nearly perfectly efficient; at 32 B/thread, block bandwidth is  $180\text{--}256\times$  the thread baseline. However, every primitive's *scale\_efficiency* steadily declines with message size, dropping toward 30-40% (approx.  $100\times$  speedup) by 8 KB/thread as interconnect saturation and sync costs erase most of the thread-cooperation benefit.

speedup gradually converges toward  $\sim 100\times$ , and the *scale\_efficiency* falls to 30-40%. This trend indicates that fine-grained collectives benefit substantially from thread cooperation at small sizes, but the advantage diminishes as bandwidth becomes limited by interconnect saturation and synchronization overhead.

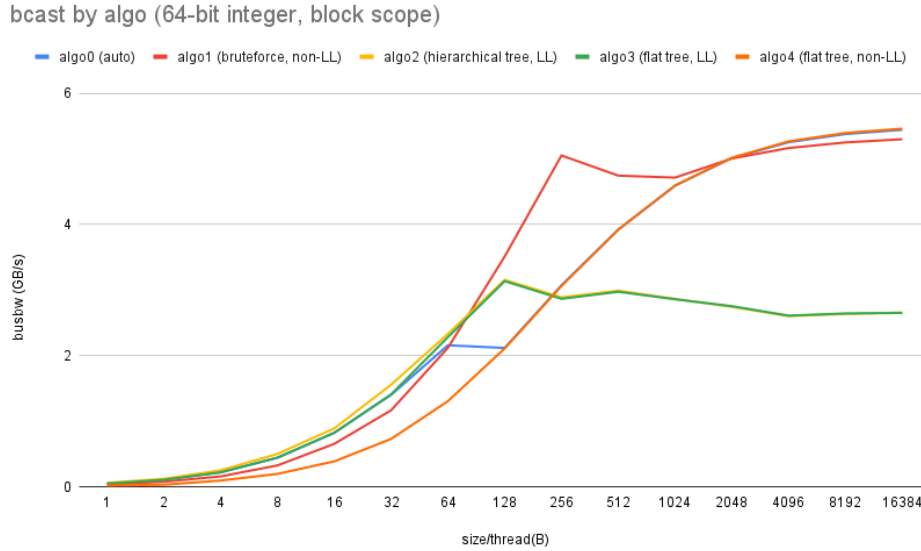
**Algorithm-level behavior.** NVSHMEM implements several algorithmic variants for collective operations, with bcast being the most illustrative example. As explained before, it supports four broadcast algorithms:

- Bruteforce all-push,
- LL topology-aware tree,
- LL flat tree, and
- Non-LL flat tree.

Figure 4.12 compares these implementations. At small message sizes (128 B per thread), LL-based algorithms achieve slightly higher performance due to reduced startup latency. However, LL methods allocate half of their effective bandwidth for control and synchronization signals, leaving less bandwidth for payload transfer. Consequently, LL algorithms plateau earlier and achieve roughly half the peak bandwidth of non-LL variants. Beyond 128 B per thread, the non-LL algorithms outperform their LL counterparts, sustaining higher throughput as data size increases. The bruteforce all-push strategy shows the steepest bandwidth ramp-up but reaches its saturation point around 256 B per thread. At larger message sizes, the non-LL flat tree

## 4.1. Simple Benchmarking

algorithm dominates, achieving both higher maximum bandwidth and more stable scaling.



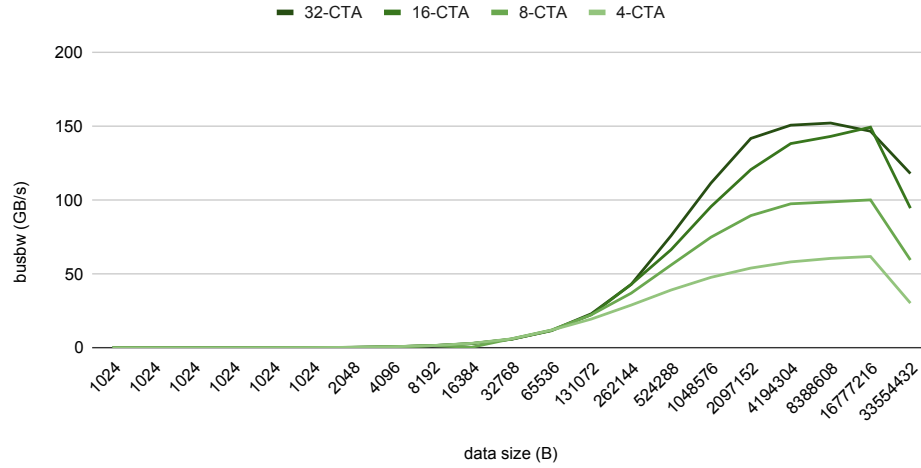
**Figure 4.12:** The bandwidth for each bcast algorithm as a function of size per thread. LL algorithms provide somewhat reduced latency, resulting in improved bandwidth for smaller data sizes. Nevertheless, the peak bandwidth for LL algorithms is only half that of their non-LL counterparts due to inherent overheads associated with the LL protocol. Within the non-LL category, the bruteforce algorithm accelerates the quickest, yet its peak bandwidth is similar to that of the non-LL flat tree.

**Multi-CTA.** Multi-CTA substantially unlocks NVLink throughput for device-initiated all-to-all. As shown in Figure 4.13, moving from few CTAs to 16 CTAs raises the peak bus bandwidth from 60–100 GB/s (4–8 CTAs) to roughly 150 GB/s. Concretely, at 16 MiB per-peer payload, bus bandwidth rises from 61.8 GB/s (4 CTAs) to 100.1 GB/s (8 CTAs) to 149.3 GB/s (16 CTAs), approaching the NVLink roofline. With 32 CTAs, the peak does not increase further: it reaches 150–152 GB/s (at 4–8 MiB), i.e., effectively at the NVLink limit. Gains become sublinear as CTAs increase once the bandwidth nears the NVLink/NVSwitch saturation point. For several sizes, 32 CTAs slightly underperform 16 CTAs because additional concurrency adds contention without raising the physical ceiling.

Note that if we fix the data size to 4 KB/thread, the bandwidth drops from 16 CTAs to 32 CTAs, for about 20%. We have discovered that this is due to the performance drop on too large message size. For 32 CTAs (and 256 TPB), 4KB / thread amounts to 32MB total data size, an inflection point of bandwidth: whatever the number of CTA we use between 1 and 32, their bandwidths drop sharply when the total data size gets to 32 MB, as shown in Figure 4.13.

alltoall bw by CTA and message size

TPB=256



**Figure 4.13:** All-to-all bus bandwidth vs message size at TPB=256 for 4, 8, 16, and 32 CTAs: increasing CTA count boosts throughput up to ~150 GB/s, with most gains realized by 16 CTAs (approaching the NVLink/NVSwitch roofline), while 32 CTAs provides little additional peak and can dip at some sizes as contention offsets added concurrency.

Recall that, as previously presented in Figure 4.7, in an internode scenario, end-to-end latency grows approximately linearly with the total message size and is largely insensitive to the GPU thread scope (thread/warp/block). This contrasts with intra-node NVLink results, where latency tracks per-thread payload because many requests can be serviced in parallel on the device interconnect and caches. In the inter-node case, transfers serialize through the GPU-PCIe/NVLink-NIC pipeline and the fabric, so the dominant term is bytes divided by effective injection bandwidth; accordingly, both puts and gets exhibit similar slopes, as they traverse the same datapath. This explains the throughput regressions seen in device-initiated all-to-all at multi-MiB sizes here. In this intranode scenario, NVLink/NVSwitch allows high concurrency, yet long destination-local bursts still occupy fabric queues and remote L2/DRAM for extended periods, creating transient hotspots, credit depletion, and head-of-line blocking across peers. Increasing the number of CTAs adds parallel flows and can partially mask the effect, but it cannot overcome the fundamental serialization limit of the inter-node path. In practice, fine-grained chunking (e.g., 64–512 KiB) with round-robin scheduling across peers and vectorized, aligned accesses sustains higher utilization at scale by shortening burst lengths and improving fairness. Apart from that, a side note: if we look at the maximal bw instead, then its maximal bw holds at the same level as 16 CTAs.

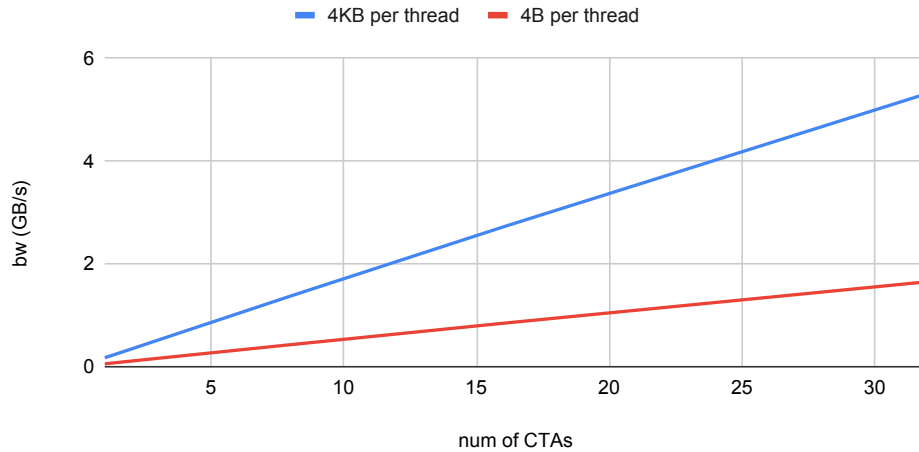


Allreduce shows a different scaling envelope. Increasing CTAs yields near-linear scaling up to CTA = 256 with  $\sim 25$  GB/s, which is far from saturating NVLink. Because the kernel is compute/latency bound rather than link-bound in this range, additional CTAs continue to help, unlike all-to-all where the interconnect becomes the limiter near  $\sim 150$  GB/s. See Figure 4.14.

**TPB-sweep.** We also experimented by keeping the data size per thread and the number of CTAs constant while sweeping the number of threads per CTA (or threads per block, TPB) between 32 and 1024. This led us to identify a notable pattern (Figure 4.15). With 4KB of data per thread and CTA=32, the bandwidth of allreduce is at its peak when TPB ranges from 224 to 288 but reduces by a quarter when TPB either reaches 384 or more, or is as low as 32. We believe this behavior is due to SM occupancy: at low TPB values, the 32 CTAs do not make full use of SM resources. When TPB is between 224 and 288, the total thread count maximizes SM utilization. Conversely, too high TPB reduces the number of CTAs that can be executed on a single SM, thus lowering parallel efficiency. We confirmed this by utilizing `cudaOccupancyMaxActiveBlocksPerMultiprocessor()` to check the number of CTAs accommodated by one SM. The function indicates that at TPB values of 224 to 288, each SM is capable of supporting 3 CTAs.

#### allreduce latency by CTA

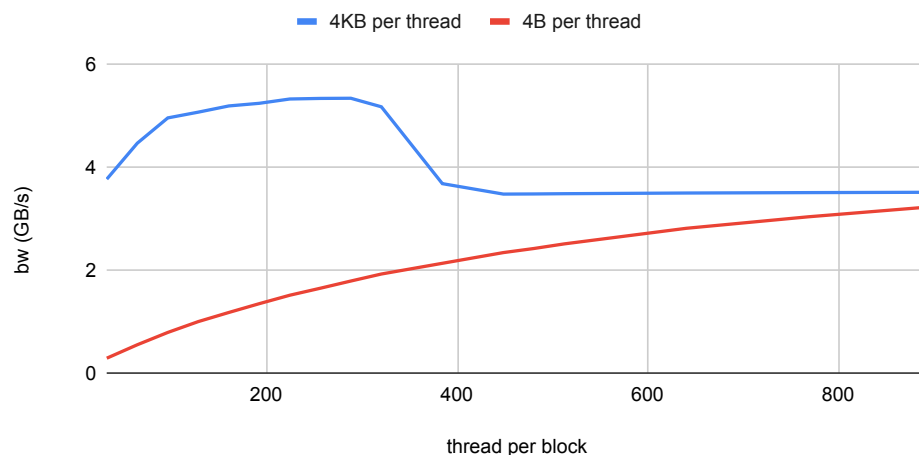
data size proportionate to thread number (TPB \* CTA). 256 TPB. int64 sum allreduce



**Figure 4.14:** Constant linear scaling of allreduce's bandwidth relative to the number of CTAs.

## allreduce latency by TPB

data size proportionate to thread number (TPB \* CTA). 32 CTA. int64 sum allreduce



**Figure 4.15:** allreduce’s bandwidth in relation to TPB. Under a small data size setting, bandwidth grows constantly throughout the whole TPB range. Under large data sizes, the bandwidth follows a unimodal trend: peaking at approximately 5 GB/s when TPB is between 224 and 288. For TPB values greater than 384, the bandwidth levels off at approximately 3.5 GB/s. Similarly, when the minimum TPB is 32, the bandwidth is also 3.5 GB/s.

## Inter-node

In the inter-node setup, bandwidth drops when scaling from 4→8 and 8→16 nodes, but levels off from 16→32 nodes. This aligns with the cluster topology: each chassis holds 7 blades with 2 nodes per blade. At 16 nodes and beyond, communication crosses chassis boundaries. The cross-chassis bisection (and associated PCIe/NVLink–NIC pipeline plus fabric hop count) becomes the dominant limiter, so doubling to 32 nodes does not further reduce per-rank throughput beyond the cross-chassis bottleneck.

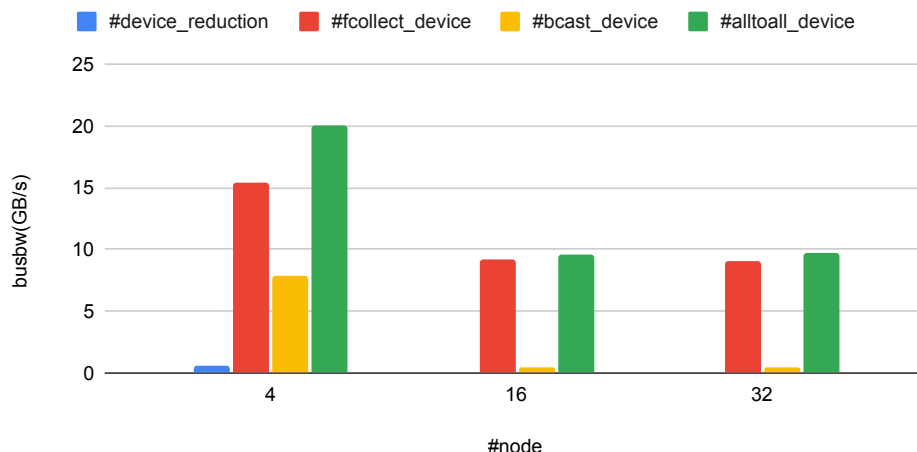
## 4.2 Performance Comparison with NCCL

In this section, we compare performance numbers from NVSHMEM v3.3.9 and NCCL v2.29 over collective communication operations. Experiments in this section are conducted on NVIDIA’s DGX node, using 4 H100 GPUs (w/ 80 GB HBM3 memory) per node, and CUDA 13.0, fully-connected via NVLink with an aggregate unidirectional bandwidth of 450 GB/s.

For NCCL, we use the NCCL-tests repo [[empty citation](#)]. This supports NCCL GIN, including the load-store accessible (LSA) implementation, the Multimem implementation, and their respective vectorization and unrolling

## Internode collective performance by #node

4 GPUs per node. CTA=1. TPB=256.



**Figure 4.16:** The maximum bandwidth achievable by each collective as a function of the number of nodes in an internode configuration. With 4 nodes, the maximum bandwidth for all these collectives remains below 20GB/s, bounded by the Cray Slingshot network. As the number of nodes increases to 16 or 32, all collectives exhibit substantial performance reductions, decreasing by at least half.

variants.

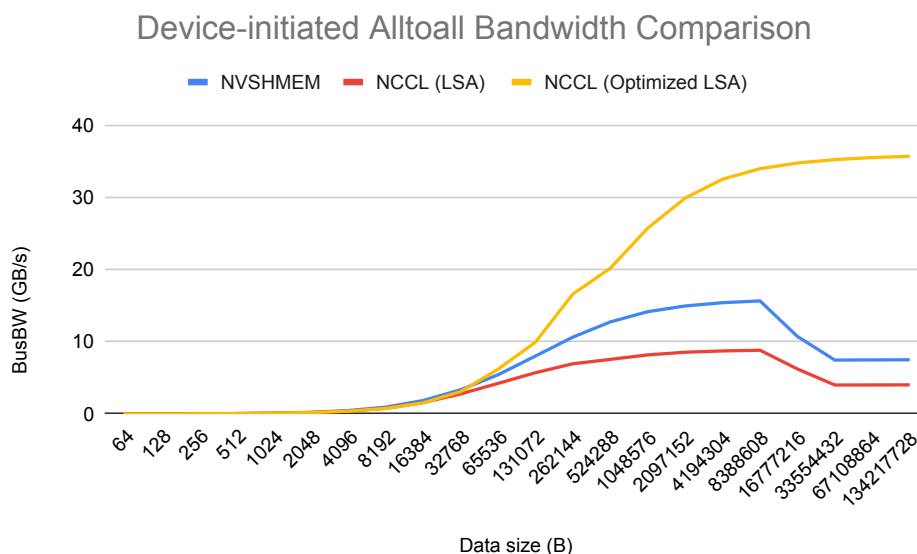
### 4.2.1 Collective perfctest

Under identical launch parameters, NVSHMEM's built-in collective APIs generally trail NCCL in peak throughput and efficiency.

Figure 4.17 presents a set of results for limited parallelism configuration. We limit NCCL to 1 CTA, the same parameter as the current internal implementation of NVSHMEM's collectives, and compare their performance of the all-to-all collective. The result shows that NCCL, with optimized LSA based on vectorization and unrolling, achieves higher bus bandwidth and sustained efficiency than NVSHMEM under this limitation. Note that this result does not represent NCCL's peak bandwidth in real scenarios. Figure 4.18 compares their performance of the allreduce collective.

NCCL benefits from *multi-channel parallelism* (while NVSHMEM's built-in collectives are single-CTA), mature and topology-aware algorithms (ring/tree variants), deep pipelining and chunking, and link-protocol specializations (e.g., LL/LL128) that sustain high utilization across NVLink/NVSwitch and NICs.

That said, NVSHMEM exposes lower-level, device-initiated operations that



**Figure 4.17:** Device-side all-to-all bandwidth vs message size, a controlled microbenchmark under 1 CTA and 512 TPBs. Here, the “optimized LSA” includes vectorization and unrolling. The result shows that NCCL can achieve higher peak bus bandwidth and sustained efficiency than NVSHMEM even under limited parallelism.

many applications use to craft collectives or fused communication kernels tailored to their computation/communication overlap and data layout. With careful implementation, including multi-CTA kernels, 64–512 KiB chunking per peer, round-robin scheduling, coalesced vector accesses, and alignment to cacheline boundaries, NVSHMEM can close a substantial portion of the gap for specific patterns, while enabling fine-grained progress that NCCL does not target.

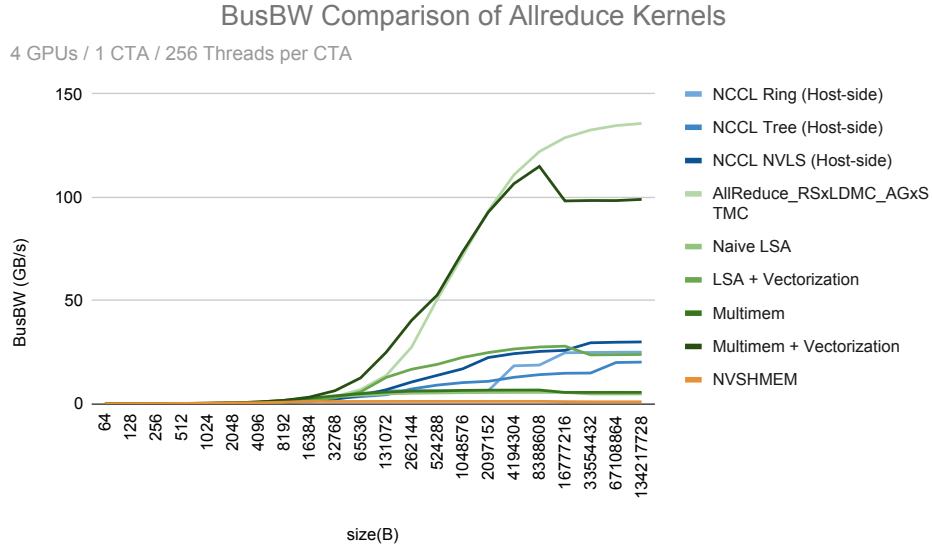
#### Performance takeaway.

- Use NCCL for bulk, throughput-oriented collectives on stable topologies.
- Use NVSHMEM primitives when you need device-initiated progress, fine-grained synchronization, or tight overlap with computation; adopt the chunking and scheduling strategies above to avoid burst-induced congestion and to approach NVLink ceilings intra-node.

#### 4.2.2 Jacobi

Jacobi iteration is a simple, widely used stationary method for solving large sparse linear systems by repeatedly updating each unknown from the previous iterate and its neighbors; we apply Dirichlet boundary conditions on the left/right boundaries and periodic boundaries on the top/bottom, yielding

## 4.2. Performance Comparison with NCCL



**Figure 4.18:** Allreduce kernel bus bandwidth comparison (4 GPUs, 1 CTA, 256 TPB) shows NCCL ring/tree/NVLS and optimized custom variants reaching higher throughput than NVSHMEM's built-in collective, highlighting the benefit of multi-channel parallelism and algorithmic specialization.

fixed-value edges in the horizontal direction and wrap-around coupling in the vertical direction. Because each update depends only on the old values, the method is naturally parallel and maps cleanly to domain-decomposed stencil computations with nearest-neighbor exchanges. These properties make Jacobi a common proxy for communication-intensive workloads in HPC, and a useful microbenchmark for evaluating runtime and communication layers such as NVSHMEM.

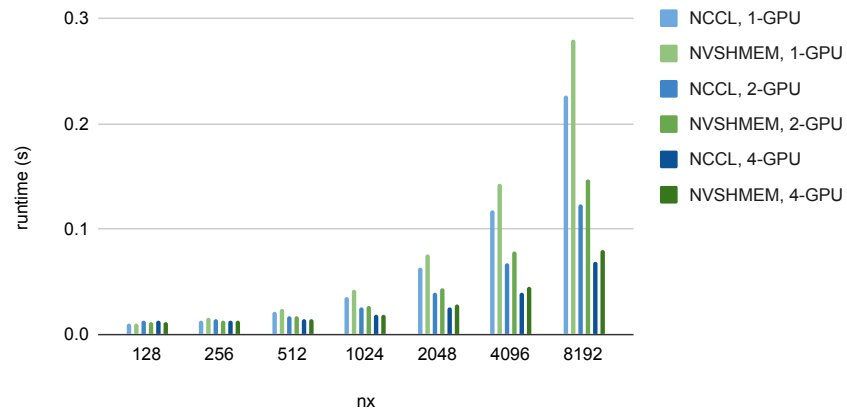
In this section, we run NVIDIA's GTC reference Jacobi solver implementation [37] based on NCCL and NVSHMEM respectively. For NVSHMEM, we turn on both optimizations set out in the repo's README: `-use_block_comm`, use block cooperative `nvshmemx_float_put_nbi_block` instead of single-thread `nvshmem_float_p` for communication; `-neighborhood_sync`, use custom neighbor only synchronization instead of collective `nvshmemx_barrier_all_on_stream`. We run a parameter sweep across node counts, GPUs per node, problem sizes (sweep `nx`, fix `ny=16384` and number of iterations = 300). The reported runtime is the average among 3 runs.

Our results show that NCCL Jacobi consistently outperforms NVSHMEM's. Nsight Systems analysis indicates that, for this Jacobi workload, NVSHMEM communication is slower than NCCL. Specifically, for a problem size of (16384,16384) over 1000 iterations, NCCL completes in about 0.4 seconds. Each iteration alternates between approximately 400 ms Jacobi kernels and

## 4.2. Performance Comparison with NCCL

### Jacobi kernel, 1-node

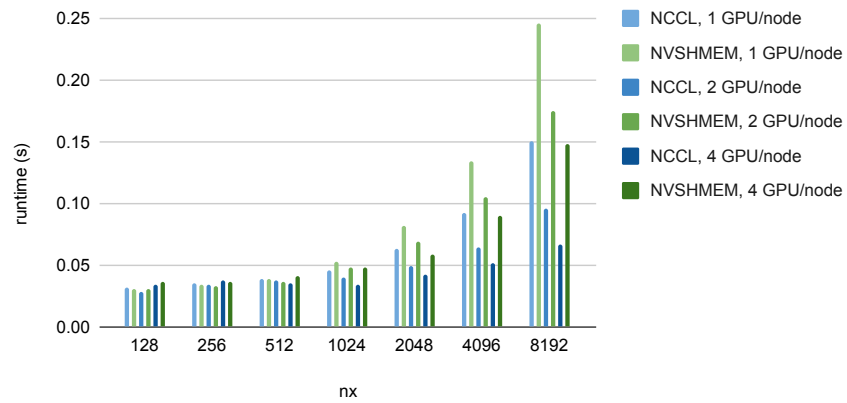
ny=16384. NVSHMEM options: -use\_block\_comm -neighborhood\_sync



(a)

### Jacobi kernel, 2-node

ny=16384. NVSHMEM options: -use\_block\_comm -neighborhood\_sync



(b)

**Figure 4.19:** Jacobi kernel runtime sweep (ny=16384, 300 iters) comparing NCCL vs NVSHMEM with -use\_block\_comm and -neighborhood\_sync: (a) 1-node and (b) 2-node results across nx and GPUs per node. NCCL constantly shows better runtime than NVSHMEM, except for very small nx.

10–20 ms NCCL communication phases (the `NCCL_Launch` NVTX range, which includes `ncclGroupEnd()` with completion semantics). Overall, the communication overhead is relatively small. While in the NVSHMEM implementation, all communication operations are embedded inside the Jacobi kernel. Aside from these communication steps, the remainder of the code is effectively identical to the NCCL version. Since both NCCL and NVSHMEM run the same number of iterations (with `cudaEventSynchronize` called exactly 1000 times), the additional kernel execution time can be attributed to communication overhead. For the (16384,16384) case, the Jacobi kernel execution time per iteration (as indicated by the kernel duration on the CUDA HW timeline) increases to approximately 490 ms. This suggests that each communication phase takes approximately 100 ms.

Based on private communication with NVIDIA, this behavior is attributed to a compiler issue (internal NVIDIA bug report) where the presence of a non-inlined device function negatively impacts kernel performance, even when the function is never invoked.

We also evaluated the effect of varying NCCL's `nx` parameter. For `nx = 1024`, communication takes about 10  $\mu$ s per iteration, while the Jacobi kernel takes approximately 30  $\mu$ s per iteration. For `nx = 65536`, communication takes about 20  $\mu$ s per iteration, and the Jacobi kernel takes approximately 1.5 ms per iteration. These results indicate that, as the problem size decreases, communication accounts for an increasingly larger fraction of the total execution time.

Following the multi-CTA approach we used for NVSHMEM collective communication operations, we attempted to parallelize the NVSHMEM Jacobi program with multi-CTA to improve performance. Unfortunately, in the NVSHMEM Jacobi implementation, a naive multi-CTA decomposition of halo exchanges using `nvshmemx_float_put_nbi_block` did not yield performance gains and could even regress versus the baseline kernel.

The root causes are twofold:

- On the compute side, switching to a degenerate  $1024 \times 1$  block shape inflated the number of CTAs, stressed instruction scheduling and increased global `atomicAdd` traffic for the L2 norm, thereby slowing down the main kernel enough to hide any communication benefit;
- On the communication side, sharding each boundary row into many short NVSHMEM puts fragmented messages, increased per-call overhead, and offered little incremental concurrency because NVSHMEM's progression engines do not scale linearly with the number of concurrent CTAs, reducing effective link utilization.

Additionally, the block-comm variant tended to initiate each halo put later (after computing a larger chunk), degrading compute–communication overlap

that the element-wise baseline enjoyed “for free.” We recovered performance by restoring the normal 2D kernel shape (e.g.,  $32 \times 32$ ) to preserve occupancy, cache locality, and low atomic pressure, and by reducing message fragmentation—preferably issuing a single, contiguous put per boundary row (or a small number of large segments) and, when possible, computing the top/bottom rows early on a separate stream so their puts overlap with interior computation. Across these configurations, the key design rules are: keep the compute kernel efficient first, minimize the number of NVSHMEM calls by sending longer contiguous segments, and structure the pipeline to maximize overlap; multi-CTA is only beneficial when it does not degrade these fundamentals.

Lastly, the NVSHMEM’s neighborhood-sync optimization only works under block comm kernel; if the normal kernel is used, it even degrades the performance.



---

# Case Study: DeepEP

---

Under this chapter, we analyze how NVSHMEM and its peer libraries like NCCL are used in user applications or user-level libraries. Let’s see how they perform respectively under different quantitative conditions, including message size, number of neighbors, and iteration length.

DeepEP [36] is a communication library for Mixture-of-Experts (MoE) workloads that targets expert parallelism, where tokens are dispatched to, and combined from, distributed experts via high-throughput all-to-all GPU kernels. It provides both bandwidth-optimized dispatch/combine paths and latency-sensitive kernels for inference, and it supports low-precision data movement (e.g., FP8) to reduce communication cost.

DeepEP depends on NVSHMEM for its inter-node communication features: the build and runtime are wired to an NVSHMEM installation, and enabling NVSHMEM activates the internode and low-latency capabilities that rely on GPU-initiated communication over RDMA-capable fabrics, including InfiniBand and RoCE. This design ties DeepEP’s expert-parallel data exchange directly to NVSHMEM’s GPU-centric one-sided communication model, making NVSHMEM a key substrate for scaling MoE dispatch/combine across nodes.

On a side note, for internode data transfers, DeepEP uses the IBGDA APIs `nvshmemi_ibgda_*`. For simplicity, we refer to these as “IB” throughout the text; however, these APIs are not limited to InfiniBand; RoCE fabrics can also support IBGDA APIs.

**Initialization using UID mode.** In DeepEP, NVSHMEM’s initialization follows the UID model. Recall that, in NVSHMEM’s UID mode (stands for unique ID) of initialization, one PE calls `nvshmemx_get_uniqueid` to acquire a unique ID, and the user takes the responsibility to distribute this unique ID to all PEs. Here, first in Python, the `Buffer` class (`buffer.py`) gathers a root unique ID from rank 0 (or RDMA rank 0 in non-low-latency mode),

---

all-gathers it across the process group, and passes it into the C++ runtime. In C++, `internode::init` initializes NVSHMEM using `nvshmemx_init_attr` with `NVSHMEMX_INIT_WITH_UNIQUEID`; then, an initialization barrier (`nvshmem_barrier_all`) ensures all ranks are synchronized before use.

**Memory management and symmetric allocation.** During DeepEP initialization process and after NVSHMEM is initialized, DeepEP allocates (`internode::alloc`) its RDMA buffer using NVSHMEM's symmetric heap via `nvshmem_align`. This ensures that each rank holds a symmetric address space suitable for direct RDMA operations. This RDMA buffer is then passed into the internode CUDA kernels (e.g. `notify_dispatch`) as the RDMA staging area for token counts, expert counts, prefixes, and data buffers. Corresponding cleanup is performed in `internode::free` and `internode::finalize`, which destroy any NVSHMEM teams and call `nvshmem_finalize`. This lifecycle is orchestrated in `Buffer::sync` and `Buffer::destroy` to ensure NVSHMEM memory and state are released correctly when the buffer is torn down.

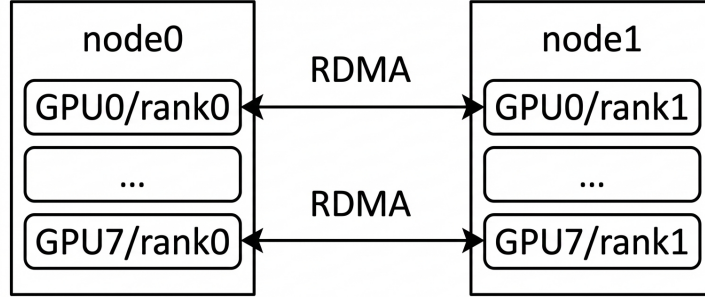
**Topology, and topology-aware teaming.** DeepEP assumes a two-level topology:

- NVLink domain size = `NUM_MAX_NVL_PEERS` (8) per node.
- RDMA rank = node-level group. `rdma_rank` = `rank / NUM_MAX_NVL_PEERS`, `nvl_rank` = `rank % NUM_MAX_NVL_PEERS`. Also, `num_rdma_ranks` = `num_ranks / NUM_MAX_NVL_PEERS`.

Internode kernels treat communication as: RDMA between nodes, then NVLink distribution inside nodes. They use `NUM_MAX_NVL_PEERS` to translate between RDMA rank and NVLink rank.

Note that during initialization (`internode::init`), in low-latency mode (where the low-latency kernels and bandwidth kernels are both callable), if the number of PEs exceeds `NUM_MAX_NVL_PEERS`, DeepEP splits the world team into sub-teams aligned with NVLink groups. This uses `nvshmem_team_split_strided` to create a `cpu_rdma_team`. Subsequent synchronization (`nvshmem_sync_with_same_gpu_idx`) calls `nvshmem_sync` on the RDMA team in low-latency mode or `nvshmem_sync_all` otherwise. However, if the number of PEs does not exceed `NUM_MAX_NVL_PEERS`, DeepEP will only call low-latency kernels and do not call bandwidth kernels, and will not initialize `cpu_rdma_team`.

**High-throughput dispatch-combine** (`internode.cu`). Next, we'll analyze the internode alltoall implementation in more detail; we skip the intranode implementation since it does not concern NVSHMEM. This includes the usual codepath that aims at maximal bandwidth, and also a set of low-latency kernels with pure RDMA to minimize delays. In this paragraph, we'll first analyze the bandwidth kernels.



**Figure 5.1:** In DeepEP, internode tokens are only sent between GPUs with the same in-node index, requiring only a linear number of internode connections.

The alltoall process, called dispatch-combine in DeepEP, is divided into a dispatch phase and a combine phase. For the dispatch phase, a meta-data communication procedure `notify_dispatch` runs first. It uses one SM (`blockIdx.x == 0`) to all-gather per-rank and per-expert token counts into a shared buffer, using `nvshmemi_ibgda_put_nbi_warp` for one-sided RDMA writes and `nvshmemi_ibgda_quiet` for completion; then computes prefix sums that define the rank-wise send offsets and the per-expert receive sizes, using a manual for-loop (but with a comment “TODO: may use NVSHMEM reduction”). Afterwards, it does a hierarchical synchronization: first, the 32th thread in the block calls a `nvshmem_sync_with_same_gpu_idx`, an RDMA-level sync; second, for NVLink-local sync, every thread in the block calls `barrier_block`, a custom block-level barrier implementation. The other SMs (`blocks ≥ 1`) concurrently compute a `channel_prefix_matrix` by scanning the `is_token_in_rank` mask (a passed-in parameter) per destination rank and taking a prefix sum over channels, which partitions each rank’s traffic into per-channel chunks. In this way, each GPU learns how many tokens it will receive and has prefix matrices for later data movement.

The custom barrier `barrier_block` is implemented as follows. Every thread uses system-scope atomic operations to add 1024 to its own signal location in the barrier signal array, and subtract 1024 from other ranks’ corresponding locations; when all signal values become  $\leq 0$ , all ranks have completed and the barrier is passed through. This custom barrier offers a low-latency and scalable synchronization mechanism, adding to DeepEP’s performance.

Then, in the actual dispatch procedure, for RDMA stage, GPUs use `nvshmemi_ibgda_put_nbi_warp` to write remote payloads into the RDMA heap of destination RDMA ranks; and for NVLink stage, data is forwarded within each node from RDMA-receiver GPUs to other local GPUs via NVLink buffers (using the NVLink buffer arrays, not NVSHMEM). DeepEP employs the warp specialization technique [38] to split the work of IB sending, IB-to-NVLink forwarding, and NVLink receiving among different warps: warps

---

on even-numbered SMs are responsible for IB-to-NVLink forwarding; among odd-numbered SMs, warps with  $ID \leq kNumDispatchRDMA\text{SenderWarps}$  are RDMA senders and the other warps are NVLink receivers.

For the combine phase, the combine kernel also features sender-receiver role separation for this forwarding: it assigns even-indexed SMs to forwarder duties, while NVLink receivers live in odd-indexed SMs. Communication is organized into logical channels; every sender-receiver pair is responsible for a channel, enabling pipelined progress with limited contention across channels. The kernel utilizes CUDA IPC APIs `ld_nc_global` and `st_na_global`, and uses loop unrolling. After copying a channel’s payload into the NVLink IPC buffer, the sender uses `st.release.sys.global.s32` to publish the channel tail index, so receivers can safely observe progress by reading the tail with `ld.acquire.sys.global`. This implements a per-channel ring buffer with head/tail synchronization: senders enqueue token payloads (data, source index, and `topk_weights`), and receivers dequeue and reduce them into the final combined output.

In short, the high-throughput internode algorithm is “RDMA first, NVLink second” with explicit metadata exchange and two-phase movement. However, this design is only adopted in bandwidth kernels; for low-latency kernels, DeepEP utilizes RDMA for both intranode and internode communication, to avoid the overheads of RDMA-NVLink message forwarding.

**Low-latency dispatch-combine** (`internode_ll.cu`). This is a different algorithmic path optimized for small batch/latency-sensitive inference. During dispatch and combine, each rank writes its token payloads and related metadata directly into the destination’s symmetric RDMA slots using IBGDA primitives `nvshmemi_ibgda_put_nbi_warp` or `nvshmemi_ibgda_rma_p`, and then uses atomic counters to advertise availability. Receivers observe these counters and consume the data without an intermediate NVLink forwarder stage, which reduces latency at small batch sizes.

This path also implements its custom low-latency barrier (`internode_ll::barrier`) based on remote atomic updates: for NVLink P2P-reachable sync buffers (direct pointer `nvshmemi_get_p2p_ptr` not NULL), it calls a direct store operation `st.release.sys.global.s32` on the direct pointer; otherwise, it calls `nvshmemi_ibgda_rma_p` for a put over IBGDA. It also provides optional masking on timeout ranks, allowing the algorithm to continue without blocking on a slow or dead rank.

**Algorithm network-topology co-design.** Finally, a crucial performance optimization is the co-design of MoE algorithm and the cluster network topology. As DeepEP is designed for clusters that utilize NVLink for intranode communications and IB for internode communications, and NVLink has a 3.2x bandwidth to IB (160 vs 50 GB/s), the DeepSeek MoE gating algorithm implementation is co-designed with the network topology of the cluster to

---

dispatch every token to at most 4 nodes, to adapt to this performance gap [39]. It sends every internode token only to the GPUs with the same in-node index on its target nodes, which in turn forwards the token to the GPUs that host its target experts via NVLink; in this one-on-one approach, only a linear number of internode connections have to be maintained. Consequently, DeepEP can save on networking costs and enable full overlap of IB and NVLink. This efficient design saves SM resources dedicated to communication; as few as 20 SMs are sufficient to fully utilize the bandwidths of IB and NVLink.

**Performance considerations on Slingshot-based clusters.** Our CSCS cluster is based on Cray Slingshot and only provides soft-RoCE support. As a result, the observed performance is unlikely to fully capture DeepEP’s characteristics in production environments with native IBGDA support. We therefore defer a thorough performance evaluation of DeepEP to future work.

---

# Discussion

---

### 6.1 Potential improvements & optimizations

Despite NVSHMEM’s transformative potential, the adoption of NVSHMEM is not without challenges. It introduces significant complexity into the software stack.

**Lack of Global Coherence.** The relaxed memory model requires expert knowledge to avoid race conditions. Developers must manually insert fences and barriers, which can be error-prone and difficult to debug. A missing fence can lead to non-deterministic data corruption that only appears at scale.

**Static Library Limitation.** Historically, CUDA’s device-code model constrained modular builds: before CUDA 5.0, device symbols could not be separately compiled and linked across translation units, effectively forcing whole-program device compilation. Even after separate compilation arrived, the device linker has only supported static libraries and ignores dynamic/shared libraries, so device linking cannot span shared-library boundaries. As a result, CUDA-using projects (including NVSHMEM, which provides device-side APIs and embeds GPU code) have often been packaged as static libraries or as a single shared library that performs the device-link step internally. Recent toolchain advances (e.g., mature separate compilation and nvJitLink/LTO support) have mitigated (but not eliminated) these build-system constraints for large modular applications such as Python bindings and large frameworks.

**Hardware Dependency.** While OpenSHMEM is portable, NVSHMEM’s performance relies heavily on NVIDIA-specific hardware features (NVLink, IBGDA). Code heavily optimized with `nvshmemx` extensions (like warp-level puts) is not performance-portable to AMD or Intel GPUs without significant refactoring to ROC\_SHMEM or OneAPI, creating vendor lock-in.

---

# Conclusion

---

NVSHMEM represents a shift towards a data-centric paradigm where the GPU acts as a fully autonomous agent, removing the OS and CPU from the critical path of fine-grained communication. In this paper, we examined NVSHMEM’s architecture and source code implementation in depth, and discussed its key components including symmetric memory, device-initiated communication, and one-sided semantics. Our benchmarks reveal distinct performance characteristics for NVSHMEM’s different granularity communication primitives under different configurations. Moreover, we analyzed how NVSHMEM is integrated into modern AI workloads like DeepEP. Ultimately, these insights provide a roadmap for developers to identify bottlenecks and for architects to make informed design decisions when building the next generation of communication libraries.

---

## Acknowledgements

---

We thank Akhil Langer, Jiri Kraus, Benjamin Glick, Craig Belusar, and Seth Howell from NVIDIA for valuable discussions and guidance. This includes but is not limited to clarifications on team semantics and collective operations, and diagnosing a compiler-related performance issue in Jacobi kernel benchmarking.

We thank Yue Yu from EPFL for raising insightful questions that helped improve this work.



---

## Bibliography

---

- [1] C.-H. Hsu and N. Imam, “Assessment of NVSHMEM for High Performance Computing,” *International Journal of Networking and Computing*, vol. 11, no. 1, pp. 78–101, 2021. doi: [10.15803/ijnc.11.1\\_78](https://doi.org/10.15803/ijnc.11.1_78).
- [2] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard, Version 5.0*, <https://www.mpi-forum.org/docs/mpi-5.0/mpi50-report.pdf>, 2025.
- [3] D. Unat *et al.*, *The Landscape of GPU-Centric Communication*, 2024. arXiv: [2409.09874](https://arxiv.org/abs/2409.09874) [cs.DC]. [Online]. Available: <https://arxiv.org/abs/2409.09874>.
- [4] *GPUDirect RDMA documentation*, NVIDIA, 2025. [Online]. Available: <https://docs.nvidia.com/cuda/gpudirect-rdma/>.
- [5] N. Maruyama *et al.*, *Scaling Scientific Computing with NVSHMEM*, <https://developer.nvidia.com/blog/scaling-scientific-computing-with-nvshmem/>, NVIDIA Technical Blog, 2020.
- [6] G. Calin, E. Derevenetc, R. Majumdar, and R. Meyer, “A Theory of Partitioned Global Address Spaces,” 2013.
- [7] CLaSS Group at Lawrence Berkeley National Laboratory, *UPC++ Wiki*, <https://bitbucket.org/berkeleylab/upcxx/wiki/Home>.
- [8] D. Bonachea and P. H. Hargrove, “GASNet-EX: A High-Performance, Portable Communication Library for Exascale,” in *Languages and Compilers for Parallel Computing (LCPC’18)*, Lawrence Berkeley National Laboratory Technical Report LBNL–2001174, Springer, 2018. doi: [10.25344/S4QP4W](https://doi.org/10.25344/S4QP4W).
- [9] Advanced Micro Devices, Inc., *rocSHMEM — ROCm OpenSHMEM GPU library*, <https://rocm.docs.amd.com/projects/rocSHMEM/>.

- 
- [10] R. Keller and R. L. Graham, "Characteristics of the Unexpected Message Queue of MPI Applications," in *Recent Advances in the Message Passing Interface*, R. Keller, E. Gabriel, M. Resch, and J. Dongarra, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 179–188, ISBN: 978-3-642-15646-5.
- [11] M. Doijade, A. Alekseenko, A. Brown, A. Gray, and S. Páll, "Re-designing GROMACS Halo Exchange: Improving Strong Scaling with GPU-initiated NVSHMEM," in *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC Workshops '25, ACM, Nov. 2025, pp. 1314–1329. doi: [10.1145/3731599.3767508](https://doi.org/10.1145/3731599.3767508). [Online]. Available: <http://dx.doi.org/10.1145/3731599.3767508>.
- [12] S. P. Pak Markthub Jim Dinan and S. Howell, *Improving Network Performance of HPC Systems Using NVIDIA Magnum IO NVSHMEM and GPUDirect Async*, [https://developer.nvidia.com/blog/improving-network-performance-of-hpc-systems-using-nvidia-magnum-io-nvshmem-and-gpudirect-async/#infiniband\\_gpudirect\\_async](https://developer.nvidia.com/blog/improving-network-performance-of-hpc-systems-using-nvidia-magnum-io-nvshmem-and-gpudirect-async/#infiniband_gpudirect_async), Section: InfiniBand GPUDirect Async, 2022.
- [13] S. Potluri, A. Goswami, D. Rossetti, C. Newburn, M. G. Venkata, and N. Imam, "GPU-Centric Communication on NVIDIA GPU Clusters with InfiniBand: A Case Study with OpenSHMEM," in *2017 IEEE 24th International Conference on High Performance Computing (HiPC)*, 2017, pp. 253–262. doi: [10.1109/HiPC.2017.00037](https://doi.org/10.1109/HiPC.2017.00037). [Online]. Available: <https://www.osti.gov/servlets/purl/1427708>.
- [14] Alan Gray, Gaurav Garg, Mahesh Doijade (NVIDIA), and Szilárd Páll (KTH/PDC), *Cutting-Edge CUDA Technologies for Molecular Dynamics and Beyond*, NVIDIA GTC, 2023. [Online]. Available: <https://www.nvidia.com/en-us/on-demand/session/gtcspring23-s51110/>.
- [15] K. Hamidouche *et al.*, *GPU-Initiated Networking for NCCL*, 2025. arXiv: [2511.15076](https://arxiv.org/abs/2511.15076) [cs.DC]. [Online]. Available: <https://arxiv.org/abs/2511.15076>.
- [16] Paul H. Hargrove, Dan Bonachea, Colin A. MacLean, Daniel Waters, "GASNet-EX Memory Kinds: Support for Device Memory in PGAS Programming Models (Extended Poster Abstract)," in *The International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)*, 2021. doi: [10.25344/S4P306](https://doi.org/10.25344/S4P306). [Online]. Available: [https://sc21.supercomputing.org/proceedings/tech\\_poster/poster\\_files/rpost109s2-file3.pdf](https://sc21.supercomputing.org/proceedings/tech_poster/poster_files/rpost109s2-file3.pdf).
- [17] Paul H. Hargrove, Dan Bonachea, "GASNet-EX RMA Communication Performance on Recent Supercomputing Systems," in *5th Annual Parallel Applications Workshop, Alternatives To MPI+X (PAW-ATM'22)*, 2022.

- DOI: [10.25344/S40C7D](https://doi.org/10.25344/S40C7D). [Online]. Available: <https://escholarship.org/uc/item/3q8590fz>.
- [18] A. Brooks, P. Marshall, D. Ozog, M. W.-u. Rahman, L. Stewart, and R. Tom, *Intel(R) SHMEM: GPU-initiated OpenSHMEM using SYCL*, 2024. arXiv: [2409.20476](https://arxiv.org/abs/2409.20476) [cs.DC]. [Online]. Available: <https://arxiv.org/abs/2409.20476>.
- [19] C. R. Trott *et al.*, “Kokkos 3: Programming Model Extensions for the Exascale Era,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 4, pp. 805–817, 2022. DOI: [10.1109/TPDS.2021.3097283](https://doi.org/10.1109/TPDS.2021.3097283).
- [20] D. A. Beckingsale, J. Burmark, R. Hornung, H. Jones, W. Killian, A. J. Kunen, O. Pearce, P. Robinson, B. S. Ryujin, T. R. W. Scogland, “RAJA: Portable Performance for Large-Scale Scientific Applications,” in *2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*, 2019. [Online]. Available: <https://conferences.computer.org/sc19w/2019/#!/toc/14>.
- [21] C.-H. Hsu, N. Imam, A. Langer, S. Potluri, and C. J. Newburn, “An Initial Assessment of NVSHMEM for High Performance Computing,” in *2020 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2020, pp. 1–10. DOI: [10.1109/IPDPSW50202.2020.00104](https://doi.org/10.1109/IPDPSW50202.2020.00104).
- [22] L. W. Yifu Wang Horace He, *PyTorch SymmetricMemory: Harnessing NVLink Programmability with Ease*, <https://dev-discuss.pytorch.org/t/pytorch-symmetricmemory-harnessing-nvlink-programmability-with-ease/2798>, 2025.
- [23] A. Langer, S. Howell, A. Goel, P. Markthub, H. Petty, and F. Oh, *Enhancing Application Portability and Compatibility across New Platforms Using NVIDIA Magnum IO NVSHMEM 3.0*, Section: CPU-assisted InfiniBand GPU Direct Async, 2024. [Online]. Available: [https://developer.nvidia.com/blog/enhancing-application-portability-and-compatibility-across-new-platforms-using-nvidia-magnum-io-nvshmem-3-0/#cpu-assisted\\_infiniband\\_gpu\\_direct\\_async%C2%A0](https://developer.nvidia.com/blog/enhancing-application-portability-and-compatibility-across-new-platforms-using-nvidia-magnum-io-nvshmem-3-0/#cpu-assisted_infiniband_gpu_direct_async%C2%A0).
- [24] *InfiniBand Architecture Specification*, InfiniBand Trade Association, 2025.
- [25] NVIDIA, *NVSHMEM and the CUDA Model — NVSHMEM Documentation*, Section: The CUDA Execution Model, 2025. [Online]. Available: <https://docs.nvidia.com/nvshmem/api/cuda-interactions.html#work-submission-in-cuda> (visited on 11/22/2025).
- [26] NVIDIA, *NVSHMEM APIs*, Documentation accessed at multiple times during development; all references verified against version 3.5.19 prior to submission., 2025. [Online]. Available: <https://docs.nvidia.com/nvshmem/api/api.html> (visited on 11/22/2025).

- 
- [27] NVSHMEM Examples, Version 3.5.19; documentation is continuously updated, 2025. [Online]. Available: <https://docs.nvidia.com/nvshmem/api/examples.html> (visited on 11/22/2025).
- [28] P. Shamis *et al.*, “UCX: an open source framework for HPC network APIs and beyond,” in *2015 IEEE 23rd Annual Symposium on High-Performance Interconnects*, IEEE, 2015, pp. 40–43.
- [29] Libfabric, <https://ofiwg.github.io/libfabric/>.
- [30] C. Perry and N. Sakharnykh, *Introducing Low-Level GPU Virtual Memory Management*, 2020. [Online]. Available: <https://developer.nvidia.com/blog/introducing-low-level-gpu-virtual-memory-management/>.
- [31] S. Wang, *Glibc Malloc Principle*, 2021. [Online]. Available: [https://www.openeuler.org/en/blog/wangshuo/Glibc%20Malloc%20Principle/Glibc\\_Malloc\\_Principle](https://www.openeuler.org/en/blog/wangshuo/Glibc%20Malloc%20Principle/Glibc_Malloc_Principle).
- [32] *Multinode NVLink User Guide*, NVIDIA, 2025. [Online]. Available: <https://docs.nvidia.com/multi-node-nvlink-systems/mnnvl-user-guide/index.html>.
- [33] NVIDIA, *CUDA C++ Programming guide*, Section: Peer-to-Peer Memory Access. [Online]. Available: <https://docs.nvidia.com/cuda/archive/12.9.1/cuda-c-programming-guide/index.html#peer-to-peer-memory-access>.
- [34] *Device-Initiated Communication — NCCL 2.29.1 documentation*, 2026. [Online]. Available: <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/deviceapi.html> (visited on 02/05/2026).
- [35] Z. Hu *et al.*, *Demystifying NCCL: An In-depth Analysis of GPU Communication Protocols and Algorithms*, 2025. arXiv: 2507.04786 [cs.DC]. [Online]. Available: <https://arxiv.org/abs/2507.04786>.
- [36] C. Zhao *et al.*, *Deepep: An efficient expert-parallel communication library*, <https://github.com/deepseek-ai/DeepEP>, 2025.
- [37] NVIDIA, *Multi GPU Jacobi solver with different multi GPU programming models*, 2025. [Online]. Available: <https://github.com/NVIDIA/multi-gpu-programming-models/tree/master>.
- [38] M. Bauer, S. Treichler, and A. Aiken, “Singe: Leveraging warp specialization for high performance on gpus,” *SIGPLAN Not.*, vol. 49, no. 8, pp. 119–130, Feb. 2014, ISSN: 0362-1340. DOI: 10.1145/2692916.2555258. [Online]. Available: <https://doi.org/10.1145/2692916.2555258>.
- [39] DeepSeek-AI *et al.*, *Deepseek-v3 technical report*, 2025. arXiv: 2412.19437 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2412.19437>.

## Appendix A

---

# nvshmem\_init() Pseudocode

---

```
function nvshmem_init():
    provider = nvshmemi_bootstrap(flags, attr)
        // PMI/MPI/SHMEM/UID selection via env or API

    ↪ nvshmemi_bootstrap.get_pg_info(&npes,&mype,&npes_node,&mype_node)
    nvshmemi_env_options_init(&nvshmemi_options)

    nvshmemi_init_nvshmemi_state(state)
        // record rank info, default nic/ll128 flags
    nvshmemi_get_hostnames_and_hashes(provider)
        // build nvshmemi_host_hashes, check npes/node
    nvshmemi_detect_same_device(state)
        // allgather PCI IDs & UUIDs per PE

    nvshmemi_cuda_library_init()
        // dlopen CUDA symbols once
    nvshmemi_get_cucontext(state)
        // assumes user already cudaSetDevice; no auto pick
    nvshmemi_query_cuda_attributes()
        // stream memops, flush, VMM, GDRDMA flags

    nvshmemi_init_symmetric_heap(state, use_vmm?, heap_kind)
        // derive heap size / alignment from env+CUDA granularity
    state->heap_obj.reserve_heap()
        // cuMemAddressReserve / cudaMalloc with alignment
    nvshmemi_detect_nvls_support(state)
        // optional multicast capability test
```

---

```

state->heap_obj.setup_symmetric_heap():
    // done once now, again on demand if dynamic
    allgather peer heap base pointers
    register heap memory:
        export CUDA shareable handles (IPC or cuMem)
        register each chunk with every enabled transport
            -> obtain rkeys / mem handles
        allgather those per-transport handles across all PEs
        cache handles for later map/import

nvshmemi_transport_init(state)
    // instantiate P2P + selected remote transports
nvshmemi_build_transport_map(state)
    // for every PE/transport, call can_reach_peer()

nvshmemi_setup_stream_priorities(state)
    // create NVSHMEM internal stream
nvshmemi_setup_cuda_handles(state)
    // events, extra streams, proxy buffers
nvshmemi_setup_nvshmem_handles(state)
    // transport selectors per peer
nvshmemi_setup_connections(state)
    // per-transport connect_endpoints() (full mesh)
state->heap_obj.setup_symmetric_heap()
    // already called earlier; no second phase

nvshmemi_coll_common_cpu_init()
state->heap_obj.setup_symmetric_heap()
    // static path already registered; dynamic path maps peers
state->heap_obj.setup_symmetric_heap_after_connections()
    // import/mmap remote handles per transport

nvshmemi_setup_mps_if_needed()
    // shared-memory slab + CUDA events if MPG run
nvshmemi_proxy_init(state, proxy_level)
    // launch host proxy thread when transports need it

nvshmemi_init_device_state(state):
    // allocate device copies of runtime metadata
    cudaMalloc arrays for peer_heap_base_{p2p,remote}
    cudaMemcpy host-side heap_base lists to device arrays
    fill nvshmemi_device_state fields (heap base/size,
        transport selectors, flags)

```

---

```
        cudaMemcpy to (__constant__ nvshmemi_device_state_d)

nvshmemi_update_device_state()
    // push latest host-state image to registered device slots

nvshmemi_boot_handle.barrier()
    // final bootstrap barrier
nvshmemi_device_state.nvshmemi_is_nvshmem_initialized = 1
return SUCCESS
```



Eidgenössische Technische Hochschule Zürich  
Swiss Federal Institute of Technology Zurich

## Declaration of originality

The signed declaration of originality is a component of every semester paper, Bachelor's thesis, Master's thesis and any other degree paper undertaken during the course of studies, including the respective electronic versions.

Lecturers may also require a declaration of originality for other written papers compiled for their courses.

---

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor.

**Title of work** (in block letters):

**Authored by** (in block letters):

*For papers written by groups the names of all authors are required.*

**Name(s):**

**First name(s):**

|  |  |
|--|--|
|  |  |
|  |  |
|  |  |
|  |  |

With my signature I confirm that

- I have committed none of the forms of plagiarism described in the '[Citation etiquette](#)' information sheet.
- I have documented all methods, data and processes truthfully.
- I have not manipulated any data.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for plagiarism.

**Place, date**

**Signature(s)**

|  |  |
|--|--|
|  |  |
|  |  |
|  |  |
|  |  |

*For papers written by groups the names of all authors are required. Their signatures collectively guarantee the entire content of the written paper.*